

A Kubernetes Scheduler Plugin for Cluster-Wide Placement Optimisation

Henrik Daniel Christensen
TV 2 Danmark A/S
Denmark
hech@tv2.dk

Saverio Giallorenzo
Università di Bologna and INRIA
Italy and France
saverio.giallorenzo@unibo.it

Jacopo Mauro
University of Southern Denmark
Denmark
mauro@imada.sdu.dk

Abstract

The default scheduler of Kubernetes, the state-of-the-art container orchestrator, uses fast, local placement decisions. Unfortunately, this design leads to resource fragmentation, reduced cluster usage, and overprovisioning. External solvers can compute global placement plans, but enforcing these plans in upstream clusters is hard. Kubernetes provides no native cross-node preemption, uncoordinated concurrent scheduling leads to inconsistencies, and replacing the default scheduler would sever deployments from upstream cycles.

We present OPSche, an open-source Kubernetes Scheduling Framework plugin where external solvers can drive cluster-wide placement decisions in concert with the default scheduler. OPSche atomically validates and enforces solver-produced plans through coordinated framework hooks and supports three trigger modes: scheduling-failure, periodic, and stable-queue – resp. triggered when a workload cannot be placed, at fixed time intervals, when the set of pending workloads stabilises. Each mode has a blocking variant for a finer tuning of placement quality, latency, and disruption.

We pair OPSche with a constraint-based optimisation solver, showing its feasibility across a broad set of cluster configurations and reporting improvements of resource usage by up to 3.0% and scheduling latency by more than a second.

CCS Concepts: • Computer systems organization → Cloud computing; • Software and its engineering → Scheduling.

Keywords: Kubernetes, scheduling framework plugin, global plan enforcement, cross-node preemption

ACM Reference Format:

Henrik Daniel Christensen, Saverio Giallorenzo, and Jacopo Mauro. 2026. A Kubernetes Scheduler Plugin for Cluster-Wide Placement Optimisation. In . ACM, New York, NY, USA, 43 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Kubernetes is the state-of-the-art for container orchestration, managing workloads across clusters that range from a handful of nodes in small enterprises to thousands in hyperscale deployments [11]. At the heart of Kubernetes sits its scheduler: the component responsible for deciding which node should run each pending pod – the smallest deployable unit in Kubernetes that encapsulates one or more containers.

The problem: local vs. global decisions. A problem that characterise Kubernetes default scheduler is that it relies on local, greedy decisions where it evaluates each pod independently, scores the available nodes, and commits to the highest-scoring one, without any coordination across the set of pods under scheduling. This local strategy is fast and simple, but it leaves performance on the table: fragmentation accumulates silently, nodes end up partially filled, and pods get stuck pending, whereas a smarter, cluster-wide rearrangement could accommodate them [6].

To get a sense of the problem, consider a cluster where two nodes are each half-full with pods of different sizes. A new pod arrives that fits on neither node individually, yet it would fit if one pod moved to the other node. In such situations, the default scheduler would see a scheduling failure and give up, whereas a cluster-wide view would reveal the fix. This inefficiency is reflected in practice: industry surveys report that in 2025 the gap between provisioned and requested resources averaged 40% for CPU and 57% for memory [5] with 99.94% of all the clusters overprovisioned on CPUs.

Techniques to improve/optimize global placement plans abound (complete search methods, heuristics, etc.), however, the obstacle is not (solely) in performance but in affordance. The Kubernetes scheduler does not natively implement a global scheduling logic and replacing it with a custom one would sever deployments from upstream release cycles.

The challenge: atomically enforcing a plan. Enforcing a global scheduling plan on Kubernetes scheduler has two main obstacles. First, Kubernetes provides no native mechanism for *cross-node preemption* – its `DefaultPreemption` routine frees capacity on a single candidate node, and cannot coordinate evictions across multiple nodes to realise a cluster-wide rearrangement. Second, the default scheduler runs multiple workers concurrently, each independently dequeuing and placing pods. Without explicit coordination, concurrent workers may consume the capacity that the plan relies on, rendering the plan obsolete before its enforcement.

One could sidestep both issues by replacing the default scheduler entirely with a custom implementation. However, doing so would sever the deployment from the upstream Kubernetes release cycle – one would need to manually port every bug fix, security patch, and feature improvement shipped in the official scheduler, imposing a significant and open-ended maintenance burden. Thus, we deem that the right

design goal is to extend the default scheduler – preserving full compatibility with upstream Kubernetes while adding the coordination layer that global optimisation requires.

The proposed solution: OPSche. We present OPSche, a Kubernetes scheduler plugin that enables an external solver to drive cluster-wide placement decisions in concert with the default scheduler. We build OPSche entirely on top of the *Kubernetes Scheduling Framework* – the official extension mechanism introduced in Kubernetes v1.15 – so that it requires no modifications to any upstream scheduler code.

The core insight behind OPSche is that, by using five hooks that the Scheduling Framework provides, one can enforce arbitrary cluster-wide placement plans atomically and safely preserving concurrent scheduling. Essentially, OPSche works by taking a snapshot of the current cluster state, invoking a solver that computes a deployment plan, and validating the returned plan against a fresh snapshot to prevent the application of stale plans on drifted states. If the plan is valid, OPSche enforces it in three phases: *eviction*, where it issues eviction requests for all pods marked for removal or relocation; *activation*, where it re-activates only the pods that the plan selects for placement, holding all other inbound pods back; and *realisation*, where it steers each pod to its designated node by constraining placement to exactly one feasible node per pod. OPSche prevents concurrent over-commitment of nodes by tracking per-node resource budgets.

To support a wide set of dynamics and operational constraints found in real-world clusters, OPSche provides three trigger modes: a *scheduling-failure* mode that reacts immediately when a pod becomes unschedulable, and two background modes – *periodic* and *stable-queue* – that run the solver proactively, either at fixed intervals or only once the pending-pod set has stabilised. Cluster-wide placement reduces to an NP-hard combinatorial optimisation problem and solver execution may incur non-negligible latency under high contention. For this reason, each mode has two variants: a *blocking*, which pauses scheduling during solver execution, and a *non-blocking*, which allows normal scheduling to proceed in parallel. This plethora of modes lets operators tune the trade-off between placement quality, scheduling latency, and operational disruption to match their workload regime.

We evaluate OPSche on emulated Kubernetes clusters built with *Kubernetes Without Kubelet* [14], driven by synthetic traces parameterised from the Alibaba GPU [3] and Google Cluster Data [10] production traces. The main experimental result is a qualitative one, i.e., the fact that OPSche correctly implements global scheduling as a Kubernetes scheduler plugin. Quantitatively, testing our plugin across 16- and 32-node clusters, two priority configurations, and three workload arrival regimes, we found that OPSche consistently improves effective resource usage by 1.2–3.0% over the default scheduler (single priority) and by 0.2–1.4% under four priority

levels, while the periodic and stable-queue modes also reduce scheduling latency by more than a second. The non-blocking variant matches the blocking variant’s resource efficiency while imposing less interference on regular scheduling, emerging as the recommended default.

Paper Outline. We review Kubernetes scheduling and the Scheduling Framework in Section 2. Then, we present the design of OPSche in Section 3, followed by the description of our evaluation setup and results in Section 4. We conclude discussing related work in Section 5 and drawing final remarks and future directions in Section 6.

2 Preliminaries

We provide the background needed to understand the design and implementation of our scheduler plugin. We review key aspects of Kubernetes’ architecture and default scheduler, and introduce the Kubernetes Scheduling Framework.

2.1 Kubernetes Architecture and Core Concepts

A Kubernetes cluster consists of a control plane (including the API server) and worker nodes. The smallest deployable unit is a pod, which groups one or more containers, each enclosing one or more applications and runtime environments.

Workload controllers (e.g., Deployments, ReplicaSets) typically create and maintain pods, keep a desired replica count – each replica corresponding to a pod instance – and manage pod lifecycle events such as pod replacement.

Each container can specify CPU and memory requests, which determine the resources reserved for the pod and guide the scheduler. Each worker node exposes a finite capacity, of which the allocatable portion is available to pods.

Kubernetes Default Scheduler. The default scheduler in Kubernetes assigns nodes to pending pods through a per-pod scheduling cycle. Pending pods are managed in a queueing system¹, and are processed in parallel by multiple scheduler workers. For each pod, a worker filters nodes based on feasibility constraints (e.g., sufficient resources, node selectors), scores the feasible nodes, and selects the highest-scoring node. As a result, the scheduler makes local, heuristic decisions based on the current cluster state, without explicit coordination across multiple pending pods.

Kubernetes Scheduling Framework. One can customise Kubernetes scheduling in multiple ways, including running a separate scheduler and modifying the scheduler code. However, these approaches typically incur significant engineering and maintenance overhead. In response, Kubernetes v1.15 introduced the *Scheduling Framework*, which allows users to inject custom logic into the default scheduler without having to implement and maintain an entire scheduler from scratch.

¹The system has three queues: *active* (ready for scheduling), *unschedulable* (failed scheduling), and *backoff* (temporary hold after repeated failures).

The Scheduling Framework structures scheduling into extension points – also called *hooks* – that execute at different stages of the scheduling cycle [13] and that plugins can attach to for implementing custom scheduling logic cleanly integrated with the default scheduler.

In particular, the default scheduler already includes a built-in preemption mechanism, `DefaultPreemption`, invoked when no feasible node is found for a pod to evict lower-priority pods. However, this mechanism has a single-node scope: it selects a candidate node and preempts pods therein until it frees enough resources. Thus, Kubernetes provides no native support for multi-node (cross-node) preemption, i.e., eviction strategies that coordinate pod removals across multiple nodes to realise a cluster-wide placement decision.

3 System Design

We present the design of OPSche, our optimisation plugin for the Kubernetes default scheduler. In particular, we discuss two key design dimensions behind OPSche: (1) when to trigger optimisation and whether to block concurrent scheduling during solver execution, and (2) how to enforce placement plans through the hooks while coordinating multiple parallel scheduler workers that independently schedule pods. Thanks to the combination of the presented design choices, OPSche implements optimal pod scheduling for Kubernetes as a plugin that one can install alongside a vanilla Kubernetes deployment, leaving its core components unchanged.

3.1 OPSche Workflow

Conceptually, the workflow of OPSche is simple, it (1) snapshots the current cluster state, (2) asks for an improved plan from a solver that may relocate or evict pods across nodes, and (3) enforces this plan by coordinating how the scheduler admits new pods and re-schedules pods marked for relocation. We visualise OPSche’s workflow in Figure 1 through an example of cluster-wide optimisation via cross-node reallocation. Initially, Node 1 hosts `pod1` and Node 2 hosts `pod2` and `pod3`. When `pod4` arrives ①, only Node 2 could have sufficient capacity to host it, but the presence of both `pod2` and `pod3` prevents its deployment. The workflow proceeds as follows: (1) OPSche captures the cluster state; (2) the solver determines that `pod4` can be placed on Node 2 if `pod3` relocates to Node 1; (3) OPSche applies the plan by evicting `pod3` ① – which re-enters the active queue ② – and pinning `pod4` to Node 2 ③ and `pod3` to Node 1 ④.

Snapshot construction. When OPSche triggers an optimisation run, it first constructs a snapshot of the current cluster state from shared informer caches by extracting the relevant node and pod state (both running and pending). Informers are event-driven clients that maintain an up-to-date in-memory cache streamed from the Kubernetes API server. The scheduler consults these caches when selecting nodes. From the extracted state, OPSche passes the snapshot

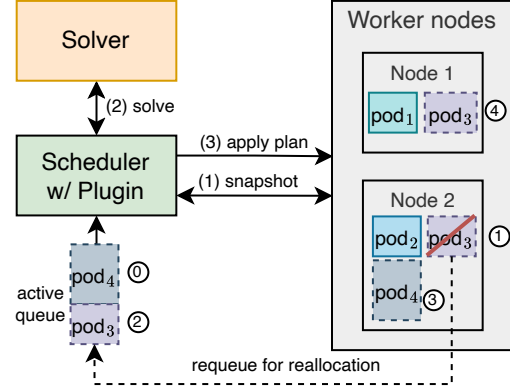


Figure 1. Example of OPSche’s workflow: a pod (`pod3`) is moved from Node 2 to Node 1 to place a pending pod (`pod4`).

to an external optimisation solver. OPSche is solver-agnostic. Any solver able to produce a placement plan that satisfies the Kubernetes placement constraints would work, whether it be based on heuristics or a complete search procedure (e.g., constraint programming, SAT, SMT or MIP). The solver should be interruptible at any time and, if possible, return the best feasible plan found so far; otherwise no plan.

Plan structure and validation. The plan returned by the solver describes three kinds of actions: (i) placements for pending pods, (ii) movements of already running pods between nodes, and (iii) evictions of pods that must be removed to free capacity. Before accepting the plan, OPSche validates that the former is still applicable, since the cluster state may have drifted after taking the snapshot.

The validation step checks the state of a fresh snapshot against the plan’s preconditions to ensure that a) pods to be moved still exist and reside on their initial node; b) pods marked for placement still exist and are pending; and c) each node is still usable. OPSche then re-evaluates the plan’s resource feasibility w.r.t. the capacity it would free (evictions) and the capacity it would consume (placements and moves) to ensure every node stays within its allocatable CPU/memory limits. If any check fails, the plan is discarded, which avoids overfilling nodes or acting on pods/nodes that have moved, terminated, or otherwise changed.

Plan enforcement. If a plan passes validation, OPSche enforces it. OPSche has at most one active plan at any time, used to store and track the pods to be placed and moved and that are held back from re-entering the scheduling queue.

Plan enforcement goes through three phases: *eviction*, *activation*, and *realisation*.

At eviction, OPSche issues eviction requests for every pod that the current plan marks for move/removal. Evictions are irrevocable: after issuing the first eviction, one cannot restore the original cluster state. To avoid scheduling pods before capacity is available, OPSche waits for the required evictions

to complete before proceeding. Concretely, OPSche polls for pods to disappear at a fixed interval (250 ms by default).

During activation, OPSche re-activates the set of pending pods that the plan selects for placement, including pods that were evicted and are expected to reappear as pending (e.g., via their controllers). Meanwhile, inbound pods that are outside the plan are held back to prevent them from consuming capacity that the placement plan relies on.

At realisation, admission and node selection are constrained to enforce the plan: selected pods are admitted and pinned to their designated destination nodes. A watcher monitors progress at a fixed interval (250 ms by default) and declares the plan complete once it observes the intended placements, otherwise, after a set timeout (4 s by default), it marks the plan as failed. Then, OPSche clears the active plan and releases any pods that were held back during execution and the scheduler resumes normal operation.

3.2 Operational Modes

OPSche delegates finding better cluster-wide placements to an external optimisation solver. There are two orthogonal axes that determine solver invocations: (1) when to run the solver, and (2) whether solver execution blocks the scheduler from processing inbound pods (*blocking* vs. *non-blocking*).

When to run the solver. Regarding the first axis, OPSche supports a *scheduling-failure* mode, which triggers a solver run immediately if the default scheduler fails to find a node for the pod to be placed, and two background modes, *periodic* and *stable-queue*. The periodic mode runs at fixed intervals, whereas the stable-queue mode triggers only after the queue of pending pods has remained stable for a idle time window.

In *scheduling-failure* mode, each scheduling failure triggers a solver run that attempts to schedule the failing pod and, if a new plan is found, rearranges or evicts other pods. The advantage of this mode is that it is responsive (to failures), but frequent pod arrivals may cause bursts of solver invocations adding latency to other pods.

In *periodic* mode, OPSche runs the solver at fixed intervals (by default 8 s). This mode amortises solver costs, but pods that become unschedulable just after the triggering of the solver would have to wait until the next run. To avoid wasting work in stable periods, OPSche records cluster states it has already solved and, if the same state reappears, the solver invocation is skipped.

In *stable-queue* mode, OPSche runs the solver only after the set of pending pods has remained unchanged for a configurable idle window (by default 8 s). Stability is checked by polling the pending set at a configurable interval (250 ms by default). Moreover, if the pending set changes during solver execution (e.g., a new pod arrives in the queue), the current solver run is cancelled, and OPSche returns to waiting for a stable window before starting a new run. By cancelling an ongoing solver run whenever the pending set changes,

OPSche reduces the likelihood of producing plans that are stale by the time they are enforced. On the downside, frequent pod arrivals prevent the occurrence of the idle window, leading to infrequent optimisation.

Blocking vs non-blocking plan computation. Regarding the second axis, OPSche supports both *blocking* and *non-blocking* modes, which determine whether the scheduler can process inbound pods during solver execution.

In *blocking* mode, OPSche acquires a plan lock before invoking the solver and holds it until a plan has been enforced or discarded. While OPSche has the lock, both new pods and parallel attempts to start a new plan are put on hold.

In *non-blocking* mode, OPSche runs the solver without taking the lock; thus, inbound pods can continue to be scheduled by the default scheduler during solver execution. If the solver returns an improving plan, OPSche checks the plan against the fresh snapshot and, if the plan is applicable (i.e., no incoming scheduling requests have made the plan outdated), it is enforced. Therefore, the non-blocking mode allows inbound pods to be scheduled at the cost of occasionally discarding plans due to state drift.²

In any combination of modes, once a plan is accepted, its enforcement proceeds under the plan lock until the plan is realised or times out.

3.3 Integrating OPSche into the Kubernetes Scheduler

OPSche implements scheduling optimisation by interacting with some of the hooks provided by the Kubernetes Scheduling Framework as visualised in Figure 2. In the following, we briefly comment on how OPSche interacts with those hooks.

Plan computation. To support the different trigger modes, OPSche invokes the solver either (i) when the scheduler worker reaches the *PostFilter* hook after a failed scheduling attempt (*scheduling-failure* mode), or (ii) from a background routine in the *periodic* and *stable-queue* modes.

Plan enforcement. The Kubernetes scheduler processes pods in parallel using multiple scheduler workers that each dequeue a pod and execute a full scheduling cycle independently. OPSche uses a small shared state that stores the current active plan (if any) to coordinate the plan enforcement. OPSche enforces the application of an active plan by constraining scheduler workers to mark only the plan-designated node as feasible for a given pod. The shared state ensures that, at any time, there exists at most one optimisation run and one active plan and that parallel workers apply consistent reservations and do not over-commit nodes. To implement this behaviour, OPSche extends the *PreEnqueue*,

²Note that to verify that the plan is still applicable, OPSche must re-validate it against a fresh snapshot, which requires acquiring the plan lock. This behaviour does not present an issue, considering that the lock is held only briefly during validation and before enforcement and not for the entire duration of the optimisation run, as in blocking mode.

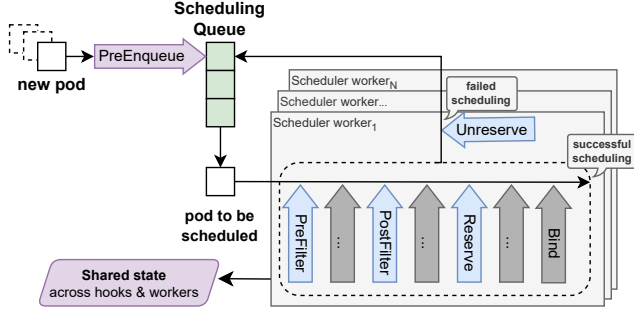


Figure 2. OPSche’s interactions with Kubernetes Scheduling Framework’s hooks to admit, filter, reserve, and bind pods.

PreFilter, *Post-Filter*, *Reserve*, and *Unreserve* hooks. Figure 2 shows the position of these hooks in the Scheduling Framework and how they interact with an active plan.

We briefly comment on the activation sequence of these hooks w.r.t. pod scheduling and how OPSche makes use of them. When a new pod arrives, the worker first calls the *PreEnqueue* hook to decide whether the pod may enter the scheduling queues. OPSche uses this hook to temporarily hold back pods that are not part of the plan, so they do not interfere with its execution. When a worker is ready to schedule the next pod, it dequeues the pod from its queue and proceeds to run the scheduling routine, passing through the remaining hooks. The worker calls the *PreFilter* hook to compute per-node feasibility. OPSche uses this hook to exclude nodes that are not part of the active optimisation plan, effectively placing pods on the nodes chosen by the plan. If no feasible node is found, the worker invokes the *Post-Filter* hook, from which OPSche’s *scheduling-failure* mode invokes the optimiser. To ensure that parallel workers do not over-commit nodes, the worker invokes the *Reserve* hook to tentatively reserve capacity for the pod on its candidate node before binding it. OPSche leverages the same mechanism when enforcing a plan: it maintains per-node budgets that track how much of each workload can still be placed on a node. When a worker reserves a pod, OPSche decrements the corresponding budget; if the scheduling attempt later fails, the *Unreserve* hook is invoked to roll back the reservation and refund the budget.³

Checking plan completion. To detect when a plan has completed, OPSche runs a background watcher that periodically compares the current cluster state against the active

plan.⁴ The watcher checks completion w.r.t. all pods relevant to the plan. The completion check treats pods differently depending on their category.

Pods in the *identity-stable* category are standalone pods, created directly as Pod objects without being managed by a higher-level controller. The identity (name/UID) of these pods is stable during their lifetime and the plan can refer to their concrete pod instance. In this case, the watcher treats a planned placement as satisfied once it observes that the pod is either bound to its designated node (i.e., `pod.spec.nodeName` matches the planned destination), or terminating/absent (in which case, the concrete instance no longer exists and plan enforcement cannot proceed). Pods in the *identity-unstable* category are controller-managed pods, created and maintained by a controller (e.g., `ReplicaSet`, `Deployment`). Controllers may delete and recreate pods at any time (e.g., after eviction), typically producing new pod names and UIDs. Consequently, the plan cannot reliably track individual pod identities. Instead, the watcher tracks progress at the workload level (via the workload owner reference) and reasons in terms of quotas: a planned workload placement is satisfied once the workload has no remaining pending pods attributed to the plan, or the workload has no pods because it was scaled down or deleted.

When the plan has completed, the watcher clears the active plan and releases all pods held back during the plan’s execution, allowing the scheduler to resume regular scheduling. To avoid blocking the system in pathological cases where pods remain pending indefinitely (e.g., due to constraints extrinsic to our model, external controllers, or incompatible node conditions) there is a plan enforcement timeout that aborts enforcement if the plan has not completed within a configurable time window (4 s by default). When the timeout expires, the plan is discarded, any held-back pods are released, and regular scheduling resumes.

4 Evaluation

This section describes our evaluation of OPSche across its operating modes and compares it with the Kubernetes default scheduler, quantifying the impact on resource usage, scheduling latency, pod deletions, and optimisation overhead.

4.1 Methodology

We evaluate OPSche using a lightweight, trace-driven framework that runs on a single machine while simulating cluster dynamics over extended time horizons. The framework

³To reliably relocate pods via eviction, OPSche identifies pods by their workload owner – i.e., the managing controller (e.g., a `ReplicaSet`) – rather than by concrete pod identities such as names or UIDs. The reason for this choice is that pod identities are ephemeral: once a pod is evicted, its controller may recreate it with a new identity. Accordingly, when a plan requires moving a pod, OPSche records the workload owner, reserves capacity for that workload on the target node, and then evicts the pod, delegating the actual recreation to the controller.

⁴Kubernetes experts might wonder why we do not use the *PostBind* hook – which runs immediately after the scheduler commits a binding to the API server – to obtain the completion signal. The reason is that pods that are part of a plan can be deleted, recreated, or scaled down by their controllers before reaching the hook, which implies that it can happen that the “last” pod, which would signal a plan’s completion, might disappear, preventing OPSche from reliably inferring the plan’s status.

Table 1. Trace generator parameters and values used in the experiments.

Trace parameter	Values used
Nodes in cluster	{16, 32}
Simulation time	2 h
Target resource usage	90%
Workload inter-arrival time: mean	{4, 8, 16} s
Workload inter-arrival time: min	0.1 ms
Workload inter-arrival time: max	$16 \times$ mean inter-arrival time
Workload lifetime: min	2 s
Workload lifetime: max	24 h
Workload priority	{1} or {1, 2, 3, 4} (geom. ratio 0.8)
Replicas per ReplicaSet	{1, 2, 3} (geom. ratio 0.8)
Per-pod resource requests: mean	0.125 (fraction of a node)
Per-pod resource requests: min	0.05 (fraction of a node)
Per-pod resource requests: max	0.5 (fraction of a node)

comprises three main elements: cluster emulation, trace generation, and trace replay.

Cluster Emulation. To simulate Kubernetes clusters, we use Kubernetes WithOut Kubelet (KWOK) [14], an open-source emulator that instantiates a Kubernetes control plane with virtual nodes. KWOK faithfully emulates Kubernetes scheduling logic while avoiding the resource overhead of executing containers. Each virtual node exposes CPU and memory capacity and pods transition through standard Kubernetes scheduling phases and consume simulated capacity based on their resource requests. Because KWOK advances the system state in real time, the duration of an experiment directly corresponds to wall-clock time and enables us to run experiments using production scheduler binaries (with and without OPSche) while achieving reproducible results.

Trace Generation. At the time of writing, there is no publicly available benchmark for Kubernetes scheduling and preemption policies. For this reason, we develop a *trace generator* that produces workload arrivals and removals, designed to reflect real-world workloads in a cluster. Each trace specifies the number of nodes in the cluster and a time-ordered sequence of workload events (arrivals and removals) over a fixed simulation horizon. We instantiate each workload as a ReplicaSet that manages one or more pod replicas. The trace generator can produce a wide range of workload patterns by configuring the distributions used to sample workload attributes, including inter-arrival times, lifetimes, priority levels, replica counts, and per-pod resource requests.

To generate realistic traces, we first analyse publicly available traces [17], particularly the Alibaba GPU [3] and Google Cluster Data 2019 [10] traces, to parameterise the distributions that underlie these attributes.⁵ These traces exhibit heavy-tailed inter-arrival times, lifetimes, and resource requests, which we model using truncated Pareto (Type I) distributions specified by a target mean and min/max bounds.

⁵We provide an analysis of the traces in Supplementary Material, Section B.

We model priorities and replica counts using truncated geometric distributions, restricting replica counts and number of priority levels to a bounded range. The list of parameters of the trace generation process, including the parameters of the distributions, is available in Table 1.

To stress-test the scheduler, we generate traces with a high degree of *resource contention* – i.e., situations where competing pods share limited cluster resources (e.g., due to high demand, fragmentation, or placement constraints). To this end, we first define the notion of *effective usage* at each time unit as the highest of normalised CPU and memory requests, and the *time-averaged effective usage* as the mean of effective usage over the simulation horizon. Thus, to achieve high resource contention, the trace generator calibrates the workload lifetime distribution so that the resulting trace reaches a target *time-averaged effective usage* through the repeated adjustment of the mean lifetime for each trace configuration. We obtain those traces by finding an initial lifetime estimate which we then refine iteratively. Practically, given the number of nodes, the mean inter-arrival time, and the configured distributions for resource requests and replica counts, we obtain an initial mean lifetime estimate via Little’s Law [16]. Starting from the initial lifetime estimate, we generate a full trace realisation and measure its time-averaged effective usage. If the measured usage deviates from the target, we scale the mean lifetime proportionally and repeat the procedure. The process stops once the relative error is within 1%.

To avoid cold-starts (where there is initially little to no contention), each generated trace includes an initial state containing workloads that are already running at the beginning of the simulation, such that their aggregate effective requested load approximately matches the target usage. Each initial workload is created with start time 0 and a sampled remaining lifetime. One could follow a naïve approach and sample the remaining lifetime directly from the truncated Pareto distribution $f(x)$ used for newly arriving workloads. However, this practice would systematically underrepresent long-lived workloads in the initial state since – intuitively, at any observation point, workloads with longer lifetimes are more likely to be present than short-lived ones. Following the standard correction for this bias [23], we shift the distribution by sampling initial workloads from the length-biased distribution $f^*(x) \propto x \cdot f(x)$, that for a Pareto distribution with $f(x) \propto x^{-(\alpha+1)}$ yields $f^*(x) \propto x^{-\alpha}$. After sampling a total lifetime X^* from f^* , we assign each initial workload a remaining lifetime $\sim \text{Uniform}(0, X^*)$, which corresponds to observing the workload at a random point during its execution. We populate the initial state by repeatedly sampling workloads (remaining lifetime, replica count, priority, and CPU/memory requests) until the effective requested load reaches the target.

Finally, we generate the newly arriving workloads (workloads after the initial state) by sampling workloads sequentially until the simulation horizon is covered. For each workload, we sample its replica count, inter-arrival time (which determines the start time), lifetime (which determines the end time) from the calibrated lifetime distribution, per-pod CPU and memory requests, and priority level from the distributions described above. This procedure generates a time-ordered sequence of workload arrivals and removals whose aggregate load matches the target usage for the corresponding trace configuration.

Trace Replay. A trace replayer executes the generated traces against the simulated cluster by translating the trace workloads into Kubernetes API operations. The replayer first provisions the specified number of virtual nodes and creates `PriorityClass` objects for all priority levels. After applying the initial workloads to reach steady-state load, the replayer issues the newly arriving trace workloads at their scheduled timestamps. A monitor samples statistics every 1 s, recording the number of running and pending pods, pod deletions, resource usage, and per-pod events (creation and running timestamps). We exclude initial workloads from the statistics.

For every simulation, we track the following metrics:

- *Usage.* At intervals of 1 s, we record the total requested CPU and memory of all running pods, normalised by the total cluster capacity for each resource. To capture the bottleneck resource at each time unit, we pick the highest between CPU and memory usage and take that metric’s average over the entire duration of the simulation. Higher values indicate more resource usage.
- *Scheduling latency.* To quantify scheduling responsiveness, we measure the time elapsed between when a pod is submitted to the cluster and when it first reaches the running state. We record this latency for each pod and report results separately for each priority level. Lower values indicate faster scheduling decisions.
- *Pod deletions.* To quantify disruption, we track pod deletions throughout the simulation. We record each deletion event and report the final cumulative count separately for each priority level. Lower values indicate less disruptive scheduling behaviour.
- *Solver runs.* We record how many times OPSche invokes optimisations during a simulation. This metric captures the intervention frequency of OPSche: fewer optimisations imply lower overhead, but may also correspond to fewer opportunities for optimisation.
- *Plan activations.* We record how many times the optimiser produces an improved placement plan that is accepted and enforced by the scheduler. This metric captures how often optimisation results in concrete scheduling actions, as opposed to solver runs that do not necessarily yield an applicable or beneficial plan.

4.2 Experimental Setup

To stress-test OPSche under challenging optimisation conditions, we intentionally instantiate it with a complete solving approach that tackles the NP-hard pod placement problem and can therefore require non-negligible computation time before producing solutions. Concretely, we implement the optimiser by adapting the classical bin-packing formulation to pod scheduling, encoded and solved using the CP-SAT constraint programming framework of Google OR-Tools [7]. Since the scope of this paper is not the design of the optimisation algorithm itself, but rather the mechanisms that allow the Kubernetes scheduler to integrate, trigger, and enforce solver-produced placement plans, we refer the interested reader to the supplementary material for full details of the optimisation model and its implementation.

We focus our evaluation on medium-sized clusters, which are (i) representative of real-world deployments since more than half of companies report operating clusters of up to 50 nodes [8], (ii) large enough to present non-trivial optimisation challenges, and (iii) small enough that their owners have not conducted extensive ad-hoc optimization attempts.⁶

As reported in Table 1, we generate traces for two cluster sizes (16 and 32 nodes), two priority configurations (one priority level and four priority levels), and three mean inter-arrival time settings to produce a range of workload patterns, from more bursty (4 s and 8 s) to more steady (16 s) arrivals, which can impact the optimisation opportunities and the behaviour of the different OPSche modes. We also generate five distinct traces for each configuration through different random seeds, which allows us to capture variability in the results due to randomness in the workload patterns. We replay every trace under the default Kubernetes scheduler with and without OPSche, evaluating OPSche’s three triggering modes – *scheduling-failure*, *periodic*, and *stable-queue* – across different timing configurations and for both *blocking* and *non-blocking* variants.

We run each experiment on an isolated virtual machine with 8 vCPUs and 48 GB of RAM (Intel Xeon Gold 6130), Ubuntu Linux 24.04, and OR-Tools 9.14.6206.

For reproducibility, the instructions and the code to run the experiments is available at <https://zenodo.org/records/19052813> (anonymised for peer review purposes).

4.3 Results

We start by comparing the *non-blocking* variants of OPSche’s three triggering modes: *scheduling-failure*, *periodic* (8 s interval), and *stable-queue* (8 s delay). This comparison is shown in Figure 3, which reports paired differences relative to the default scheduler⁷ under two priority configurations: a single

⁶For larger clusters, we expect that a faster but incomplete technique (e.g., local search) would likely be preferable. Exploring this design space is left as future work.

⁷Mathematically, each measure corresponds to OPSche’s performance minus the corresponding performance of the default scheduler.

priority level and four priority levels. Each colour denotes an OPSche mode, and point shapes indicate the number of nodes in the cluster.

For each trace setting, we report the mean difference across all runs of that trace setting as a diamond and each individual run for a specific seed as a circle (16 nodes) or square (32 nodes). The rows correspond to the following metrics: (1) effective usage (higher is better), (2) scheduling latency in milliseconds (lower is better), (3) pod deletions (lower is better), (4) solver runs, and (5) plan activations. The last two rows reports both the runtime overhead introduced by OPSche and how effectively each mode converts optimization runs into valid plans (the gap between solver runs and plan activations) that can be activated rather than discarded.

All reported ranges refer to the variation across trace settings, computed from the mean over the five seeds for each setting. In the following, we first comment on the results of the different measures (we discuss solver runs and plan activation together due to their inherent interplay). Then, we briefly present a set of ablation studies on variables such as (non-)blocking variants, usage of the DefaultPreemption, parameters of the periodic and stable-queue modes.

Usage. Almost all OPSche modes consistently improve usage compared to the default scheduler. The improvements are generally more pronounced for larger inter-arrival times, and when running with a single priority level. The explanation of these results is straightforward: larger inter-arrival times provide more opportunities for optimisation. Moreover, with one priority, the default scheduler struggles to react to suboptimal placements since it does not reschedule pods after their first placement.

With one priority, *periodic* achieves improvements ranging from +1.4% to +3.0%, and *stable-queue* ranges from +1.4% to +2.9%, whereas *scheduling-failure* ranges from +1.2% to +3.0%. With four priority levels, *periodic* yields improvements in the range +0.7% to +1.3% and *stable-queue* ranges from +0.7% to +1.4%, while *scheduling-failure* varies from -0.2% to +1.2%.

Overall, *stable-queue* and *periodic* achieve the best performance across the settings while *scheduling-failure* is less effective, although it still improves usage in most settings.

Scheduling latency. Looking at scheduling latency, *periodic* and *stable-queue* generally improve upon the default scheduler, whereas *scheduling-failure* increases latency. We expected this behaviour, since the latter mode triggers optimisation at each scheduling failure, potentially causing many solver runs and latency due to solve and plan-enforcement time, which can also delay other pending pods. Again, improvements are more pronounced for larger inter-arrival times, and when running with a single priority level.

For a single priority level, *periodic* impacts latency between -4, 235 and +83 ms (lower is better), and *stable-queue*

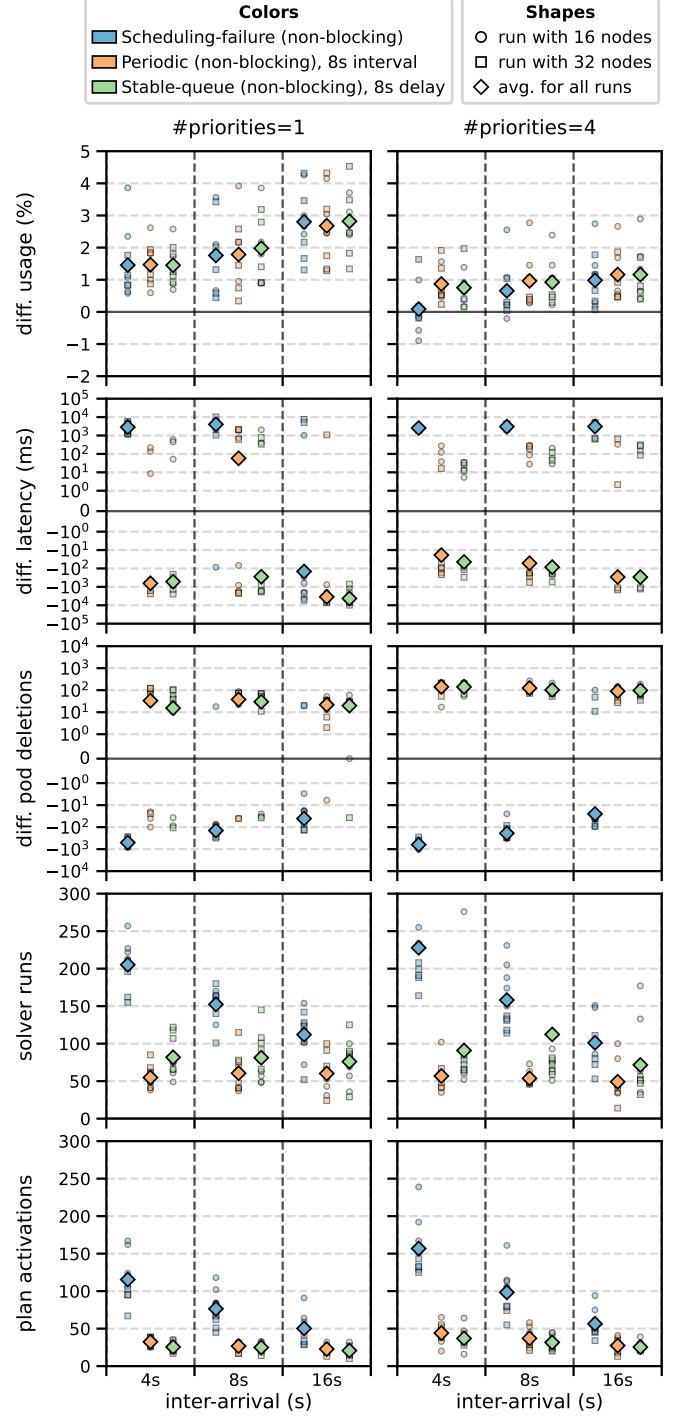


Figure 3. Comparison of OPSche modes with *non-blocking* variants. Each point shows the difference (OPSche’s mode minus default) for a single run (circles: 16 nodes; squares: 32 nodes), and diamonds denote the mean difference per trace setting. Left column: one priority level; right column: four priority levels. Rows report effective usage (%), scheduling latency (ms), pod deletions, solver runs, and plan activations.

between $-5,695$ and $+83$ ms, whereas *scheduling-failure* between $-1,258$ to $+5,491$ ms. With four priority levels, *periodic* ranges from -439 to $+65$ ms and *stable-queue* ranges from -351 to -3 ms, while *scheduling-failure* varies from $+2,513$ to $+3,329$ ms. The narrower improvement range with four priorities is explained by the Kubernetes default scheduler’s *DefaultPreemption* mechanism, which already provides a priority-aware eviction mechanism leaving less room for improvement. Specifically, the default scheduler, upon a scheduling failure, calls a *DefaultPreemption* routine that evicts one or more lower-priority pods to free resources on a *single* candidate node to accommodate the pending pod. Looking at the per-priority breakdown (see Table 8 in the supplementary material), the latency improvements of *periodic* and *stable-queue* are driven almost entirely by the lowest-priority class, which sees reductions of up to $1,467$ ms for *periodic* and up to $1,216$ ms for *stable-queue*, while higher-priority classes experience only marginal changes of tens of milliseconds. In contrast, *scheduling-failure* increases latency across all priority classes by similar magnitudes, with the lowest-priority class increases ranging from $+2,465$ to $+3,843$ ms.

Similarly to the usage metric, *stable-queue* and *periodic* achieve the best performance across the settings, with a slight edge to *stable-queue*, which is more consistent across settings, likely derived from its reactive triggering.

Pod deletions. Considering pod deletions, as expected from the results of the previous two metrics, *periodic* and *stable-queue* most often increase the number of deletions. Their latency gains derive indeed from pod preemption. Overall, the two modes incur a similar number of deletions. However, at lower inter-arrival times, *periodic* tends to trigger fewer deletions than the default scheduler, whereas *stable-queue* incurs more deletions than the default scheduler across all settings. Interestingly, *scheduling-failure* reverses the above pattern, substantially reducing pod deletions w.r.t the default scheduler to gain small usage improvements at the cost of higher latency.

For a single priority level, *periodic* impacts the number of deletions (lower is better) between $+2.2$ and $+64.4$, *stable-queue* between -1.4 and $+33.2$, whereas *scheduling-failure* between -528.2 and -7.8 . With four priority levels, *periodic* impacts between $+70.8$ and $+159$ deletions and *stable-queue* between $+73$ and $+145.6$, while *scheduling-failure* between -752.6 and -19.4 .

Runtime overhead. Looking at solver runs and plan activations (fourth and fifth rows), *scheduling-failure* triggers far more optimisations than *periodic* and *stable-queue* – as expected, given its scheduling-failure activation behaviour – while *stable-queue* triggers slightly more optimisations than *periodic* – consistent with the reactive activation behaviour of the latter mode.

The number of plan activations is generally lower than the number of solver runs, indicating that not every optimisation yields an accepted plan – either because the solver finds no improvement, or because the resulting plan is not applicable.

Periodic stands out as the mode in which plan activations most closely track solver runs, suggesting that most of its solver invocations translate into accepted plans.

With a single priority level, *periodic* triggers between 46 and 76 solver runs and *stable-queue* triggers between 60 and 102 solver runs, whereas *scheduling-failure* triggers between 110 and 226 solver runs. With four priority levels, *periodic* ranges from 40 to 59 solver runs and *stable-queue* ranges from 50 to 154 solver runs, while *scheduling-failure* varies from 84 to 266 solver runs.

Overall, the results show that OPSche improves cluster usage and, in most settings, also reduces scheduling latency relative to the default scheduler. Across the evaluated settings, *periodic* and *stable-queue* are the strongest modes, achieving the best trade-off between usage and latency improvements and pod deletions compared to the default scheduler.

4.3.1 Ablation studies. We complement the previous experiments and results with another set of experiments conducted to explore configuration variables that can influence the behaviour of OPSche. We evaluate (i) the difference between *blocking* and *non-blocking* variants of the modes, (ii) the impact of the *DefaultPreemption* mechanism, and (iii) the sensitivity of the *periodic* and *stable-queue* modes to different intervals and stable-queue idle window durations. For space reasons, we report the detailed results of these experiments in the supplementary material.

Blocking vs. non-blocking. Table 2 shows a comparison between the *blocking* and *non-blocking* variants. Overall, effective usage is essentially unchanged ($\leq 0.1\%$), and both solver runs and plan activations remain comparable. The clearest differences are visible for scheduling latency and pod deletions, where the preferred variant depends on how each trigger mode interacts with the mean inter-arrival time.

For *scheduling-failure*, the non-blocking variant increases latency at the two shortest inter-arrival times ($+66$ ms at 4 s, $+303$ ms at 8 s) but reduces it at 16 s (-209 ms). Although it may seem counterintuitive that *non-blocking* can yield higher latency at shorter inter-arrival times, this occurs because the cluster state changes frequently during solver execution, making computed plans more likely to become stale and be discarded. As a result, the pods that triggered optimisation may remain unschedulable until a subsequent solver run, increasing their waiting time. In contrast, with longer inter-arrival times, the plans are more likely to remain valid, allowing *non-blocking* to proceed with scheduling while the solver runs, which can reduce latency. At the same time, non-blocking produces substantially fewer pod deletions at 4 s (-142). This trade-off is consistent with higher trigger

Table 2. Mean difference (non-blocking – blocking) per inter-arrival time, aggregated over node counts and all priority configurations, with DefaultPreemption enabled.

Metric	Scheduling-failure (non-blocking – blocking)			Periodic, 8s interval (non-blocking – blocking)			Stable-queue, 8s delay (non-blocking – blocking)		
	Inter-arrival = 4s	8s	16s	4s	8s	16s	4s	8s	16s
Diff. usage (%)	+0.1	+0.0	+0.0	+0.0	+0.0	-0.1	+0.0	+0.0	+0.0
Diff. latency (ms)	+66	+303	-209	-133	+32	+245	+30	-3	-266
Diff. pod deletions	-142.0	-10.2	-4.2	+20.6	+9.2	+2.6	+11.4	-2.0	+11.4
Diff. solver runs	+4	+1	+1	+1	+0	+2	+6	+5	+0
Diff. plan activations	-7	+2	+1	+0	+2	+0	-1	+0	+1

rates and lower inter-arrival times: blocking delays pods until each plan is available, which helps in keeping plans valid, but the resulting more frequent rearrangements increase deletions. Non-blocking avoids these deletions by allowing pods to be scheduled, at the cost of producing stale plans and increasing latency.

For *periodic*, non-blocking reduces latency at 4 s (−133 ms) but increases it at 16 s (+245 ms), while modestly increasing pod deletions across all inter-arrival times. The latency improvement for non-blocking under lower inter-arrival times is expected: proceeding regular scheduling while the solver runs is beneficial when pods arrive frequently.

For *stable-queue*, the two variants are nearly indistinguishable at the shorter inter-arrival times, but non-blocking yields a latency improvement at 16 s (−266 ms). This result matches the design: stable-queue triggers optimisation only after the scheduling queue remained stable for the configured delay, so, with longer inter-arrival times, the queue stabilises more frequently, allowing for more optimisations. In that regard, non-blocking solving imposes little interference (few arrivals during the solve) while avoiding the unnecessary scheduling delay introduced by blocking. Pod deletions remain small and show no consistent trend.

Overall, the results suggest that the non-blocking variant is a reasonable default choice: it achieves essentially the same effective resource usage as the blocking variant, while reducing interference with normal scheduling and exhibiting only small differences in scheduling latency.

Sensitivity to timing parameters in periodic and stable-queue modes. To assess sensitivity to timing parameters, we vary the triggering configuration for both periodic and stable-queue modes by testing shorter (4 s) and longer (16 s) values around the 8 s baseline. These values are chosen to mirror the mean inter-arrival times used in our traces (4 s, 8 s, and 16 s), allowing us to study how optimisation frequency interacts with different workload arrival regimes. Table 3 reports the mean paired differences between the 8 s baseline and the alternative configurations.

The dominant effect is on scheduling latency, which mainly depends on the mean inter-arrival time, while the remaining metrics largely reflect the expected change in optimisation frequency. For *periodic*, using a shorter interval (8 s to 4 s) yields the largest latency reduction with 16 s inter-arrival (−703 ms), whereas using a longer interval (8 s to 16 s) increases latency under lower inter-arrival times (+187 ms at 4 s). For *stable-queue*, latency shifts are smaller: a shorter idle window (8 s to 4 s) slightly reduces latency at 4–8 s, while a longer idle window (8 s to 16 s) increases latency across inter-arrival times.

Therefore, as expected, shorter intervals/windows increase the number of solver invocations, which can reduce latency, but this higher optimisation frequency typically increases pod deletions because more frequent plan activations lead to more evictions and relocations.

Impact of DefaultPreemption. To isolate the impact of OPSche’s preemption and plan-enforcement mechanisms from Kubernetes’ built-in behaviour, we also repeat the experiments with *DefaultPreemption* disabled in the default scheduler. This configuration prevents Kubernetes from evicting pods opportunistically upon a scheduling failure, ensuring that any observed evictions and reallocations originate solely from OPSche.

As shown in Table 4, having *DefaultPreemption* enabled primarily shifts the trade-off between latency and pod deletions, with only small effects on usage and number of solver runs. Across all three trigger modes and inter-arrival times, effective usage changes by $\leq 0.1\%$. In contrast, scheduling latency consistently decreases when *DefaultPreemption* is enabled: the latency difference (enabled – disabled) is negative in every setting, ranging from −459 ms to −2,194 ms for *scheduling-failure*, −728 ms to −1,558 ms for *periodic*, and −684 ms to −2,185 ms for *stable-queue*, with the largest reductions occurring at longer inter-arrival times. This latency improvement is explained by the built-in DefaultPreemption heuristic that allows the scheduler to quickly evict lower-priority pods to make room for pending pods, rather than

Table 3. Mean difference between 8s baseline and other timing variants per inter-arrival time, aggregated over node counts and priority configurations, for non-blocking runs with DefaultPreemption enabled.

Metric	Periodic						Stable-queue					
	Interval: 8s → 4s			Interval: 8s → 16s			Delay: 8s → 4s			Delay: 8s → 16s		
	Inter-arrival = 4s	8s	16s	4s	8s	16s	4s	8s	16s	4s	8s	16s
Diff. usage (%)	+0.1	+0.2	+0.1	-0.2	-0.1	+0.0	+0.1	+0.1	+0.3	-0.2	-4.6	+0.0
Diff. latency (ms)	+21	-217	-703	+187	+22	-88	-57	-22	+29	+105	+78	+76
Diff. pod deletions	+43.4	+24.9	+17.4	-45.8	-34.1	-16.7	+37.2	+23.8	+1.9	-45.0	-118.2	-17.9
Diff. solver runs	+38	+32	+21	-24	-24	-22	+82	+32	+39	-47	-47	-31
Diff. plan activations	+21	+13	+8	-15	-11	-6	+16	+7	+5	-12	-9	-4

Table 4. Mean difference (DefaultPreemption enabled – disabled) per inter-arrival time, aggregated over node counts and all priority configurations, for non-blocking runs.

Metric	Scheduling-failure (enabled – disabled)			Periodic, 8s interval (enabled – disabled)			Stable-queue, 8s delay (enabled – disabled)		
	Inter-arrival = 4s	8s	16s	4s	8s	16s	4s	8s	16s
Diff. usage (%)	-0.1	+0.0	+0.0	+0.0	-0.1	-0.1	-0.1	+0.0	+0.1
Diff. latency (ms)	-459	-1501	-2194	-728	-1033	-1558	-684	-1291	-2185
Diff. pod deletions	+260.4	+158.8	+84.8	+525.7	+297.9	+144.5	+533.9	+289.9	+149.4
Diff. solver runs	+0	+3	+0	+3	+2	+4	+12	+17	-8
Diff. plan activations	+0	+5	+0	+1	+2	+1	+0	+1	+1

waiting for a solver-produced plan to be computed and enforced. However, this improvement comes at the cost of more pod deletions because *DefaultPreemption* evicts pods greedily on a single node without considering the global cluster state. As a result, enabling *DefaultPreemption* increases deletions in all settings (from +84.8 up to +533.9), with the largest increases typically observed at the shorter inter-arrival times. Solver runs and plan activations change little in comparison.

Furthermore, with multiple priorities, the latency gains are concentrated in the lowest-priority: nearly all improvements come from the lowest-priority pods, while the three higher-priority classes change only marginally, consistent with *DefaultPreemption* already protecting high-priority pods.

5 Related Work

We structure the comparison of our proposal with related work following the categorisation from recent surveys [2, 4, 25, 26] along three main directions: *(meta)heuristics*, *machine learning* (ML), and *solver-based* approaches.

(Meta)Heuristics. Several works use (meta)heuristics for pod placement, either at scheduling time or by rescheduling pods to consolidate resources or reduce load imbalance [25].

Moritz de Carvalho Neto et al. [20] propose Kubernetes Scheduling Extension (KSE), which introduces periodic, metric-driven load balancing by migrating pods from over- to under-loaded nodes, based on CPU and memory usage. Zhang et

al. [30] introduce ATASL, a custom Kubernetes scheduler that augments scheduling with application-type awareness. Fu et al. [9] propose ProCon, a scheduler that adds scheduling logic to both the control-plane and the worker nodes. In ProCon, workers monitor per-container progress, and the control plane estimates remaining runtimes to rank nodes and reduce resource contention. Wei-Guo et al. [28] propose a cost-aware scheduling model using metaheuristics that combines particle swarm optimisation and ant colony optimisation.

Overall, (meta)heuristic-based approaches can find good placements quickly but offer no optimality guarantees and require careful parameter tuning [22]. Integration-wise, we did not find a proposal able to co-exist with the default scheduler as OPSche does. However, since OPSche is solver agnostic, it can exploit these kind of (meta)heuristics assuming they can be incorporated into the external solver to compute the placement plans. We leave this integration as future work.

Machine Learning. Machine learning approaches for Kubernetes scheduling fall into two subcategories: *deep learning forecasting*, which predicts indicators (e.g., upcoming usage levels, queue length, estimated pod runtimes) and feeds them into scoring or scheduling constraints; *deep reinforcement learning*, which trains an agent to learn a scheduling policy that maximises rewards such as usage [4].

Yang et al. [29] exemplify the deep learning forecasting approach by combining a grey model with long short-term memory to predict resource usage dynamics. Their design augments the cluster with monitoring and prediction modules to generate forecasts, which are then consumed by a scheduling module that periodically reallocates resources based on the predicted trends. Jian et al. [12] instead exemplify reinforcement learning with DRS, which is a Deep Q-Network-based (DQN) scheduler that comes as an alternative Kubernetes scheduler.

Similarly to (meta)heuristics, we did not find ML-based methods that integrate into Kubernetes as OPSche does. ML-based approaches also often face practical limitations due to the need for substantial training data and careful model tuning. Moreover, their performance can degrade under workload shifts. Clearly, better forecasts and learned policies could be used as a guidance for producing better placements plans, and OPSche could integrate them as well by using them as part of the external solver.

Solver-based Approaches. There are a few proposals that incorporate exact solvers for cluster-wide placement decisions we were inspired by.

Boreas [15] is a scheduler that targets optimal placements subject to hard constraints by batching newly arriving pods and solving a placement optimisation problem for each batch. Boreas is probably the closest work to ours, but differently than OPSche they execute the optimisation outside the default scheduler running the Zephyrus2 optimiser [1] at every decision point.

SAGE [18] operates at pre-deployment stage: given an application description and a catalogue of cloud offers it computes both a cost-minimising node-pool selection and a placement plan but lacks real-time knowledge of resources consumed by Kubernetes at deployment time.

DCM [27] is an alternative scheduler (architecture) where one can express cluster-management policies as SQL constraints stored in a relational database. A compiler translates the SQL schema and constraint views into an encoder that builds a optimisation model, solved with Google OrTools' CP-SAT backend. At runtime, DCM's scheduler invokes this encoder, constructs the optimisation model, and returns actions applied via the API server.

The considered solver-based proposals either replace the default scheduler or indirectly operate it. A common denominator of all these approaches is the renowned trade-off of solver-based scheduling: computing online cluster-wide placements requires solving an NP-hard combinatorial problem, which can endure high latency. OPSche builds on a solver-based approach, but it is an upstream-compatible plugin that smoothly integration with Kubernetes default scheduler to minimize the number of calls to the external solver and use as much as possible the default scheduler's logic to make placement decisions.

6 Conclusions

We presented OPSche, a Kubernetes scheduler plugin that enables an external optimisation solver to drive cluster-wide placement decisions while reusing the default Kubernetes scheduler as-is. Unlike prior work that replaces or sidesteps the default scheduler, OPSche extends it via the Kubernetes Scheduling Framework, achieving clean integration with no modifications to upstream scheduler code.

Our experiments show that OPSche consistently improves resource usage without any major drawback and the periodic and stable-queue modes reduce scheduling latency in most settings. The non-blocking variant achieves resource usage comparable to the blocking variant while reducing interference with normal scheduling, which makes it a sound default choice. Our ablation studies confirm that these results are robust across a range of trigger intervals and idle-window durations.

Our evaluation instantiated OPSche with a complete, NP-hard bin-packing formulation solved via Google OR-Tools CP-SAT, deliberately stressing the integration mechanism rather than optimising for speed. A natural and direct extension is to replace this solver with faster, incomplete approaches – such as local search heuristics or metaheuristic methods – which would reduce per-invocation latency and expand the operating coverage of OPSche to larger clusters and higher-arrival-rate regimes. In this regard, we view metaheuristic solvers as promising drop-in replacements one can integrate without changes to the plugin architecture.

The current design is solver-agnostic but does not exploit runtime signals beyond the cluster snapshot. Integrating machine-learning forecasts of workload arrival rates, pod lifetimes, or resource demands as soft constraints or objective modifiers could allow the solver to make anticipatory placement decisions, reducing reactive disruption and further improving long-term resource efficiency.

References

- [1] Erika Ábrahám, Florian Corzilius, Einar Broch Johnsen, Gereon Kremer, and Jacopo Mauro. 2016. Zephyrus2: On the Fly Deployment Optimization Using SMT and CP Technologies. In *Dependable Software Engineering: Theories, Tools, and Applications*, Martin Fränzle, Deepak Kapur, and Naijun Zhan (Eds.). Springer International Publishing, Cham, 229–245. doi:10.1007/978-3-319-47677-3_15
- [2] Imtiaz Ahmad, Mohammad Gh. AlFailakawi, Asayel AlMutawa, and Latifa Alsaman. 2022. Container scheduling techniques: A Survey and assessment. *Journal of King Saud University - Computer and Information Sciences* 34, 7 (July 2022), 3934–3947. doi:10.1016/j.jksuci.2021.03.002
- [3] Alibaba. 2025. Alibaba/cluster-trace-gpu-v2025. <https://github.com/alibaba/clusterdata/tree/master/cluster-trace-gpu-v2025>
- [4] Carmen Carrión. 2023. Kubernetes Scheduling: Taxonomy, Ongoing Issues and Challenges. *ACM Comput. Surv.* 55, 7 (July 2023), 1–37. doi:10.1145/3539606
- [5] CAST AI. 2025. 2025 Kubernetes Cost Benchmark Report. <https://cast.ai/reports/kubernetes-cost-benchmark/>.

- [6] Yue Cheng, Ali Anwar, and Xuejing Duan. 2018. Analyzing Alibaba's Co-located Datacenter Workloads. In *IEEE International Conference on Big Data (IEEE BigData 2018)*, Seattle, WA, USA, December 10-13, 2018, Naoki Abe, Huan Liu, Calton Pu, Xiaohua Hu, Nesreen K. Ahmed, Mu Qiao, Yang Song, Donald Kossmann, Bing Liu, Kisung Lee, Jiliang Tang, Jingrui He, and Jeffrey S. Saltz (Eds.). IEEE, 292–297. doi:10.1109/BIGDATA.2018.8622518
- [7] Henrik Daniel Christensen, Saverio Giallorenzo, and Jacopo Mauro. 2025. Priority Matters: Optimising Kubernetes Clusters Usage with Constraint-Based Pod Packing. doi:10.48550/arXiv.2511.08373 arXiv:2511.08373 [cs].
- [8] CNCF [Cloud Native Computing Foundation]. 2023. Kubernetes and unexpected cloud spend: taming, not riding, the tiger. (Nov. 2023). <https://www.cncf.io/wp-content/uploads/2023/12/CNCF%5FFinops-Microsurvey-2023.pdf>
- [9] Yuqi Fu, Shaolun Zhang, Jose Terrero, Ying Mao, Guangya Liu, Sheng Li, and Dingwen Tao. 2019. ProCon: Progress-based Container Scheduling for Short-lived Applications in a Kubernetes Cluster. In *2019 IEEE International Conference on Big Data (Big Data)*. 278–287. doi:10.1109/BigData47090.2019.9006427
- [10] Google Developers. 2025. Google/ClusterData2019. <https://github.com/google/cluster-data/blob/master/ClusterData2019.md>
- [11] Stephen Hendrick. 2025. *Cloud Native 2024: Approaching a Decade of Code, Cloud, and Change*. Technical Report. Cloud Native Computing Foundation (CNCF) / The Linux Foundation. <https://www.cncf.io/wp-content/uploads/2025/04/cncf%5FAnnual%5FSurvey24%5F031225a.pdf> CNCF Annual Survey 2024. Accessed: 2026-03.
- [12] Zhaolong Jian, Xueshuo Xie, Yaozheng Fang, Yibing Jiang, Ye Lu, Ankan Dash, Tao Li, and Guiling Wang. 2024. DRS: A deep reinforcement learning enhanced Kubernetes scheduler for microservice-based system. *Software: Practice and Experience* 54, 10 (2024), 2102–2126. doi:10.1002/spe.3284
- [13] Kubernetes Developers. 2024. Scheduling Framework. <https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/> Section: docs.
- [14] Kubernetes Developers. 2025. KWOK: Kubernetes WithOut Kubelet. <https://kwok.sigs.k8s.io/>
- [15] Torgeir Lebesbye, Jacopo Mauro, Gianluca Turin, and Ingrid Chieh Yu. 2021. Boreas – A Service Scheduler for Optimal Kubernetes Deployment. In *Service-Oriented Computing*, Hakim Hacid, Odej Kao, Massimo Mecella, Naouel Moha, and Hye-young Paik (Eds.). Vol. 13121. Springer International Publishing, Cham, 221–237. doi:10.1007/978-3-030-91431-8_14
- [16] John D. C. Little. 1961. A Proof for the Queuing Formula: $L = \lambda W$. *Operations Research* 9, 3 (1961), 383–387. <https://www.jstor.org/proxy1-bib.sdu.dk:2048/stable/167570> Publisher: INFORMS.
- [17] Guozhi Liu, Weiwei Lin, Haotong Zhang, Jianpeng Lin, Shaoliang Peng, and Keqin Li. 2025. Public Datasets for Cloud Computing: A Comprehensive Survey. *ACM Comput. Surv.* 57, 8 (March 2025), 198:1–198:38. doi:10.1145/3719003
- [18] Vlad-Ioan Luca and Mădălina Eraqcu. 2023. SAGE - A Tool for Optimal Deployments in Kubernetes Clusters. In *2023 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. 10–17. doi:10.1109/CloudCom59040.2023.00016
- [19] MiniZinc Team. 2025. MiniZinc Challenge. <https://www.minizinc.org/challenge/>.
- [20] Pedro Moritz de Carvalho Neto, Márcio Castro, and Frank Siqueira. 2025. Dynamic Load Balancing in Kubernetes Environments With Kubernetes Scheduling Extension (KSE). *Concurrency and Computation: Practice and Experience* 37, 3 (2025), e8344. doi:10.1002/cpe.8344
- [21] Laurent Perron and Vincent Furnon. 2024. *OR-Tools*. Google.
- [22] Yousef Qawqzeh, Mafawez T. Alharbi, Ayman Jaradat, and Khalid Nazim Abdul Sattar. 2021. A review of swarm intelligence algorithms deployment for scheduling and optimization in cloud computing environments. *PeerJ Comput. Sci.* 7 (Aug. 2021), e696. doi:10.7717/peerj-cs.696
- [23] Jing Qin. 2017. Examples and Basic Theories for Length Biased Sampling Problems. In *Biased Sampling, Over-identified Parameter Problems and Beyond*, Jing Qin (Ed.). Springer, Singapore, 1–9. doi:10.1007/978-981-10-4856-2_1
- [24] Charles Reiss, Alexey Tumanov, Gregory R. Ganger, Randy H. Katz, and Michael A. Kozuch. 2012. Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In *Proceedings of the Third ACM Symposium on Cloud Computing (SoCC '12)*. Association for Computing Machinery, New York, NY, USA, 1–13. doi:10.1145/2391229.2391236
- [25] Zeineb Rejiba and Javad Chamanara. 2023. Custom Scheduling in Kubernetes: A Survey on Common Problems and Solution Approaches. *ACM Comput. Surv.* 55, 7 (July 2023), 1–37. doi:10.1145/3544788
- [26] Khaldoun Senjab, Sohail Abbas, Naveed Ahmed, and Atta Ur Rehman Khan. 2023. A survey of Kubernetes scheduling algorithms. *J Cloud Comp* 12, 1 (June 2023), 87. doi:10.1186/s13677-023-00471-1
- [27] Lalith Suresh, João Loff, Faria Kalim, Sangeetha Abdu Jyothi, UC Irvine, Nina Narodytska, Leonid Ryzhyk, Sahan Gamage, Brian Oki, Pranshu Jain, and Michael Gasch. 2020. DCM - Building Scalable and Flexible Cluster Managers Using Declarative Programming. (Nov. 2020).
- [28] Zhang Wei-guo, Ma Xi-lin, and Zhang Jin-zhong. 2018. Research on Kubernetes' Resource Scheduling Scheme. In *Proceedings of the 8th International Conference on Communication and Network Security*. ACM, Qingdao China, 144–148. doi:10.1145/3290480.3290507
- [29] Ying Yang and Lijun Chen. 2019. Design of Kubernetes Scheduling Strategy Based on LSTM and Grey Model. In *2019 IEEE 14th International Conference on Intelligent Systems and Knowledge Engineering (ISKE)*. 701–707. doi:10.1109/ISKE47853.2019.9170419
- [30] Zheqi Zhang, Yaling Xun, Haifeng Yang, and Jianghui Cai. 2025. ATASL: Application type awareness pod-level and system-level container scheduling. *Future Generation Computer Systems* 173 (Dec. 2025), 107898. doi:10.1016/j.future.2025.107898

A Algorithms

In this appendix, we detailed the algorithms that are used in our system design.

A.1 Optimisation Flow Procedure

Algorithm 1 presents the entry point of the optimisation flow. The procedure is atomically guarded so at most one run is in progress at a time (line 2). In *blocking* mode, scheduling is paused before the snapshot (lines 4–5); in *non-blocking* mode, it is paused only after validation (lines 12–13). The snapshot is taken (line 6) and the run is skipped if no pods are pending (lines 7–8). The optimiser runs under a wall-clock timeout (line 9) and the resulting plan is validated against a fresh snapshot (lines 10–11). If valid, the plan is installed, making it visible to scheduling workers (line 14), evictions are carried out (lines 15–16), planned pods are activated (line 17), and an asynchronous convergence monitor is started (line 18). TEARDOWNRUN (lines 20–22) clears the plan, unblocks scheduling, and requeues held-back pods.

Algorithm 1 Optimisation Flow Procedure.

```

1: procedure OPTIMIZATIONFLOW(blockingOptimizer,
  timeoutOptimizer)
2:   if OPTIMIZATIONINPROGRESS() then
3:     return  $\triangleright$  already optimising; skip this run
4:   if blockingOptimizer then
5:     BLOCKSCHEDULING()  $\triangleright$  blocking: block scheduling
      before taking snapshot
6:   state  $\leftarrow$  STATESNAPSHOT()
7:   if not PODSPENDING(state) then
8:     return TEARDOWNRUN()  $\triangleright$  no pending pods
9:   plan = (placements, moves, evictions)  $\leftarrow$ 
    RUNOPTIMIZER(state, timeoutOptimizer)
10:  if not VALIDPLAN(plan, state) then
11:    return TEARDOWNRUN()  $\triangleright$  plan failed validation
12:  if not blockingOptimizer then
13:    BLOCKSCHEDULING()  $\triangleright$  non-blocking: first block
      scheduling here before enforcing plan
14:  INSTALLPLAN(plan)  $\triangleright$  make plan visible to workers
15:  if not EVICTPODS(evictions  $\cup$  moves) then
16:    return TEARDOWNRUN()  $\triangleright$  evict failed or timeout
17:  ACTIVATEPLANNEDPODS(placements)  $\triangleright$  enqueue
    pending pods selected for placement or moved
18:  STARTASYNC(CONVERGENCEMONITOR())  $\triangleright$  start
    async. monitor plan convergence in background
19:  return plan

20: procedure TEARDOWNRUN()
21:   UNINSTALLPLAN()
22:   UNBLOCKSCHEDULING()

```

A.2 Hook Procedures for Plan Enforcement

Algorithms 2–4 present the hook procedures used to enforce a plan through the scheduling workers.

Admission control (PreEnqueue). While a plan is active, PREENQUEUE blocks pods not covered by the plan (line 2) and records them for later requeuing (lines 3–4).

Algorithm 2 PreEnqueue Hook: Admission Control.

```

1: procedure PREENQUEUE(pod, plan)
2:   if ISPLANACTIVE() and not ISPODPLANNED(pod, plan)
   then
3:     ADDBLOCKEDPOD(pod)
4:     BLOCKPOD(pod)

```

Node constraining (PreFilter). PREFILTER restricts candidate nodes to those permitted by the plan for non-protected pods (lines 2–3); otherwise regular scheduling proceeds.

Algorithm 3 PreFilter Hook: Node Constraining.

```

1: procedure PREFILTER(pod, nodes, plan)
2:   if ISPLANACTIVE() and not ISPODPROTECTED(pod)
   then
3:     CONSTRAINTONODES(pod, nodes, plan)

```

Concurrency-safe reservations (Reserve/Unreserve). RESERVE enforces per-workload quotas for controller-managed pods (line 4); quota exhaustion or a pod-node mismatch causes rejection (line 5). UNRESERVE restores consumed quota if the scheduling attempt later fails (line 7).

Algorithm 4 Reserve and Unreserve Hooks: Concurrency-Safe Scheduling.

```

1: procedure RESERVE(pod, node, plan)
2:   if not ISPLANACTIVE() or ISPODPROTECTED(pod)
   or not HASWORKLOADOWNER(pod) then
3:     return  $\triangleright$  no reservation
4:   if not MAKERESERVATION(pod, node) then
5:     return REJECTPOD(pod)  $\triangleright$  over quota or pod-node
      mismatch
6: procedure UNRESERVE(pod, node, plan)
7:   RETURNRESERVATION(pod, node, plan)

```

A.3 Trigger Modes Procedures

This subsection details the procedures for the three trigger modes supported by OPSche: *scheduling-failure*, which invokes optimisation synchronously in the scheduling path; *periodic*, which runs it at fixed intervals; and *stable-queue*, which defers it until the pending queue stabilises.

A.3.1 Scheduling-Failure Mode. Algorithm 5 presents the *scheduling-failure* trigger. When a pod fails to find a feasible node, the *PostFilter* hook invokes the optimisation flow synchronously (line 2) and, if a plan is produced, nominates a node for the triggering pod (line 3).

Algorithm 5 PostFilter Hook Procedure for Scheduling-Failure Trigger Mode.

```

1: procedure POSTFILTER(pod)
2:   plan ← OPTIMIZATIONFLOW()
3:   NOMINATENODE(plan)

```

A.3.2 Periodic and Stable-Queue Modes. Algorithm 6 presents the loop shared by the *periodic* and *stable-queue* modes. The loop sleeps for the configured interval (line 3), snapshots the cluster state (line 4), and skips the tick if an optimisation is already in progress (line 5). In *stable-queue* mode, an in-progress run is torn down when the pending set changes (lines 6–7). The tick is also skipped if the state has not changed since the last solved run (lines 9–10). In *stable-queue* mode, the pending set must additionally have remained stable for the configured stability window (lines 11–12). If all guards pass, the optimisation flow is started asynchronously (line 13).

Algorithm 6 Background Loop for Periodic and Stable-Queue Trigger Modes.

```

1: procedure BACKGROUNDLOOP(mode, timeoutInterval, timeoutStability)
2:   loop
3:     SLEEP(timeoutInterval)
4:     state ← STATESNAPSHOT()
5:     if ISOPTIMIZATIONINPROGRESS() then
6:       if mode = STABLEQUEUE
7:         and PENDINGCHANGED(state) then
8:         TEARDOWNRUN()
9:       continue ▷ plan being enforced or pending set changed
10:    if ALREADYSOLVED(state) then
11:      continue ▷ state unchanged since last solve
12:    if mode = STABLEQUEUE
13:      and not QUEUESTABLE(timeoutStability) then
14:      continue ▷ wait for stability window
15:    STARTASYNC(OPTIMIZATIONFLOW())

```

A.4 Convergence Procedure for Plan Monitoring

Algorithm 7 presents the convergence monitor, an asynchronous routine started after a plan becomes active. The monitor records its start time (line 2) and periodically sleeps for the configured check interval (line 4). If the plan deadline is exceeded, the run is torn down (lines 5–6). Otherwise, the cluster state is snapshoted (line 7) and convergence is evaluated via two conditions (line 8): STABLEPODSPLACEDOK checks identity-stable pods by verifying that each non-terminated pod is observed on its designated node; missing or terminated pods are treated as satisfied (lines 12–16). UNSTABLEPODSQUOTASOK checks identity-unstable (controller-managed) pods at the workload level, verifying that per-node quotas have been fully consumed or no active replicas remain (lines 19–24). If both conditions hold, the run is torn down and regular scheduling resumes (line 9).

Algorithm 7 Convergence Procedure for Plan Monitoring.

```

1: procedure CONVERGENCEMONITOR(plan, cvrgInterval, timeoutPlan)
2:   startTime ← NOW()
3:   loop
4:     SLEEP(cvrgInterval)
5:     if (NOW() – startTime) ≥ timeoutPlan then
6:       return TEARDOWNRUN() ▷ plan timed out
7:     state ← STATESNAPSHOT()
8:     if STABLEPODSPLACEDOK(state, plan)
9:       and UNSTABLEPODSQUOTASOK(state, plan) then
10:      return TEARDOWNRUN() ▷ plan converged
11: procedure STABLEPODSPLACEDOK(state, placeByName)
12:   for (pod, wantNode) in placeByName do
13:     if ISPODGONE(pod) then
14:       continue ▷ satisfied; terminated or deleted
15:     if not ISPODONNODE(pod, wantNode) then
16:       return false ▷ not satisfied; pod not on desired node
17:   return true ▷ all pods placed on desired nodes
18: procedure UNSTABLEPODSQUOTASOK(state, wkQuotas)
19:   for (wk, remaining) in wkQuotas do
20:     if remaining ≤ 0 or not HASWKACTIVEPODS(wk) then
21:       continue ▷ workload quota satisfied or workload scaled down or deleted
22:     if HASPENDINGPODS(wk) then
23:       return false ▷ not satisfied; pending pods with workload quota remaining
24:   return true ▷ all workload quotas satisfied

```

B Analysis and Plots of Public Traces

The two analyzed public traces—the Alibaba GPU trace [3] and Google Cluster Data 2019 [10]—show similar heavy-tailed patterns in inter-arrival times, workload lifetimes, and resource requests. This is consistent with prior analyses [24]. Resource requests are also skewed, although the Alibaba request histograms appear more discretized. Despite these differences, we model inter-arrivals, lifetimes, and CPU/memory requests using the same bounded Pareto distribution to keep the generator consistent and simple.

For discrete-valued attributes, we use truncated geometric distributions. Priorities—available in the Google trace—are well approximated by a geometric distribution, capturing that most workloads concentrate in a small number of low priority levels while higher levels occur with diminishing frequency. Replica counts are not available in either public trace, so we likewise model them geometrically, reflecting the common pattern that most workloads run with few replicas while a smaller fraction scale out to many.

Figure 4 shows histograms from the public traces along with the fitted distributions used to inform the synthetic generator.

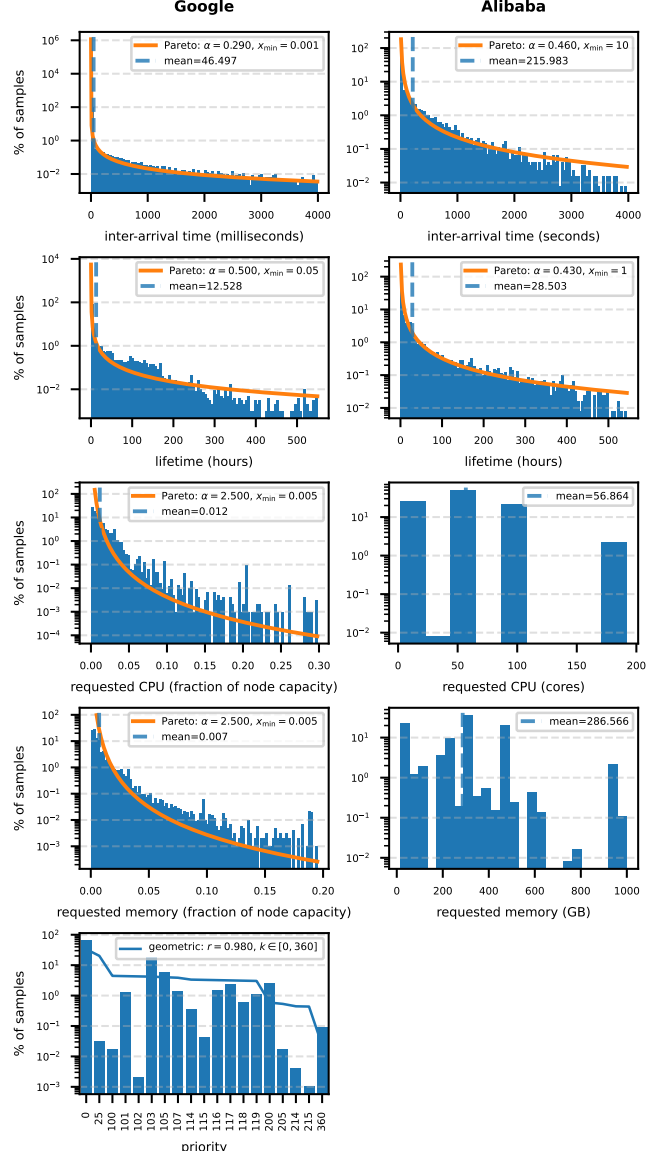


Figure 4. Histograms of pods' inter-arrival times, lifetimes, CPU/memory requests, and priorities from publicly available cluster traces. The orange lines show fitted Pareto distributions for inter-arrival times, lifetimes, and resource requests. Priorities are fitted with a geometric distribution. No priorities are available in the Alibaba trace.

Algorithm 8 Optimisation Algorithm.

```

1:  $m \leftarrow \text{Model}()$ 
2: for  $pr \in \text{range}(p_{\max} + 1)$  do
3:    $m.\text{add\_constraint}(\text{bin\_packing\_constraints}(pr))$ 
4:    $\triangleright$  (1) maximise number of placed pods
5:    $\text{metric} \leftarrow \sum_{p_i \in P: p_i.\text{priority} \leq pr} \sum_{n_j \in N} x_{i,j}$ 
6:    $\text{sol} \leftarrow m.\text{max}(\text{metric}, \text{get\_timeout}())$ 
7:   if  $\text{sol.status} = \text{OPTIMAL}$  then
8:      $m.\text{add\_constraint}(\text{metric} = \text{sol}(\text{metric}))$ 
9:   else
10:     $m.\text{add\_constraint}(\text{metric} \geq \text{sol}(\text{metric}))$ 
11:     $\triangleright$  (2) minimise number of pod moves
12:     $\text{metric} \leftarrow$ 
13:       $\sum_{p_i \in P: p_i.\text{priority} \leq pr} \left( \sum_{n_j \in N} x_{i,j} + 2x_{i,p_i.\text{where}} \right)$ 
14:       $\wedge p_i.\text{where} \neq 0$ 
14:     $\text{sol} \leftarrow m.\text{max}(\text{metric}, \text{get\_timeout}())$ 
15:    if  $\text{sol.status} = \text{OPTIMAL}$  then
16:       $m.\text{add\_constraint}(\text{metric} = \text{sol}(\text{metric}))$ 
17:    else
18:       $m.\text{add\_constraint}(\text{metric} \leq \text{sol}(\text{metric}))$ 
19: return  $\text{sol}$ 

```

C Optimisation Model and External Solver

In this appendix we detail the optimisation model we designed and the external solver we use to implement it.

C.1 Optimisation problem

We solve the optimisation problem incrementally, in a loop that iterates over all priorities. At every priority tier, we run a solver twice to 1) maximise the number of placed pods, and 2) minimise the number of allocated pods we need to move.

Notation: we use $n \in N$ to denote nodes. We define with $n.\text{ram}$ and $n.\text{cpu}$ the capacities of n . For each pod $p \in P$, let $p.\text{ram}$ and $p.\text{cpu}$ be its resource requests, $p.\text{priority} \in [0, p_{\max}]$ its priority level (with lower values denoting higher priority), and $p.\text{where}$ the index of the node where the pod is currently placed ($p.\text{where} = 0$ if the pod is not scheduled). The binary variables $x_{i,j} \in \{0, 1\}$ indicate whether pod p_i is assigned to node n_j .

We report in Algorithm 8 the pseudocode of the optimisation loop, where a model m collects constraints and supports a solver that maximises a given metric.

At Line 1, we initialise the model without constraints. Then, for each priority level pr from the highest (0) to the lowest ($p_{\max}$) we add (multi-dimensional) bin-packing constraints (Line 3) to restrict solutions to valid placements, and use these to (1) maximise the pods allocated up to priority pr (Lines 5–10), and (2) to minimise the removal of deployed pods (Lines 12–18).

Constraints impose that the sum of the requested resources by the pod with priority up to pr are within the respective node's RAM and CPU capacities, and ensures assignment of a pod with priority up to pr to at most one node.

At Line 6 of Algorithm 8, we maximise the number of pods within the current priority by maximising of the metric

$$\sum_{p_i \in P: p_i.\text{priority} \leq pr} \sum_{n_j \in N} x_{i,j}.$$

If the solver finds an OPTIMAL solution, we update the model by adding a constraint that imposes to find future solutions with exactly the same amount of pods up to priority pr (Line 8). If the solver did not terminate within the allocated time, thus not proving that the solution is optimal but providing only a FEASIBLE solution, we add a constraint that imposes to find solutions with at least the same amount of pods up to priority pr (Line 10).

In the second stage, we minimise the disruptions caused by avoidable pod evictions. To do so, we minimise the evictions of allocated pods up to priority pr by asking the solver, at Line 14, to maximise the following metric:

$$\sum_{p_i \in P: p_i.\text{priority} \leq pr \wedge p_i.\text{where} \neq 0} \left(\sum_{n_j \in N} x_{i,j} + 2x_{i,p_i.\text{where}} \right)$$

where we give a weight of 2 to the pods that remain in place (i.e., assigned to the same node they are already allocated onto) and a weight of 1 to the pods to move to another node.

As the optimisation problem is NP-hard, we chose to use a fixed wall-clock timeout T_{total} , which limits the total solving time across all priority tiers. To avoid that a single priority tier exhausts the entire time budget, thus compromising the possibility to find a solution involving also lower priorities pods, we dedicate a fraction $\alpha \in [0, 1]$ of the total time to each priority tier, leaving the remaining $(1 - \alpha)T_{\text{total}}$ time for the opportunistic termination of the solver executions. Moreover, for every priority, we split in half the reserved time between the two solving phases (maximisation of pods and minimisation of evictions). In particular, in Algorithm 8, we use function `get_timeout` at every solver invocation (Line 6 and 14) to compute the available time budget. Formally, assuming that `unused` is the amount of unreserved time yet to consume, $\alpha \in [0, 1]$

$$\text{get_timeout}() = \alpha T_{\text{total}} / (p_{\max} + 1) + \text{unused}$$

C.2 Solver

We implement our approach by using OR-Tools [21] – an open-source suite developed by Google for optimisation tasks using, among other approaches, a lazy-constraint approach. Notably, OR-Tools is the winner of the latest MiniZinc Challenge [19], a competition evaluating the performance of constraint programming solvers across diverse benchmarks. In particular, we use the OR-Tools' CP-SAT Python API, which runs several complementary search strategies in parallel. Since, unlike popular SMT solvers (e.g., Z3), CP-SAT does not support incremental push/pop of constraints, we re-solve the model after each place/move step. To try to

speed up the search, we use CP-SAT hints to warm-start the next solve by providing the current assignment.

We integrate the solver and its optimisation model into OPSche via the Scheduling Framework, where OPSche calls a Python script that invokes OR-Tools deployed inside the scheduler container.

D Results

This appendix complements the results in Section 4.3 with: (i) figures for *blocking* variants and with *DefaultPreemption* disabled, (ii) sensitivity figures for the *periodic* interval and *stable-queue* idle window, (iii) tables reporting the mean and standard deviation of each metric for every mode and trace setting, and (iv) a comparison of OPSche’s modes with *DefaultPreemption* enabled vs. disabled.

The appendix is structured as follows: Section D.1 presents results with *DefaultPreemption* enabled. Section D.2 presents results with *DefaultPreemption* disabled. Finally, Section D.3 compares OPSche’s modes with *DefaultPreemption* enabled vs. disabled.

D.1 Results with DefaultPreemption enabled

This subsection presents additional results for OPSche’s modes when running with *DefaultPreemption* enabled.

Figure 5 shows the results for *blocking* variants of OPSche’s modes with *DefaultPreemption* enabled.

Figure 6 compares OPSche’s modes with *non-blocking* vs. *blocking* variants when running with *DefaultPreemption* enabled.

Figures 7 and 8 shows how the *periodic* interval and *stable-queue* idle window affect usage, scheduling latency, solver runs and plan activations. Each mode is evaluated with shorter (4 s) and longer (16 s) settings relative to the default 8 s, using the *non-blocking* and *blocking* variants, respectively, with *DefaultPreemption* enabled.

Tables 5–14, report the mean and standard deviation of each metric for each mode and trace setting when running with *DefaultPreemption* enabled containing all runs (*non-blocking* and *blocking* variants) for both priority configurations (one and four priority levels).

D.2 Results with DefaultPreemption disabled

This subsection reports results for OPSche’s modes with *DefaultPreemption* disabled. The same modes are run, as in Section 4.3, but the default scheduler is prevented from preempting pods in *PostFilter*. This isolates the impact of OPSche’s preemption mechanism.

Figures 9 and 10 summarises results when running OPSche’s modes in *non-blocking* and *blocking* variants with *DefaultPreemption* disabled, respectively.

Figure 11 compares OPSche’s modes with *non-blocking* vs. *blocking* variants when running with *DefaultPreemption* disabled.

Figures 12 and 13 reports the sensitivity of the *periodic* and *stable-queue* modes to their interval and idle window, respectively, when running with *non-blocking* and *blocking* variants, respectively.

Tables 15–24, report the mean and standard deviation of each metric for each mode and trace setting when running with *DefaultPreemption* disabled containing all runs

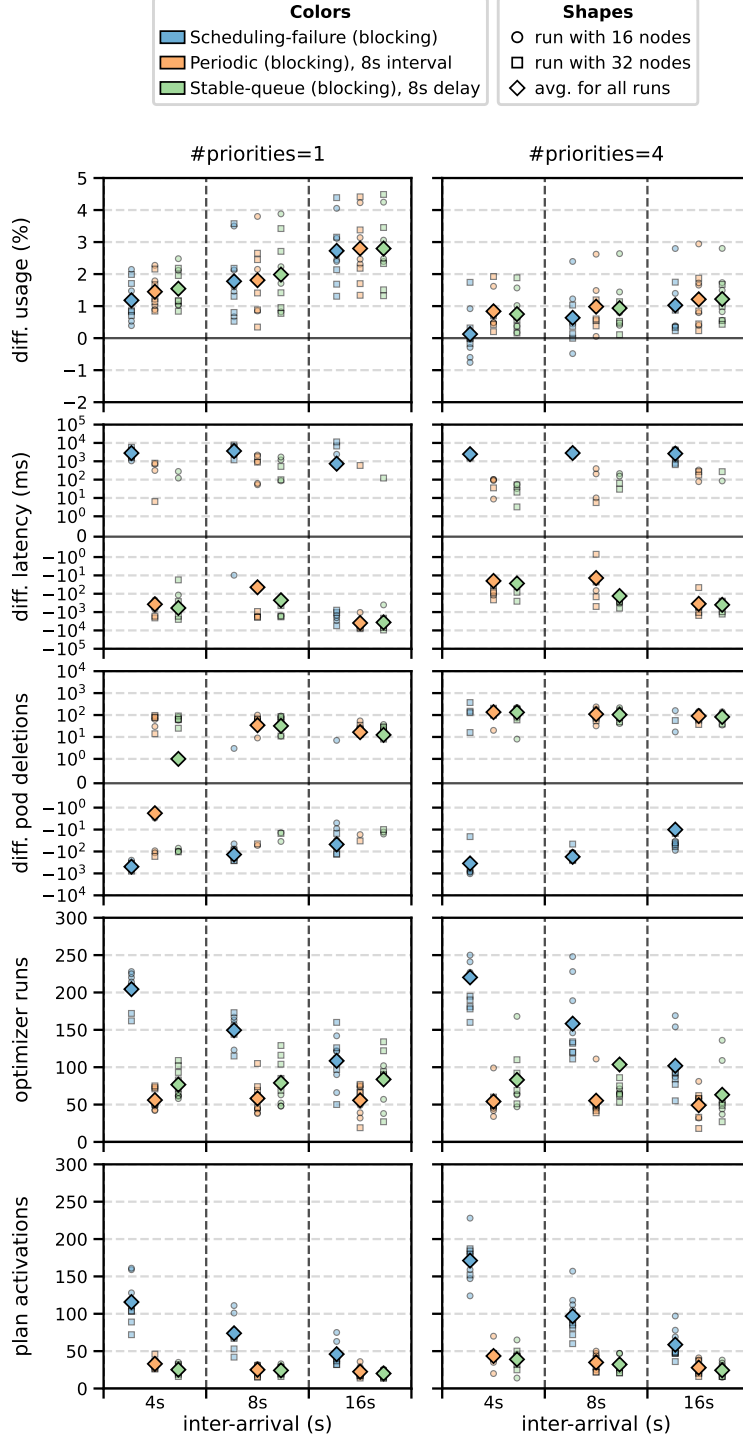


Figure 5. Comparison of OPSche modalities with *blocking* variants and *DefaultPreemption* enabled. Each point shows the difference (OPSche’s mode minus default) for a single run (circles: 16 nodes; squares: 32 nodes), and diamonds denote the mean difference per trace setting. Left column: one priority level; right column: four priority levels. Rows report effective usage (%), scheduling latency (ms), pod deletions, solver runs, and plan activations.

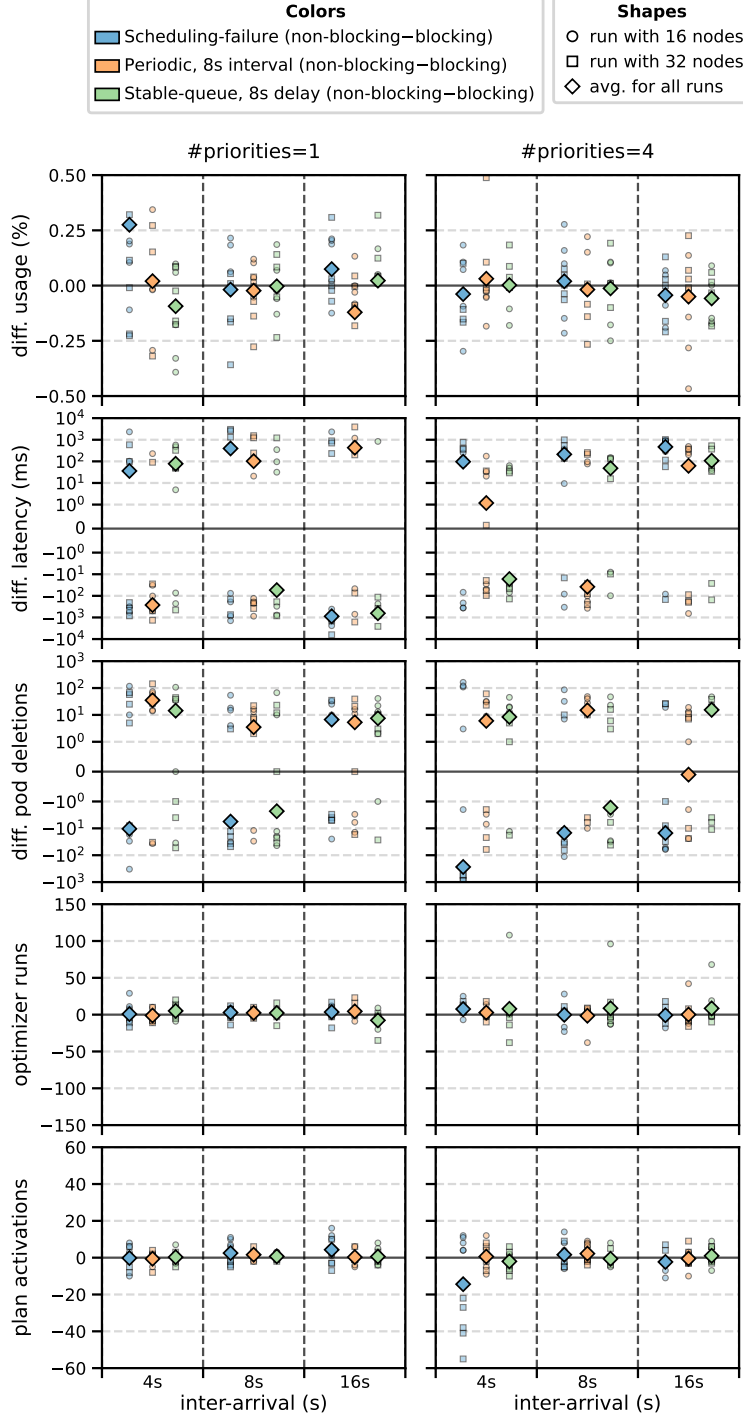


Figure 6. Comparison of OPSche modalities (*non-blocking* vs. *blocking*; *DefaultPreemption* enabled). Each point shows the difference (*non-blocking* minus *blocking*) for a single run (circles: 16 nodes; squares: 32 nodes), and diamonds denote the mean difference per trace setting (mean across five seeds). Left column: one priority level; right column: four priority levels. Rows report effective usage (%), scheduling latency (ms), pod deletions, solver runs, and plan activations.

(*non-blocking* and *blocking* variants) for both priority configurations (one and four priority levels).

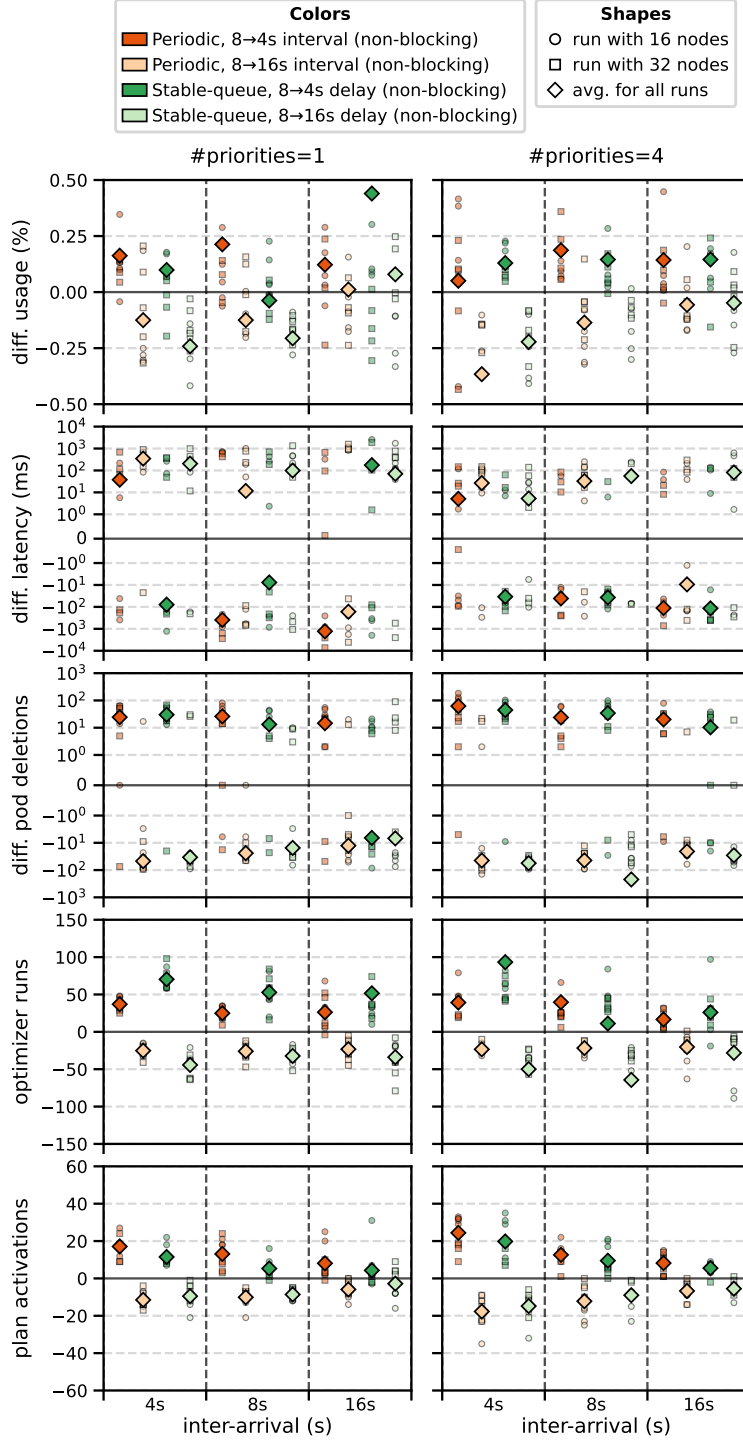


Figure 7. Comparison of OPSche modalities (*non-blocking*; *DefaultPreemption* enabled) with a sensitivity analysis of the *periodic* interval and *stable-queue* idle window. The sensitivity setting compares a shorter (4 s) and longer (16 s) interval/idle window against the default 8 s configuration (8 s→4 s and 8 s→16 s). Each point shows the difference (OPSche minus default) for a single run (circles: 16 nodes; squares: 32 nodes), and diamonds denote the mean difference per trace setting (mean across five seeds). Left column: one priority level; right column: four priority levels. Rows report effective usage (%), scheduling latency (ms), pod deletions, solver runs, and plan activations.

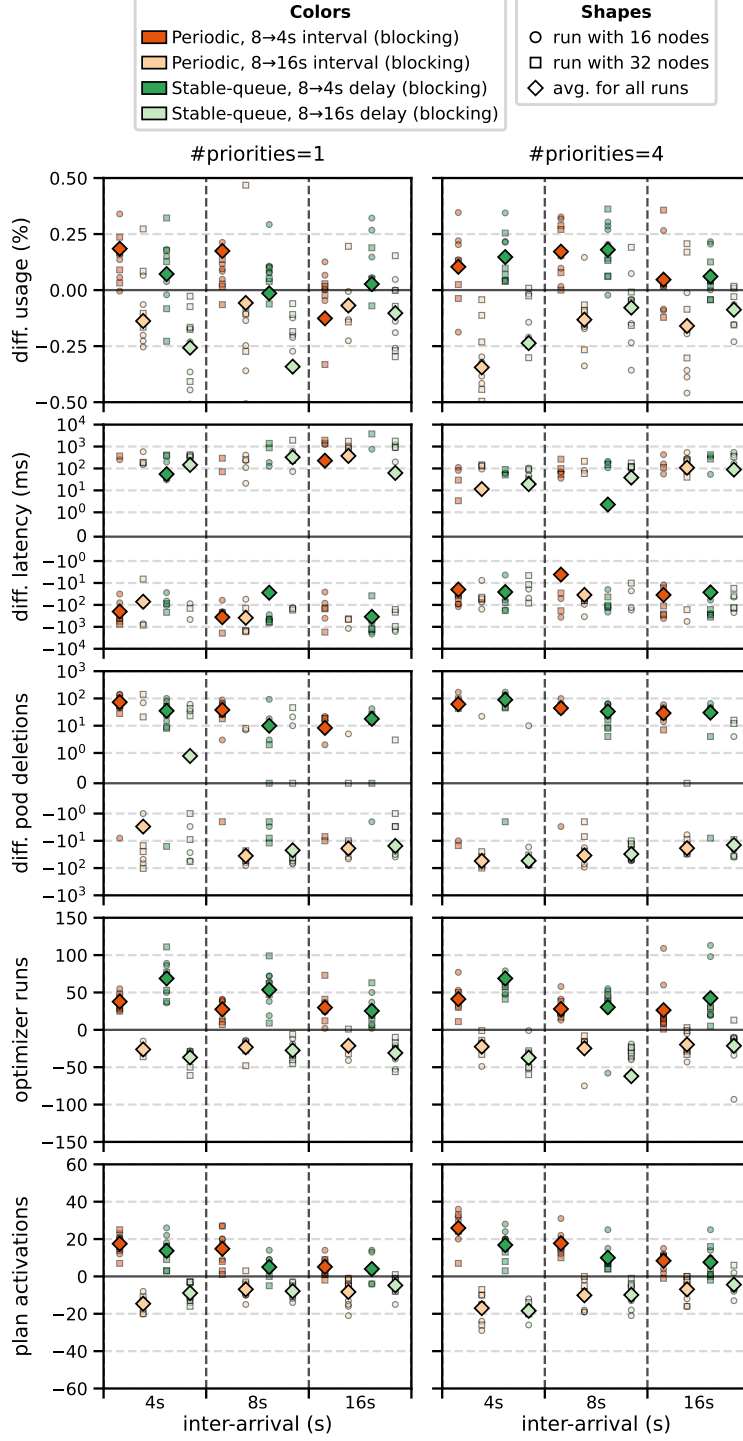


Figure 8. Comparison of OPSche modalities (*blocking*; *DefaultPreemption* enabled) with a sensitivity analysis of the *periodic* interval and *stable-queue* idle window. The sensitivity setting compares a shorter (4 s) and longer (16 s) interval/idle window against the default 8 s configuration (8 s→4 s and 8 s→16 s). Each point shows the difference (OPSche minus default) for a single run (circles: 16 nodes; squares: 32 nodes), and diamonds denote the mean difference per trace setting (mean across five seeds). Left column: one priority level; right column: four priority levels. Rows report effective usage (%), scheduling latency (ms), pod deletions, solver runs, and plan activations.

Table 5. Mean paired differences in effective resource usage (%) between the plugin with DefaultPreemption enabled and the default scheduler for runs with one priority (mean $\pm$ std).

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	+1.1 $\pm$ 0.7	+1.9 $\pm$ 1.1	+2.9 $\pm$ 0.7	+1.2 $\pm$ 0.6	+1.7 $\pm$ 1.2	+2.5 $\pm$ 1.2
Scheduling-failure (non-blocking)	+1.7 $\pm$ 1.4	+2.0 $\pm$ 1.0	+3.0 $\pm$ 0.7	+1.2 $\pm$ 0.4	+1.6 $\pm$ 1.2	+2.6 $\pm$ 1.3
Periodic (blocking), 4s interval	+1.7 $\pm$ 0.8	+2.2 $\pm$ 1.1	+3.0 $\pm$ 0.8	+1.5 $\pm$ 0.5	+1.8 $\pm$ 1.3	+2.3 $\pm$ 1.0
Periodic (non-blocking), 4s interval	+1.6 $\pm$ 0.7	+2.2 $\pm$ 1.1	+3.1 $\pm$ 0.8	+1.6 $\pm$ 0.6	+1.8 $\pm$ 1.3	+2.5 $\pm$ 1.3
Periodic (blocking), 8s interval	+1.5 $\pm$ 0.6	+2.1 $\pm$ 1.1	+3.0 $\pm$ 0.8	+1.4 $\pm$ 0.5	+1.5 $\pm$ 1.0	+2.6 $\pm$ 1.3
Periodic (non-blocking), 8s interval	+1.5 $\pm$ 0.8	+2.1 $\pm$ 1.1	+3.0 $\pm$ 0.7	+1.4 $\pm$ 0.4	+1.5 $\pm$ 0.9	+2.4 $\pm$ 1.3
Periodic (blocking), 16s interval	+1.4 $\pm$ 0.7	+1.8 $\pm$ 1.0	+2.9 $\pm$ 0.7	+1.3 $\pm$ 0.3	+1.7 $\pm$ 1.1	+2.6 $\pm$ 1.3
Periodic (non-blocking), 16s interval	+1.3 $\pm$ 0.8	+1.9 $\pm$ 0.9	+2.9 $\pm$ 0.8	+1.4 $\pm$ 0.5	+1.5 $\pm$ 0.9	+2.5 $\pm$ 1.3
Stable-queue (blocking), 4s delay	+1.7 $\pm$ 0.7	+2.2 $\pm$ 1.1	+3.1 $\pm$ 0.7	+1.5 $\pm$ 0.4	+1.7 $\pm$ 0.9	+2.5 $\pm$ 1.4
Stable-queue (non-blocking), 4s delay	+1.7 $\pm$ 0.8	+2.2 $\pm$ 1.1	+3.9 $\pm$ 2.4	+1.4 $\pm$ 0.4	+1.7 $\pm$ 0.9	+2.6 $\pm$ 1.3
Stable-queue (blocking), 8s delay	+1.6 $\pm$ 0.7	+2.1 $\pm$ 1.1	+3.0 $\pm$ 0.8	+1.5 $\pm$ 0.6	+1.9 $\pm$ 1.2	+2.6 $\pm$ 1.3
Stable-queue (non-blocking), 8s delay	+1.5 $\pm$ 0.8	+2.1 $\pm$ 1.1	+2.9 $\pm$ 0.5	+1.4 $\pm$ 0.5	+1.8 $\pm$ 1.1	+2.7 $\pm$ 1.3
Stable-queue (blocking), 16s delay	+1.3 $\pm$ 0.6	+1.8 $\pm$ 0.9	+2.9 $\pm$ 0.8	+1.3 $\pm$ 0.5	+1.5 $\pm$ 1.1	+2.5 $\pm$ 1.3
Stable-queue (non-blocking), 16s delay	+1.1 $\pm$ 0.8	+1.9 $\pm$ 0.9	+2.9 $\pm$ 0.9	+1.3 $\pm$ 0.5	+1.7 $\pm$ 1.1	+2.9 $\pm$ 1.4

Table 6. Mean paired differences in effective resource usage (%) between the plugin with DefaultPreemption enabled and the default scheduler for runs with 4 priorities (mean $\pm$ std). p1 is the lowest priority.

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	-0.2 $\pm$ 0.7	+0.8 $\pm$ 1.1	+1.2 $\pm$ 1.0	+0.4 $\pm$ 0.8	+0.4 $\pm$ 0.4	+0.8 $\pm$ 0.6
Scheduling-failure (non-blocking)	-0.2 $\pm$ 0.7	+0.9 $\pm$ 1.1	+1.2 $\pm$ 1.0	+0.3 $\pm$ 0.7	+0.4 $\pm$ 0.4	+0.8 $\pm$ 0.7
Periodic (blocking), 4s interval	+1.0 $\pm$ 0.6	+1.4 $\pm$ 1.0	+1.5 $\pm$ 0.9	+0.9 $\pm$ 0.7	+0.9 $\pm$ 0.4	+1.0 $\pm$ 0.5
Periodic (non-blocking), 4s interval	+0.9 $\pm$ 0.7	+1.5 $\pm$ 1.1	+1.6 $\pm$ 1.0	+0.9 $\pm$ 0.7	+0.8 $\pm$ 0.3	+1.0 $\pm$ 0.6
Periodic (blocking), 8s interval	+0.9 $\pm$ 0.5	+1.2 $\pm$ 0.9	+1.5 $\pm$ 1.0	+0.8 $\pm$ 0.7	+0.8 $\pm$ 0.3	+0.9 $\pm$ 0.7
Periodic (non-blocking), 8s interval	+0.8 $\pm$ 0.5	+1.3 $\pm$ 0.9	+1.3 $\pm$ 0.9	+0.9 $\pm$ 0.7	+0.7 $\pm$ 0.4	+1.0 $\pm$ 0.6
Periodic (blocking), 16s interval	+0.4 $\pm$ 0.3	+1.1 $\pm$ 0.9	+1.2 $\pm$ 0.9	+0.6 $\pm$ 0.5	+0.6 $\pm$ 0.3	+0.9 $\pm$ 0.5
Periodic (non-blocking), 16s interval	+0.4 $\pm$ 0.3	+1.1 $\pm$ 0.9	+1.3 $\pm$ 0.9	+0.6 $\pm$ 0.7	+0.6 $\pm$ 0.3	+0.9 $\pm$ 0.6
Stable-queue (blocking), 4s delay	+1.0 $\pm$ 0.6	+1.4 $\pm$ 0.9	+1.6 $\pm$ 0.9	+0.8 $\pm$ 0.7	+0.8 $\pm$ 0.4	+1.0 $\pm$ 0.5
Stable-queue (non-blocking), 4s delay	+0.9 $\pm$ 0.5	+1.4 $\pm$ 1.0	+1.6 $\pm$ 1.0	+0.9 $\pm$ 0.7	+0.7 $\pm$ 0.4	+1.0 $\pm$ 0.6
Stable-queue (blocking), 8s delay	+0.8 $\pm$ 0.6	+1.2 $\pm$ 0.9	+1.5 $\pm$ 0.9	+0.7 $\pm$ 0.7	+0.6 $\pm$ 0.4	+1.0 $\pm$ 0.5
Stable-queue (non-blocking), 8s delay	+0.7 $\pm$ 0.5	+1.2 $\pm$ 0.8	+1.4 $\pm$ 1.0	+0.8 $\pm$ 0.7	+0.7 $\pm$ 0.3	+0.9 $\pm$ 0.5
Stable-queue (blocking), 16s delay	+0.5 $\pm$ 0.3	+1.1 $\pm$ 0.9	+1.4 $\pm$ 0.9	+0.6 $\pm$ 0.6	+0.6 $\pm$ 0.3	+0.9 $\pm$ 0.5
Stable-queue (non-blocking), 16s delay	+0.4 $\pm$ 0.4	+1.0 $\pm$ 0.8	+1.3 $\pm$ 1.0	+0.6 $\pm$ 0.6	-17.1 $\pm$ 39.7	+0.9 $\pm$ 0.6

D.3 Results for DefaultPreemption Enabled vs. Disabled

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009

Figure 14 compares OPSche’s modes with *DefaultPreemption* enabled vs. disabled when running with *non-blocking* and *blocking* variants, respectively.

Table 7. Mean paired differences in scheduling latency (ms) between the plugin with DefaultPreemption enabled and the default scheduler for runs with one priority (mean $\pm$ std).

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	+2487 $\pm$ 1089	+2966 $\pm$ 2214	-679 $\pm$ 2186	+3100 $\pm$ 1746	+4283 $\pm$ 2644	+2156 $\pm$ 6819
Scheduling-failure (non-blocking)	+2725 $\pm$ 1793	+2548 $\pm$ 1703	-1258 $\pm$ 2069	+2935 $\pm$ 2137	+5491 $\pm$ 3312	+965 $\pm$ 5276
Periodic (blocking), 4s interval	-146 $\pm$ 451	+29 $\pm$ 1353	-2864 $\pm$ 1375	-1008 $\pm$ 948	-846 $\pm$ 1657	-4552 $\pm$ 4468
Periodic (non-blocking), 4s interval	-208 $\pm$ 497	+38 $\pm$ 1632	-3551 $\pm$ 1528	-1000 $\pm$ 1027	-707 $\pm$ 1272	-6048 $\pm$ 3682
Periodic (blocking), 8s interval	-59 $\pm$ 571	+312 $\pm$ 1417	-3052 $\pm$ 1203	-692 $\pm$ 1184	-401 $\pm$ 1662	-4817 $\pm$ 3359
Periodic (non-blocking), 8s interval	-174 $\pm$ 423	+32 $\pm$ 1401	-2777 $\pm$ 1382	-1109 $\pm$ 850	+83 $\pm$ 2143	-4235 $\pm$ 3271
Periodic (blocking), 16s interval	-89 $\pm$ 423	+455 $\pm$ 1431	-2697 $\pm$ 1711	-805 $\pm$ 852	-1317 $\pm$ 1354	-4424 $\pm$ 3608
Periodic (non-blocking), 16s interval	+174 $\pm$ 519	+139 $\pm$ 1414	-2966 $\pm$ 2438	-763 $\pm$ 1000	-0 $\pm$ 1711	-4378 $\pm$ 3504
Stable-queue (blocking), 4s delay	-186 $\pm$ 484	+32 $\pm$ 1383	-3400 $\pm$ 1887	-902 $\pm$ 797	-541 $\pm$ 1183	-4602 $\pm$ 2749
Stable-queue (non-blocking), 4s delay	-296 $\pm$ 610	-2 $\pm$ 1169	-2895 $\pm$ 2473	-905 $\pm$ 868	-575 $\pm$ 1033	-5338 $\pm$ 4059
Stable-queue (blocking), 8s delay	-192 $\pm$ 430	+205 $\pm$ 1311	-2553 $\pm$ 1235	-1007 $\pm$ 1052	-658 $\pm$ 1042	-4751 $\pm$ 3805
Stable-queue (non-blocking), 8s delay	-43 $\pm$ 609	+83 $\pm$ 1324	-2889 $\pm$ 1302	-1000 $\pm$ 921	-645 $\pm$ 1188	-5695 $\pm$ 3359
Stable-queue (blocking), 16s delay	-124 $\pm$ 497	+412 $\pm$ 1338	-2695 $\pm$ 2339	-785 $\pm$ 931	-215 $\pm$ 1824	-4485 $\pm$ 4094
Stable-queue (non-blocking), 16s delay	+59 $\pm$ 526	+284 $\pm$ 1510	-2443 $\pm$ 1905	-695 $\pm$ 1075	-648 $\pm$ 1554	-5999 $\pm$ 4070

Table 8. Mean paired differences in scheduling latency (ms) between the plugin with DefaultPreemption enabled and the default scheduler for runs with 4 priorities (mean $\pm$ std). p1 is the lowest priority.

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	total: +2722 $\pm$ 436 p1: +2871 $\pm$ 591 p2: +2693 $\pm$ 412 p3: +2549 $\pm$ 545 p4: +2627 $\pm$ 349	total: +2773 $\pm$ 600 p1: +2676 $\pm$ 1098 p2: +2740 $\pm$ 606 p3: +2688 $\pm$ 533 p4: +2914 $\pm$ 1059	total: +2778 $\pm$ 1212 p1: +1715 $\pm$ 2229 p2: +3308 $\pm$ 956 p3: +2926 $\pm$ 747 p4: +3181 $\pm$ 751	total: +2167 $\pm$ 508 p1: +1801 $\pm$ 403 p2: +2459 $\pm$ 669 p3: +2200 $\pm$ 564 p4: +2285 $\pm$ 659	total: +2855 $\pm$ 576 p1: +2758 $\pm$ 1120 p2: +3082 $\pm$ 792 p3: +2620 $\pm$ 356 p4: +2871 $\pm$ 603	total: +2467 $\pm$ 1625 p1: +1485 $\pm$ 3770 p2: +3262 $\pm$ 1335 p3: +2680 $\pm$ 749 p4: +2493 $\pm$ 784
Scheduling-failure (non-blocking)	total: +2569 $\pm$ 558 p1: +2465 $\pm$ 888 p2: +2813 $\pm$ 561 p3: +2412 $\pm$ 678 p4: +2463 $\pm$ 213	total: +2829 $\pm$ 806 p1: +2812 $\pm$ 1581 p2: +2765 $\pm$ 662 p3: +2556 $\pm$ 454 p4: +3048 $\pm$ 1178	total: +3329 $\pm$ 1317 p1: +2691 $\pm$ 2728 p2: +3923 $\pm$ 1236 p3: +3160 $\pm$ 1188 p4: +3289 $\pm$ 599	total: +2513 $\pm$ 407 p1: +2534 $\pm$ 738 p2: +2650 $\pm$ 535 p3: +2415 $\pm$ 572 p4: +2377 $\pm$ 408	total: +3222 $\pm$ 912 p1: +3843 $\pm$ 1735 p2: +3165 $\pm$ 917 p3: +2681 $\pm$ 777 p4: +2754 $\pm$ 685	total: +2849 $\pm$ 2076 p1: +2581 $\pm$ 4929 p2: +3266 $\pm$ 1463 p3: +2655 $\pm$ 639 p4: +2731 $\pm$ 1286
Periodic (blocking)	total: +13 $\pm$ 63 p1: -87 $\pm$ 160	total: +55 $\pm$ 245 p1: +30 $\pm$ 784	total: -232 $\pm$ 232 p1: -955 $\pm$ 697	total: -93 $\pm$ 103 p1: -366 $\pm$ 319	total: -91 $\pm$ 149 p1: -451 $\pm$ 460	total: -547 $\pm$ 750 p1: -1841 $\pm$ 2405
4s interval	p2: +51 $\pm$ 38 p3: +47 $\pm$ 64 p4: +47 $\pm$ 42	p2: +58 $\pm$ 50 p3: +68 $\pm$ 47 p4: +47 $\pm$ 34	p2: +45 $\pm$ 64 p3: +49 $\pm$ 40 p4: +70 $\pm$ 95	p2: +29 $\pm$ 14 p3: +45 $\pm$ 36 p4: +25 $\pm$ 27	p2: +75 $\pm$ 34 p3: +51 $\pm$ 39 p4: +72 $\pm$ 29	p2: +40 $\pm$ 42 p3: +33 $\pm$ 40 p4: +25 $\pm$ 40
Periodic (non-blocking)	total: +55 $\pm$ 107 p1: +52 $\pm$ 259	total: -2 $\pm$ 180 p1: -169 $\pm$ 575	total: -189 $\pm$ 285 p1: -861 $\pm$ 888	total: -83 $\pm$ 134 p1: -335 $\pm$ 405	total: -184 $\pm$ 224 p1: -671 $\pm$ 690	total: -620 $\pm$ 684 p1: -2057 $\pm$ 2180
4s interval	p2: +68 $\pm$ 81 p3: +38 $\pm$ 43 p4: +45 $\pm$ 51	p2: +55 $\pm$ 52 p3: +73 $\pm$ 14 p4: +61 $\pm$ 42	p2: +61 $\pm$ 45 p3: +39 $\pm$ 35 p4: +60 $\pm$ 71	p2: +34 $\pm$ 36 p3: +32 $\pm$ 31 p4: +39 $\pm$ 19	p2: +33 $\pm$ 31 p3: +43 $\pm$ 32 p4: +53 $\pm$ 33	p2: +57 $\pm$ 96 p3: +12 $\pm$ 47 p4: +29 $\pm$ 57
Periodic (blocking)	total: +35 $\pm$ 97 p1: +55 $\pm$ 304	total: +104 $\pm$ 194 p1: +249 $\pm$ 621	total: -275 $\pm$ 453 p1: -975 $\pm$ 1645	total: -75 $\pm$ 92 p1: -293 $\pm$ 286	total: -132 $\pm$ 213 p1: -485 $\pm$ 671	total: -434 $\pm$ 806 p1: -1515 $\pm$ 2462
8s interval	p2: +29 $\pm$ 17 p3: +26 $\pm$ 12 p4: +12 $\pm$ 19	p2: +68 $\pm$ 110 p3: +25 $\pm$ 39 p4: +22 $\pm$ 41	p2: +20 $\pm$ 46 p3: +7 $\pm$ 15 p4: +40 $\pm$ 48	p2: +24 $\pm$ 13 p3: +37 $\pm$ 10 p4: +19 $\pm$ 7	p2: +47 $\pm$ 48 p3: +22 $\pm$ 18 p4: +20 $\pm$ 29	p2: +25 $\pm$ 29 p3: +37 $\pm$ 55 p4: +49 $\pm$ 52
Periodic (non-blocking)	total: +65 $\pm$ 141 p1: +119 $\pm$ 433	total: +7 $\pm$ 204 p1: -89 $\pm$ 704	total: -145 $\pm$ 270 p1: -657 $\pm$ 846	total: -103 $\pm$ 92 p1: -361 $\pm$ 289	total: -111 $\pm$ 342 p1: -409 $\pm$ 1109	total: -439 $\pm$ 856 p1: -1467 $\pm$ 2809
8s interval	p2: +50 $\pm$ 32 p3: +42 $\pm$ 30 p4: +35 $\pm$ 21	p2: +70 $\pm$ 81 p3: +35 $\pm$ 33 p4: +8 $\pm$ 55	p2: +89 $\pm$ 85 p3: +34 $\pm$ 29 p4: +30 $\pm$ 47	p2: +13 $\pm$ 34 p3: +27 $\pm$ 33 p4: +14 $\pm$ 22	p2: +27 $\pm$ 36 p3: +26 $\pm$ 43 p4: +41 $\pm$ 31	p2: +44 $\pm$ 77 p3: +25 $\pm$ 30 p4: +26 $\pm$ 53
Periodic (blocking)	total: +34 $\pm$ 65 p1: +106 $\pm$ 213	total: +10 $\pm$ 187 p1: -17 $\pm$ 688	total: -144 $\pm$ 245 p1: -526 $\pm$ 833	total: -51 $\pm$ 73 p1: -181 $\pm$ 237	total: -108 $\pm$ 262 p1: -332 $\pm$ 845	total: -349 $\pm$ 691 p1: -1126 $\pm$ 2197
16s interval	p2: +5 $\pm$ 29 p3: -8 $\pm$ 18 p4: +3 $\pm$ 25	p2: +37 $\pm$ 80 p3: +2 $\pm$ 31 p4: +3 $\pm$ 37	p2: +12 $\pm$ 48 p3: +8 $\pm$ 33 p4: +7 $\pm$ 44	p2: +18 $\pm$ 14 p3: +21 $\pm$ 17 p4: -10 $\pm$ 25	p2: +8 $\pm$ 25 p3: -9 $\pm$ 28 p4: +8 $\pm$ 15	p2: -12 $\pm$ 23 p3: +5 $\pm$ 36 p4: +1 $\pm$ 68
Periodic (non-blocking)	total: +33 $\pm$ 98 p1: +60 $\pm$ 329	total: +20 $\pm$ 68 p1: +50 $\pm$ 270	total: -155 $\pm$ 403 p1: -591 $\pm$ 1340	total: -18 $\pm$ 86 p1: -74 $\pm$ 241	total: -58 $\pm$ 354 p1: -187 $\pm$ 1088	total: -448 $\pm$ 743 p1: -1454 $\pm$ 2397
16s interval	p2: +22 $\pm$ 19 p3: +1 $\pm$ 19 p4: +16 $\pm$ 31	p2: +10 $\pm$ 31 p3: +21 $\pm$ 25 p4: -1 $\pm$ 16	p2: +8 $\pm$ 83 p3: +15 $\pm$ 25 p4: -29 $\pm$ 52	p2: +10 $\pm$ 33 p3: +9 $\pm$ 29 p4: +7 $\pm$ 36	p2: +12 $\pm$ 26 p3: +5 $\pm$ 19 p4: +12 $\pm$ 34	p2: +24 $\pm$ 53 p3: +11 $\pm$ 33 p4: +10 $\pm$ 42
Stable-queue (blocking)	total: -25 $\pm$ 83 p1: -110 $\pm$ 268	total: -21 $\pm$ 135 p1: -109 $\pm$ 472	total: -336 $\pm$ 371 p1: -1212 $\pm$ 1232	total: -81 $\pm$ 156 p1: -272 $\pm$ 482	total: -240 $\pm$ 170 p1: -762 $\pm$ 547	total: -523 $\pm$ 770 p1: -1730 $\pm$ 2431
4s delay	p2: +13 $\pm$ 13 p3: -6 $\pm$ 13 p4: +3 $\pm$ 22	p2: +3 $\pm$ 38 p3: +22 $\pm$ 23 p4: +5 $\pm$ 32	p2: +49 $\pm$ 66 p3: +4 $\pm$ 17 p4: +7 $\pm$ 20	p2: +8 $\pm$ 22 p3: +8 $\pm$ 25 p4: +9 $\pm$ 28	p2: +11 $\pm$ 33 p3: -11 $\pm$ 37 p4: +1 $\pm$ 36	p2: +7 $\pm$ 36 p3: +5 $\pm$ 39 p4: -6 $\pm$ 53
Stable-queue (non-blocking)	total: -35 $\pm$ 76 p1: -133 $\pm$ 232	total: -73 $\pm$ 165 p1: -247 $\pm$ 555	total: -225 $\pm$ 323 p1: -1071 $\pm$ 1158	total: -122 $\pm$ 149 p1: -382 $\pm$ 463	total: -170 $\pm$ 230 p1: -553 $\pm$ 718	total: -599 $\pm$ 730 p1: -2004 $\pm$ 2220
4s delay	p2: +0 $\pm$ 17 p3: +4 $\pm$ 19 p4: +6 $\pm$ 9	p2: -4 $\pm$ 23 p3: -7 $\pm$ 10 p4: -10 $\pm$ 29	p2: +252 $\pm$ 519 p3: +35 $\pm$ 24 p4: -11 $\pm$ 22	p2: -2 $\pm$ 23 p3: +2 $\pm$ 25 p4: -3 $\pm$ 31	p2: +13 $\pm$ 18 p3: +6 $\pm$ 20 p4: +7 $\pm$ 35	p2: +39 $\pm$ 50 p3: +16 $\pm$ 55 p4: +12 $\pm$ 42
Stable-queue (blocking)	total: -2 $\pm$ 43 p1: +2 $\pm$ 131	total: -75 $\pm$ 255 p1: -294 $\pm$ 841	total: -305 $\pm$ 239 p1: -1063 $\pm$ 759	total: -53 $\pm$ 124 p1: -174 $\pm$ 373	total: -191 $\pm$ 271 p1: -595 $\pm$ 859	total: -501 $\pm$ 618 p1: -1597 $\pm$ 1972
8s delay	p2: +1 $\pm$ 6 p3: -13 $\pm$ 16 p4: -6 $\pm$ 16	p2: +2 $\pm$ 29 p3: +6 $\pm$ 13 p4: -3 $\pm$ 23	p2: +3 $\pm$ 44 p3: +3 $\pm$ 35 p4: +6 $\pm$ 55	p2: -1 $\pm$ 3 p3: +19 $\pm$ 23 p4: -13 $\pm$ 10	p2: -11 $\pm$ 30 p3: -4 $\pm$ 12 p4: +15 $\pm$ 22	p2: -4 $\pm$ 11 p3: -8 $\pm$ 47 p4: -3 $\pm$ 27
Stable-queue (non-blocking)	total: -3 $\pm$ 54 p1: -27 $\pm$ 173	total: -30 $\pm$ 187 p1: -89 $\pm$ 648	total: -240 $\pm$ 234 p1: -860 $\pm$ 774	total: -85 $\pm$ 135 p1: -273 $\pm$ 432	total: -140 $\pm$ 261 p1: -449 $\pm$ 815	total: -351 $\pm$ 792 p1: -1216 $\pm$ 2585
8s delay	p2: +5 $\pm$ 20 p3: -2 $\pm$ 18 p4: +2 $\pm$ 20	p2: -13 $\pm$ 14 p3: -12 $\pm$ 24 p4: -10 $\pm$ 40	p2: -10 $\pm$ 26 p3: +2 $\pm$ 23 p4: +0 $\pm$ 24	p2: +0 $\pm$ 8 p3: +21 $\pm$ 19 p4: -8 $\pm$ 14	p2: +8 $\pm$ 25 p3: +8 $\pm$ 18 p4: -5 $\pm$ 28	p2: +38 $\pm$ 52 p3: +13 $\pm$ 41 p4: +5 $\pm$ 56
Stable-queue (blocking)	total: +31 $\pm$ 69 p1: +113 $\pm$ 261	total: -103 $\pm$ 133 p1: -371 $\pm$ 451	total: -131 $\pm$ 381 p1: -374 $\pm$ 1340	total: -47 $\pm$ 81 p1: -144 $\pm$ 237	total: -86 $\pm$ 266 p1: -261 $\pm$ 864	total: -498 $\pm$ 713 p1: -1561 $\pm$ 2317
16s delay	p2: +5 $\pm$ 17 p3: -13 $\pm$ 25 p4: -4 $\pm$ 19	p2: +14 $\pm$ 35 p3: +7 $\pm$ 29 p4: +1 $\pm$ 30	p2: -9 $\pm$ 70 p3: +3 $\pm$ 29 p4: -15 $\pm$ 24	p2: -1 $\pm$ 20 p3: +4 $\pm$ 23 p4: -1 $\pm$ 17	p2: +7 $\pm$ 27 p3: -11 $\pm$ 14 p4: +19 $\pm$ 26	p2: -10 $\pm$ 37 p3: +6 $\pm$ 29 p4: -8 $\pm$ 44
Stable-queue (non-blocking)	total: -11 $\pm$ 60 p1: -71 $\pm$ 168	total: +20 $\pm$ 116 p1: +57 $\pm$ 394	total: -46 $\pm$ 521 p1: -195 $\pm$ 1763	total: -66 $\pm$ 95 p1: -203 $\pm$ 291	total: -79 $\pm$ 263 p1: -289 $\pm$ 843	total: -382 $\pm$ 741 p1: -1209 $\pm$ 2460
16s delay	p2: +14 $\pm$ 21 p3: -2 $\pm$ 29 p4: +12 $\pm$ 25	p2: +23 $\pm$ 43 p3: -0 $\pm$ 43 p4: -18 $\pm$ 30	p2: +5 $\pm$ 38 p3: +3 $\pm$ 30 p4: +11 $\pm$ 48	p2: -1 $\pm$ 9 p3: +5 $\pm$ 16 p4: -12 $\pm$ 17	p2: +23 $\pm$ 25 p3: +4 $\pm$ 25 p4: +37 $\pm$ 47	p2: -16 $\pm$ 41 p3: -12 $\pm$ 40 p4: +8 $\pm$ 41

Table 9. Mean paired differences in number of pod deletions between the plugin with DefaultPreemption enabled and the default scheduler for runs with one priority (mean $\pm$ std).

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	-430.0 $\pm$ 104.5	-87.6 $\pm$ 72.3	-17.6 $\pm$ 23.0	-557.6 $\pm$ 156.2	-187.6 $\pm$ 75.3	-77.4 $\pm$ 52.0
Scheduling-failure (non-blocking)	-480.2 $\pm$ 221.9	-75.8 $\pm$ 57.6	-7.8 $\pm$ 17.0	-528.2 $\pm$ 172.3	-210.6 $\pm$ 84.7	-73.8 $\pm$ 65.0
Periodic (blocking), 4s interval	+58.0 $\pm$ 55.5	+83.8 $\pm$ 30.9	+33.0 $\pm$ 20.5	+84.2 $\pm$ 85.6	+60.6 $\pm$ 34.5	+16.6 $\pm$ 29.0
Periodic (non-blocking), 4s interval	+35.4 $\pm$ 75.7	+72.6 $\pm$ 48.0	+54.4 $\pm$ 33.1	+80.0 $\pm$ 104.7	+55.0 $\pm$ 36.1	+18.4 $\pm$ 40.4
Periodic (blocking), 8s interval	-23.4 $\pm$ 77.7	+31.0 $\pm$ 56.8	+21.8 $\pm$ 25.4	+19.8 $\pm$ 109.6	+37.6 $\pm$ 46.6	+11.2 $\pm$ 25.9
Periodic (non-blocking), 8s interval	+2.2 $\pm$ 81.7	+29.4 $\pm$ 46.8	+22.6 $\pm$ 22.1	+64.4 $\pm$ 60.8	+46.2 $\pm$ 49.3	+21.0 $\pm$ 15.7
Periodic (blocking), 16s interval	-33.4 $\pm$ 59.9	-10.8 $\pm$ 46.1	-2.6 $\pm$ 19.9	+23.8 $\pm$ 39.0	+8.0 $\pm$ 26.1	-2.4 $\pm$ 31.2
Periodic (non-blocking), 16s interval	-35.4 $\pm$ 36.1	+8.4 $\pm$ 33.3	+0.8 $\pm$ 21.8	+7.8 $\pm$ 73.3	+19.2 $\pm$ 44.2	+17.4 $\pm$ 13.1
Stable-queue (blocking), 4s delay	+29.4 $\pm$ 77.1	+50.0 $\pm$ 60.0	+31.6 $\pm$ 22.9	+43.0 $\pm$ 96.1	+33.8 $\pm$ 39.3	+28.8 $\pm$ 18.5
Stable-queue (non-blocking), 4s delay	+31.0 $\pm$ 62.7	+54.2 $\pm$ 53.2	+21.0 $\pm$ 51.2	+59.8 $\pm$ 83.4	+31.4 $\pm$ 41.2	+5.6 $\pm$ 23.6
Stable-queue (blocking), 8s delay	-25.4 $\pm$ 88.6	+26.0 $\pm$ 55.6	+10.8 $\pm$ 23.8	+27.4 $\pm$ 77.1	+37.8 $\pm$ 40.0	+14.0 $\pm$ 15.9
Stable-queue (non-blocking), 8s delay	-1.4 $\pm$ 60.3	+26.0 $\pm$ 50.7	+26.2 $\pm$ 21.2	+32.2 $\pm$ 86.5	+33.2 $\pm$ 46.7	+13.6 $\pm$ 28.7
Stable-queue (blocking), 16s delay	-23.2 $\pm$ 58.7	-5.8 $\pm$ 35.8	-15.4 $\pm$ 21.5	+27.0 $\pm$ 55.8	+25.0 $\pm$ 40.1	+9.4 $\pm$ 18.6
Stable-queue (non-blocking), 16s delay	-53.4 $\pm$ 72.5	+10.6 $\pm$ 39.8	-14.2 $\pm$ 25.8	+16.2 $\pm$ 61.2	+17.8 $\pm$ 32.3	+40.2 $\pm$ 49.2

Table 10. Mean paired differences in number of pod deletions between the plugin with DefaultPreemption enabled and the default scheduler for runs with 4 priorities (mean $\pm$ std). p1 is the lowest priority.

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	total: -831.2 ± 139.0 p1: -155.4 ± 169.0 p2: -304.6 ± 28.5 p3: -203.0 ± 12.2 p4: -168.2 ± 14.8	total: -188.0 ± 70.0 p1: $+80.6 \pm 88.7$ p2: -109.0 ± 14.1 p3: -90.8 ± 11.5 p4: -68.8 ± 12.7	total: $+1.6 \pm 95.7$ p1: $+116.2 \pm 88.7$ p2: -47.6 ± 10.3 p3: -40.6 ± 16.6 p4: -26.4 ± 4.8	total: $+128.6 \pm 153.5$ p1: $+622.4 \pm 154.7$ p2: -217.2 ± 55.5 p3: -151.2 ± 36.0 p4: -125.4 ± 20.5	total: -162.6 ± 86.5 p1: $+46.4 \pm 46.6$ p2: -80.4 ± 19.2 p3: -69.8 ± 14.7 p4: -58.8 ± 16.9	total: -21.8 ± 46.6 p1: $+59.8 \pm 36.9$ p2: -35.8 ± 8.4 p3: -25.4 ± 9.4 p4: -20.4 ± 6.1
Scheduling-failure (non-blocking)	total: -752.6 ± 201.0 p1: -95.0 ± 214.7 p2: -286.0 ± 27.5 p3: -194.4 ± 21.5 p4: -177.2 ± 16.8	total: -189.6 ± 117.3 p1: $+83.8 \pm 111.3$ p2: -109.8 ± 8.6 p3: -92.0 ± 17.0 p4: -71.6 ± 16.8	total: -30.8 ± 76.9 p1: $+95.2 \pm 80.6$ p2: -56.6 ± 6.7 p3: -41.8 ± 17.2 p4: -27.6 ± 6.8	total: -497.2 ± 172.4 p1: $+28.4 \pm 118.5$ p2: -226.0 ± 58.6 p3: -164.6 ± 38.9 p4: -135.0 ± 21.1	total: -190.4 ± 100.8 p1: $+23.6 \pm 58.8$ p2: -82.6 ± 25.7 p3: -72.8 ± 18.4 p4: -58.6 ± 15.4	total: -19.4 ± 52.6 p1: $+65.4 \pm 46.6$ p2: -38.2 ± 12.9 p3: -27.4 ± 7.0 p4: -19.2 ± 3.6
Periodic (blocking) 4s interval	total: $+206.4 \pm 125.8$ p1: $+232.4 \pm 111.9$ p2: -10.0 ± 6.9 p3: -7.6 ± 6.5 p4: -8.4 ± 9.8	total: $+176.8 \pm 60.4$ p1: $+190.2 \pm 62.4$ p2: -3.4 ± 6.7 p3: -6.4 ± 5.0 p4: -3.6 ± 1.5	total: $+143.2 \pm 49.2$ p1: $+148.0 \pm 51.0$ p2: -1.4 ± 5.2 p3: -2.4 ± 2.6 p4: -1.0 ± 1.0	total: $+188.0 \pm 72.5$ p1: $+208.6 \pm 79.0$ p2: -8.0 ± 4.1 p3: -8.6 ± 6.0 p4: -4.0 ± 5.4	total: $+131.2 \pm 16.3$ p1: $+139.8 \pm 20.2$ p2: -3.0 ± 4.2 p3: -4.0 ± 4.0 p4: -1.6 ± 1.5	total: $+95.0 \pm 25.4$ p1: $+94.8 \pm 25.1$ p2: $+0.2 \pm 1.1$ p3: $+0.4 \pm 1.1$ p4: -0.4 ± 0.9
Periodic (non-blocking) 4s interval	total: $+253.6 \pm 71.1$ p1: $+276.2 \pm 69.4$ p2: -10.2 ± 5.6 p3: -7.2 ± 9.3 p4: -5.2 ± 6.1	total: $+196.0 \pm 69.9$ p1: $+203.2 \pm 69.5$ p2: -0.6 ± 2.5 p3: -3.6 ± 4.7 p4: -3.0 ± 2.0	total: $+138.4 \pm 58.8$ p1: $+141.8 \pm 61.3$ p2: -1.0 ± 1.7 p3: -2.0 ± 2.1 p4: -0.4 ± 0.9	total: $+154.0 \pm 52.2$ p1: $+173.0 \pm 60.6$ p2: -10.0 ± 6.6 p3: -6.2 ± 6.1 p4: -2.8 ± 1.3	total: $+101.0 \pm 20.1$ p1: $+109.6 \pm 20.1$ p2: -1.4 ± 1.5 p3: -3.2 ± 1.1 p4: -4.0 ± 2.9	total: $+82.0 \pm 37.4$ p1: $+83.8 \pm 38.8$ p2: -0.6 ± 2.2 p3: -0.8 ± 0.8 p4: -0.4 ± 0.9
Periodic (blocking) 8s interval	total: $+132.0 \pm 70.3$ p1: $+143.6 \pm 70.3$ p2: -3.8 ± 5.5 p3: -3.2 ± 3.8 p4: -4.6 ± 3.5	total: $+135.0 \pm 79.3$ p1: $+140.2 \pm 79.8$ p2: $+0.2 \pm 1.9$ p3: -3.0 ± 3.7 p4: -2.4 ± 1.5	total: $+108.0 \pm 31.9$ p1: $+110.2 \pm 32.8$ p2: -1.4 ± 2.5 p3: -0.6 ± 0.9 p4: -0.2 ± 0.4	total: $+139.0 \pm 41.9$ p1: $+150.4 \pm 39.0$ p2: -4.0 ± 3.7 p3: -3.6 ± 4.7 p4: -3.8 ± 3.5	total: $+84.6 \pm 21.3$ p1: $+88.4 \pm 22.5$ p2: -0.6 ± 2.5 p3: -1.8 ± 1.5 p4: -1.4 ± 2.7	total: $+72.4 \pm 28.0$ p1: $+76.2 \pm 29.1$ p2: -1.8 ± 2.2 p3: -1.0 ± 1.9 p4: -1.0 ± 1.7
Periodic (non-blocking) 8s interval	total: $+144.2 \pm 83.9$ p1: $+160.4 \pm 80.3$ p2: -6.4 ± 3.4 p3: -4.6 ± 5.4 p4: -5.2 ± 2.9	total: $+159.0 \pm 67.8$ p1: $+162.6 \pm 67.2$ p2: -0.2 ± 3.9 p3: -1.4 ± 2.1 p4: -2.0 ± 1.6	total: $+109.4 \pm 38.5$ p1: $+112.0 \pm 40.4$ p2: -1.4 ± 5.0 p3: -0.6 ± 0.9 p4: -0.6 ± 0.9	total: $+138.8 \pm 68.9$ p1: $+148.2 \pm 74.6$ p2: -4.8 ± 6.9 p3: -1.2 ± 2.7 p4: -3.4 ± 3.1	total: $+90.4 \pm 13.7$ p1: $+95.0 \pm 17.8$ p2: $+0.2 \pm 2.2$ p3: -2.2 ± 1.9 p4: -2.6 ± 2.9	total: $+70.8 \pm 39.3$ p1: $+73.4 \pm 40.3$ p2: -1.0 ± 1.4 p3: -1.6 ± 2.3 p4: $+0.0 \pm 0.0$
Periodic (blocking) 16s interval	total: $+79.6 \pm 31.9$ p1: $+81.6 \pm 32.8$ p2: $+0.6 \pm 3.7$ p3: -0.2 ± 2.4 p4: -2.4 ± 2.2	total: $+85.4 \pm 56.6$ p1: $+86.2 \pm 57.0$ p2: $+2.6 \pm 4.7$ p3: -1.2 ± 3.7 p4: -2.2 ± 2.5	total: $+85.4 \pm 38.7$ p1: $+86.4 \pm 41.3$ p2: -0.2 ± 3.9 p3: -0.6 ± 1.3 p4: -0.2 ± 0.4	total: $+81.6 \pm 23.8$ p1: $+86.0 \pm 26.4$ p2: -1.6 ± 4.3 p3: -2.0 ± 3.9 p4: -0.8 ± 1.6	total: $+65.0 \pm 27.0$ p1: $+67.2 \pm 27.9$ p2: $+0.4 \pm 1.9$ p3: -1.8 ± 1.8 p4: -0.8 ± 1.8	total: $+57.6 \pm 17.9$ p1: $+58.2 \pm 18.3$ p2: $+0.0 \pm 0.0$ p3: -0.4 ± 0.5 p4: -0.2 ± 0.4
Periodic (non-blocking) 16s interval	total: $+89.8 \pm 50.4$ p1: $+93.6 \pm 51.7$ p2: -1.6 ± 5.9 p3: -1.6 ± 0.5 p4: -0.6 ± 1.7	total: $+95.2 \pm 68.0$ p1: $+94.6 \pm 64.0$ p2: $+2.8 \pm 3.3$ p3: -0.8 ± 2.6 p4: -1.4 ± 1.3	total: $+78.4 \pm 40.8$ p1: $+79.6 \pm 39.3$ p2: -0.4 ± 4.6 p3: -0.4 ± 0.9 p4: -0.4 ± 0.5	total: $+104.2 \pm 72.5$ p1: $+110.8 \pm 73.2$ p2: -2.0 ± 2.9 p3: -3.2 ± 2.2 p4: -1.4 ± 2.3	total: $+65.8 \pm 16.8$ p1: $+67.8 \pm 17.7$ p2: -0.4 ± 1.7 p3: -0.2 ± 0.4 p4: -1.4 ± 1.3	total: $+60.4 \pm 29.1$ p1: $+60.2 \pm 28.8$ p2: $+0.0 \pm 1.4$ p3: $+0.2 \pm 0.4$ p4: $+0.0 \pm 0.0$
Stable-queue (blocking) 4s delay	total: $+258.4 \pm 107.5$ p1: $+263.0 \pm 105.4$ p2: -2.4 ± 2.8 p3: -0.4 ± 4.7 p4: -1.8 ± 1.3	total: $+180.6 \pm 71.9$ p1: $+180.4 \pm 74.6$ p2: $+1.2 \pm 1.6$ p3: $+0.6 \pm 2.7$ p4: -1.6 ± 1.1	total: $+139.8 \pm 35.3$ p1: $+141.8 \pm 36.1$ p2: -1.2 ± 2.8 p3: -0.4 ± 0.9 p4: -0.4 ± 0.9	total: $+184.8 \pm 65.8$ p1: $+187.2 \pm 65.2$ p2: -1.6 ± 2.5 p3: -0.0 ± 2.2 p4: -0.8 ± 1.6	total: $+94.2 \pm 24.3$ p1: $+96.2 \pm 26.9$ p2: -1.2 ± 1.9 p3: -0.6 ± 0.9 p4: -0.2 ± 1.1	total: $+84.4 \pm 32.2$ p1: $+85.0 \pm 32.6$ p2: -0.4 ± 0.9 p3: -0.2 ± 0.4 p4: $+0.0 \pm 0.0$
Stable-queue (non-blocking) 4s delay	total: $+206.4 \pm 106.5$ p1: $+211.6 \pm 106.1$ p2: -2.6 ± 3.0 p3: -0.8 ± 3.5 p4: -1.8 ± 1.8	total: $+181.4 \pm 71.4$ p1: $+182.0 \pm 69.7$ p2: $+1.2 \pm 2.5$ p3: -0.2 ± 1.6 p4: -1.6 ± 1.5	total: $+130.4 \pm 47.0$ p1: $+132.0 \pm 47.8$ p2: -0.8 ± 3.1 p3: -0.8 ± 1.1 p4: $+0.0 \pm 0.0$	total: $+164.0 \pm 60.8$ p1: $+166.6 \pm 62.8$ p2: -1.4 ± 2.1 p3: -0.8 ± 4.0 p4: -0.4 ± 1.5	total: $+92.2 \pm 20.1$ p1: $+93.2 \pm 22.7$ p2: -0.2 ± 1.5 p3: -0.4 ± 0.9 p4: -0.4 ± 1.1	total: $+85.0 \pm 32.8$ p1: $+86.0 \pm 33.1$ p2: -0.2 ± 0.4 p3: -0.6 ± 1.3 p4: -0.6 ± 1.3
Stable-queue (blocking) 8s delay	total: $+130.0 \pm 86.9$ p1: $+129.2 \pm 87.4$ p2: $+0.8 \pm 2.8$ p3: $+0.8 \pm 1.6$ p4: -0.8 ± 1.9	total: $+126.4 \pm 68.7$ p1: $+127.2 \pm 70.1$ p2: -0.4 ± 2.6 p3: $+0.2 \pm 1.1$ p4: -0.6 ± 1.3	total: $+98.2 \pm 41.3$ p1: $+99.0 \pm 41.9$ p2: -0.4 ± 2.7 p3: -0.4 ± 0.9 p4: $+0.0 \pm 0.0$	total: $+134.8 \pm 53.3$ p1: $+135.2 \pm 52.4$ p2: -0.6 ± 1.3 p3: $+0.2 \pm 3.1$ p4: $+0.0 \pm 1.2$	total: $+82.0 \pm 23.2$ p1: $+82.8 \pm 23.9$ p2: $+0.0 \pm 1.4$ p3: -0.8 ± 1.8 p4: $+0.0 \pm 0.7$	total: $+65.8 \pm 15.8$ p1: $+66.4 \pm 17.0$ p2: -0.4 ± 1.7 p3: $+0.2 \pm 0.4$ p4: $+0.4 \pm 0.9$
Stable-queue (non-blocking) 8s delay	total: $+145.6 \pm 72.3$ p1: $+147.0 \pm 70.8$ p2: -1.0 ± 1.4 p3: -1.2 ± 3.3 p4: -1.2 ± 1.8	total: $+127.6 \pm 59.0$ p1: $+125.8 \pm 61.4$ p2: $+3.2 \pm 5.0$ p3: -0.4 ± 2.3 p4: -1.0 ± 1.0	total: $+121.8 \pm 53.0$ p1: $+122.0 \pm 52.5$ p2: -0.2 ± 2.4 p3: -0.4 ± 0.9 p4: $+0.0 \pm 0.0$	total: $+136.0 \pm 56.2$ p1: $+137.0 \pm 55.8$ p2: -0.8 ± 1.6 p3: -0.6 ± 2.9 p4: $+0.4 \pm 2.2$	total: $+77.4 \pm 17.1$ p1: $+79.6 \pm 18.9$ p2: -0.2 ± 0.8 p3: -1.2 ± 1.6 p4: -0.8 ± 1.1	total: $+73.0 \pm 31.1$ p1: $+73.0 \pm 31.1$ p2: $+0.0 \pm 0.0$ p3: $+0.0 \pm 0.0$ p4: $+0.0 \pm 0.0$
Stable-queue (blocking) 16s delay	total: $+83.4 \pm 49.3$ p1: $+81.8 \pm 47.9$ p2: -0.6 ± 2.6 p3: $+1.6 \pm 1.7$ p4: $+0.6 \pm 0.9$	total: $+87.6 \pm 55.5$ p1: $+86.8 \pm 54.3$ p2: $+1.8 \pm 1.5$ p3: -0.2 ± 2.4 p4: -0.8 ± 0.8	total: $+76.2 \pm 36.4$ p1: $+77.8 \pm 36.7$ p2: -1.6 ± 3.4 p3: $+0.0 \pm 0.0$ p4: $+0.0 \pm 0.0$	total: $+73.6 \pm 47.2$ p1: $+73.8 \pm 46.5$ p2: -1.0 ± 1.9 p3: $+0.4 \pm 2.3$ p4: $+0.4 \pm 1.1$	total: $+59.4 \pm 25.4$ p1: $+58.4 \pm 24.5$ p2: $+1.0 \pm 1.4$ p3: -0.4 ± 0.5 p4: $+0.4 \pm 0.5$	total: $+59.2 \pm 28.9$ p1: $+58.6 \pm 28.3$ p2: $+0.4 \pm 0.9$ p3: -0.2 ± 0.4 p4: $+0.0 \pm 0.0$
Stable-queue (non-blocking) 16s delay	total: $+91.0 \pm 50.0$ p1: $+90.0 \pm 51.8$ p2: $+1.4 \pm 3.9$ p3: $+0.2 \pm 1.1$ p4: -0.6 ± 1.1	total: $+86.2 \pm 52.1$ p1: $+85.2 \pm 55.0$ p2: $+1.8 \pm 6.4$ p3: $+0.4 \pm 1.1$ p4: -1.2 ± 1.6	total: $+80.4 \pm 37.0$ p1: $+81.8 \pm 38.9$ p2: -1.2 ± 2.8 p3: -0.2 ± 0.4 p4: $+0.0 \pm 0.0$	total: $+78.8 \pm 40.1$ p1: $+77.6 \pm 38.4$ p2: -0.2 ± 0.8 p3: $+0.8 \pm 2.0$ p4: $+0.6 \pm 0.5$	total: -323.4 ± 846.2 p1: -100.8 ± 353.1 p2: -85.0 ± 188.4 p3: -81.2 ± 179.9 p4: -56.4 ± 125.0	total: $+56.8 \pm 27.2$ p1: $+56.8 \pm 27.3$ p2: $+0.6 \pm 0.9$ p3: -0.6 ± 0.9 p4: $+0.0 \pm 0.0$

Table 11. Mean paired differences in number of solver runs between the plugin with DefaultPreemption enabled and the default scheduler for runs with one priority (mean $\pm$ std).

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	218 $\pm$ 10.3	152 $\pm$ 17.5	108 $\pm$ 30.0	191 $\pm$ 22.2	147 $\pm$ 20.9	109 $\pm$ 40.4
Scheduling-failure (non-blocking)	226 $\pm$ 18.5	156 $\pm$ 18.0	114 $\pm$ 30.4	184 $\pm$ 24.1	149 $\pm$ 30.2	110 $\pm$ 35.1
Periodic (blocking), 4s interval	86 $\pm$ 14.4	80 $\pm$ 17.2	75 $\pm$ 28.8	101 $\pm$ 19.5	92 $\pm$ 23.4	96 $\pm$ 42.8
Periodic (non-blocking), 4s interval	87 $\pm$ 15.9	76 $\pm$ 15.6	80 $\pm$ 36.0	96 $\pm$ 15.5	95 $\pm$ 22.9	93 $\pm$ 42.6
Periodic (blocking), 8s interval	47 $\pm$ 4.1	45 $\pm$ 8.8	51 $\pm$ 16.7	65 $\pm$ 11.1	71 $\pm$ 21.6	61 $\pm$ 23.8
Periodic (non-blocking), 8s interval	47 $\pm$ 9.8	46 $\pm$ 9.7	50 $\pm$ 13.6	63 $\pm$ 14.8	76 $\pm$ 26.5	70 $\pm$ 29.5
Periodic (blocking), 16s interval	23 $\pm$ 4.8	26 $\pm$ 4.8	27 $\pm$ 10.4	37 $\pm$ 10.8	44 $\pm$ 11.4	42 $\pm$ 12.8
Periodic (non-blocking), 16s interval	25 $\pm$ 5.7	25 $\pm$ 4.8	30 $\pm$ 7.7	34 $\pm$ 7.3	44 $\pm$ 15.0	44 $\pm$ 16.6
Stable-queue (blocking), 4s delay	129 $\pm$ 25.2	110 $\pm$ 29.6	98 $\pm$ 42.5	162 $\pm$ 39.7	155 $\pm$ 53.4	120 $\pm$ 48.3
Stable-queue (non-blocking), 4s delay	131 $\pm$ 19.4	111 $\pm$ 24.2	132 $\pm$ 104.2	173 $\pm$ 37.2	157 $\pm$ 52.6	123 $\pm$ 46.6
Stable-queue (blocking), 8s delay	62 $\pm$ 2.9	57 $\pm$ 12.1	74 $\pm$ 26.1	92 $\pm$ 15.1	101 $\pm$ 24.0	93 $\pm$ 41.5
Stable-queue (non-blocking), 8s delay	65 $\pm$ 10.6	60 $\pm$ 14.2	69 $\pm$ 24.3	99 $\pm$ 24.3	102 $\pm$ 28.6	83 $\pm$ 34.5
Stable-queue (blocking), 16s delay	31 $\pm$ 4.5	33 $\pm$ 8.0	40 $\pm$ 15.2	49 $\pm$ 4.7	70 $\pm$ 19.5	65 $\pm$ 30.9
Stable-queue (non-blocking), 16s delay	28 $\pm$ 3.4	31 $\pm$ 6.7	45 $\pm$ 18.9	47 $\pm$ 9.9	66 $\pm$ 18.7	39 $\pm$ 11.0

Table 12. Mean paired differences in number of solver runs between the plugin with DefaultPreemption enabled and the default scheduler for runs with 4 priorities (mean $\pm$ std). p1 is the lowest priority.

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	260 $\pm$ 56.7	193 $\pm$ 44.5	123 $\pm$ 35.8	181 $\pm$ 13.5	123 $\pm$ 9.5	80 $\pm$ 15.7
Scheduling-failure (non-blocking)	266 $\pm$ 67.2	190 $\pm$ 30.7	118 $\pm$ 29.3	190 $\pm$ 16.5	127 $\pm$ 10.0	84 $\pm$ 23.5
Periodic (blocking), 4s interval	104 $\pm$ 41.4	95 $\pm$ 23.6	94 $\pm$ 56.1	86 $\pm$ 13.7	71 $\pm$ 8.9	57 $\pm$ 21.5
Periodic (non-blocking), 4s interval	109 $\pm$ 41.9	117 $\pm$ 67.8	78 $\pm$ 35.1	84 $\pm$ 16.1	69 $\pm$ 9.3	54 $\pm$ 23.3
Periodic (blocking), 8s interval	56 $\pm$ 25.8	65 $\pm$ 26.5	52 $\pm$ 18.4	53 $\pm$ 4.8	46 $\pm$ 5.1	46 $\pm$ 19.3
Periodic (non-blocking), 8s interval	59 $\pm$ 26.3	58 $\pm$ 10.6	58 $\pm$ 30.0	55 $\pm$ 9.9	50 $\pm$ 3.3	40 $\pm$ 15.1
Periodic (blocking), 16s interval	29 $\pm$ 13.5	31 $\pm$ 2.8	30 $\pm$ 6.2	35 $\pm$ 10.3	29 $\pm$ 4.9	28 $\pm$ 7.0
Periodic (non-blocking), 16s interval	34 $\pm$ 24.4	32 $\pm$ 7.7	31 $\pm$ 7.2	33 $\pm$ 9.5	32 $\pm$ 5.5	26 $\pm$ 7.0
Stable-queue (blocking), 4s delay	167 $\pm$ 88.5	161 $\pm$ 98.6	135 $\pm$ 86.1	137 $\pm$ 13.0	106 $\pm$ 9.7	76 $\pm$ 26.2
Stable-queue (non-blocking), 4s delay	241 $\pm$ 251.5	142 $\pm$ 46.0	120 $\pm$ 73.4	128 $\pm$ 21.8	105 $\pm$ 11.9	75 $\pm$ 24.0
Stable-queue (blocking), 8s delay	85 $\pm$ 48.1	139 $\pm$ 140.7	76 $\pm$ 44.1	81 $\pm$ 22.5	68 $\pm$ 12.7	50 $\pm$ 14.3
Stable-queue (non-blocking), 8s delay	108 $\pm$ 94.5	154 $\pm$ 185.7	93 $\pm$ 59.6	74 $\pm$ 12.6	71 $\pm$ 9.4	50 $\pm$ 13.2
Stable-queue (blocking), 16s delay	54 $\pm$ 63.8	43 $\pm$ 18.2	44 $\pm$ 18.6	37 $\pm$ 10.0	40 $\pm$ 8.3	40 $\pm$ 16.7
Stable-queue (non-blocking), 16s delay	44 $\pm$ 37.5	63 $\pm$ 61.2	50 $\pm$ 24.2	38 $\pm$ 8.3	42 $\pm$ 10.3	37 $\pm$ 11.1

Table 13. Mean paired differences in number of plan activations between the plugin with DefaultPreemption enabled and the default scheduler for runs with one priority (mean $\pm$ std).

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	134 $\pm$ 24.4	87 $\pm$ 17.4	56 $\pm$ 13.0	97 $\pm$ 17.1	60 $\pm$ 12.5	36 $\pm$ 4.3
Scheduling-failure (non-blocking)	135 $\pm$ 27.4	94 $\pm$ 15.3	64 $\pm$ 16.1	96 $\pm$ 18.5	58 $\pm$ 9.9	37 $\pm$ 9.5
Periodic (blocking), 4s interval	51 $\pm$ 7.4	48 $\pm$ 5.8	34 $\pm$ 9.5	50 $\pm$ 12.6	32 $\pm$ 9.4	21 $\pm$ 6.1
Periodic (non-blocking), 4s interval	50 $\pm$ 5.1	47 $\pm$ 3.7	39 $\pm$ 11.7	49 $\pm$ 10.2	33 $\pm$ 6.7	23 $\pm$ 5.4
Periodic (blocking), 8s interval	32 $\pm$ 4.4	29 $\pm$ 2.2	27 $\pm$ 6.9	34 $\pm$ 8.1	21 $\pm$ 6.6	18 $\pm$ 3.7
Periodic (non-blocking), 8s interval	31 $\pm$ 5.0	30 $\pm$ 2.1	27 $\pm$ 3.6	34 $\pm$ 6.5	23 $\pm$ 6.3	19 $\pm$ 4.2
Periodic (blocking), 16s interval	17 $\pm$ 5.0	19 $\pm$ 3.7	14 $\pm$ 1.8	20 $\pm$ 5.4	17 $\pm$ 4.7	15 $\pm$ 4.0
Periodic (non-blocking), 16s interval	20 $\pm$ 3.0	18 $\pm$ 4.4	19 $\pm$ 2.4	22 $\pm$ 2.8	16 $\pm$ 6.5	15 $\pm$ 3.0
Stable-queue (blocking), 4s delay	46 $\pm$ 6.5	35 $\pm$ 3.6	29 $\pm$ 5.8	32 $\pm$ 4.6	23 $\pm$ 6.1	19 $\pm$ 6.2
Stable-queue (non-blocking), 4s delay	41 $\pm$ 8.1	37 $\pm$ 2.2	33 $\pm$ 11.4	33 $\pm$ 7.6	24 $\pm$ 5.0	17 $\pm$ 5.0
Stable-queue (blocking), 8s delay	27 $\pm$ 5.3	27 $\pm$ 5.3	23 $\pm$ 3.5	24 $\pm$ 7.0	21 $\pm$ 4.7	17 $\pm$ 4.1
Stable-queue (non-blocking), 8s delay	27 $\pm$ 5.9	28 $\pm$ 4.5	25 $\pm$ 4.3	24 $\pm$ 7.0	21 $\pm$ 5.1	17 $\pm$ 6.3
Stable-queue (blocking), 16s delay	16 $\pm$ 3.8	17 $\pm$ 5.0	16 $\pm$ 3.3	16 $\pm$ 7.4	15 $\pm$ 4.0	14 $\pm$ 5.1
Stable-queue (non-blocking), 16s delay	15 $\pm$ 4.2	18 $\pm$ 4.5	16 $\pm$ 3.0	17 $\pm$ 4.5	15 $\pm$ 4.5	20 $\pm$ 4.1

Table 14. Mean paired differences in number of plan activations between the plugin with DefaultPreemption enabled and the default scheduler for runs with 4 priorities (mean $\pm$ std). p1 is the lowest priority.

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	168 $\pm$ 39.4	116 $\pm$ 25.0	72 $\pm$ 16.3	175 $\pm$ 14.1	77 $\pm$ 11.3	45 $\pm$ 5.3
Scheduling-failure (non-blocking)	175 $\pm$ 42.5	120 $\pm$ 23.5	66 $\pm$ 18.6	138 $\pm$ 12.8	77 $\pm$ 15.0	46 $\pm$ 8.2
Periodic (blocking), 4s interval	72 $\pm$ 22.3	61 $\pm$ 6.8	41 $\pm$ 11.2	67 $\pm$ 10.0	44 $\pm$ 7.0	32 $\pm$ 3.8
Periodic (non-blocking), 4s interval	74 $\pm$ 17.0	59 $\pm$ 15.3	40 $\pm$ 12.6	63 $\pm$ 9.3	40 $\pm$ 7.8	31 $\pm$ 8.5
Periodic (blocking), 8s interval	43 $\pm$ 18.4	40 $\pm$ 10.1	32 $\pm$ 7.0	44 $\pm$ 2.9	29 $\pm$ 7.8	24 $\pm$ 8.0
Periodic (non-blocking), 8s interval	42 $\pm$ 16.9	44 $\pm$ 13.0	30 $\pm$ 10.1	46 $\pm$ 7.4	30 $\pm$ 5.5	25 $\pm$ 8.5
Periodic (blocking), 16s interval	24 $\pm$ 11.2	27 $\pm$ 2.9	23 $\pm$ 5.9	29 $\pm$ 6.9	23 $\pm$ 6.9	19 $\pm$ 2.7
Periodic (non-blocking), 16s interval	23 $\pm$ 9.0	26 $\pm$ 11.3	22 $\pm$ 7.6	30 $\pm$ 9.4	24 $\pm$ 7.1	19 $\pm$ 5.0
Stable-queue (blocking), 4s delay	62 $\pm$ 20.7	51 $\pm$ 15.2	40 $\pm$ 5.4	50 $\pm$ 8.2	33 $\pm$ 7.4	24 $\pm$ 5.9
Stable-queue (non-blocking), 4s delay	64 $\pm$ 23.6	51 $\pm$ 10.0	36 $\pm$ 7.3	50 $\pm$ 10.0	31 $\pm$ 4.5	26 $\pm$ 4.8
Stable-queue (blocking), 8s delay	40 $\pm$ 18.4	37 $\pm$ 9.8	28 $\pm$ 9.2	38 $\pm$ 9.9	27 $\pm$ 5.8	21 $\pm$ 4.6
Stable-queue (non-blocking), 8s delay	38 $\pm$ 18.1	36 $\pm$ 10.2	29 $\pm$ 6.7	36 $\pm$ 5.8	27 $\pm$ 5.3	22 $\pm$ 1.1
Stable-queue (blocking), 16s delay	21 $\pm$ 13.6	23 $\pm$ 8.4	22 $\pm$ 4.4	20 $\pm$ 8.0	22 $\pm$ 5.8	18 $\pm$ 4.1
Stable-queue (non-blocking), 16s delay	22 $\pm$ 10.3	24 $\pm$ 8.4	22 $\pm$ 5.8	23 $\pm$ 7.4	21 $\pm$ 3.9	18 $\pm$ 4.5

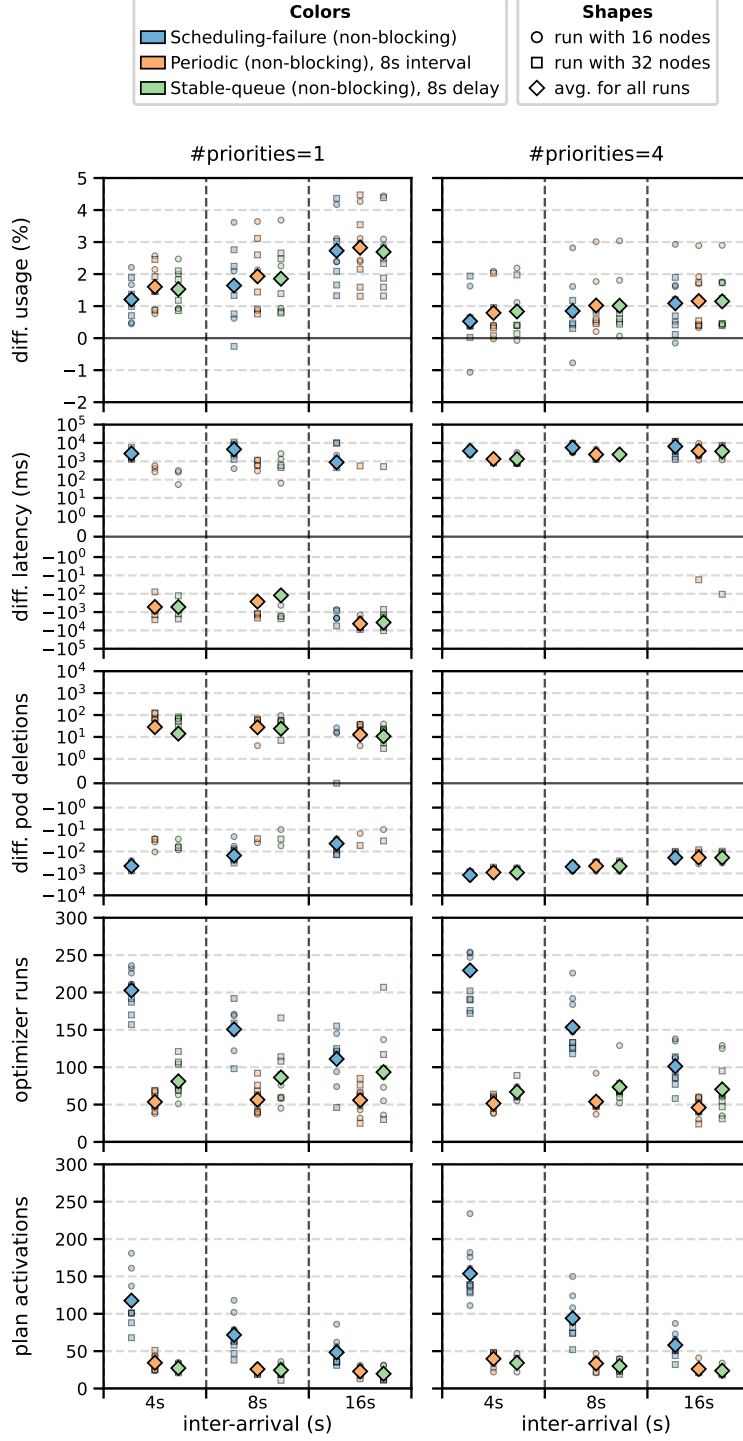


Figure 9. Comparison of OPSche modalities with *non-blocking* variants and *DefaultPreemption* disabled. Each point shows the difference (OPSche’s mode minus default) for a single run (circles: 16 nodes; squares: 32 nodes), and diamonds denote the mean difference per trace setting. Left column: one priority level; right column: four priority levels. Rows report effective usage (%), scheduling latency (ms), pod deletions, solver runs, and plan activations.

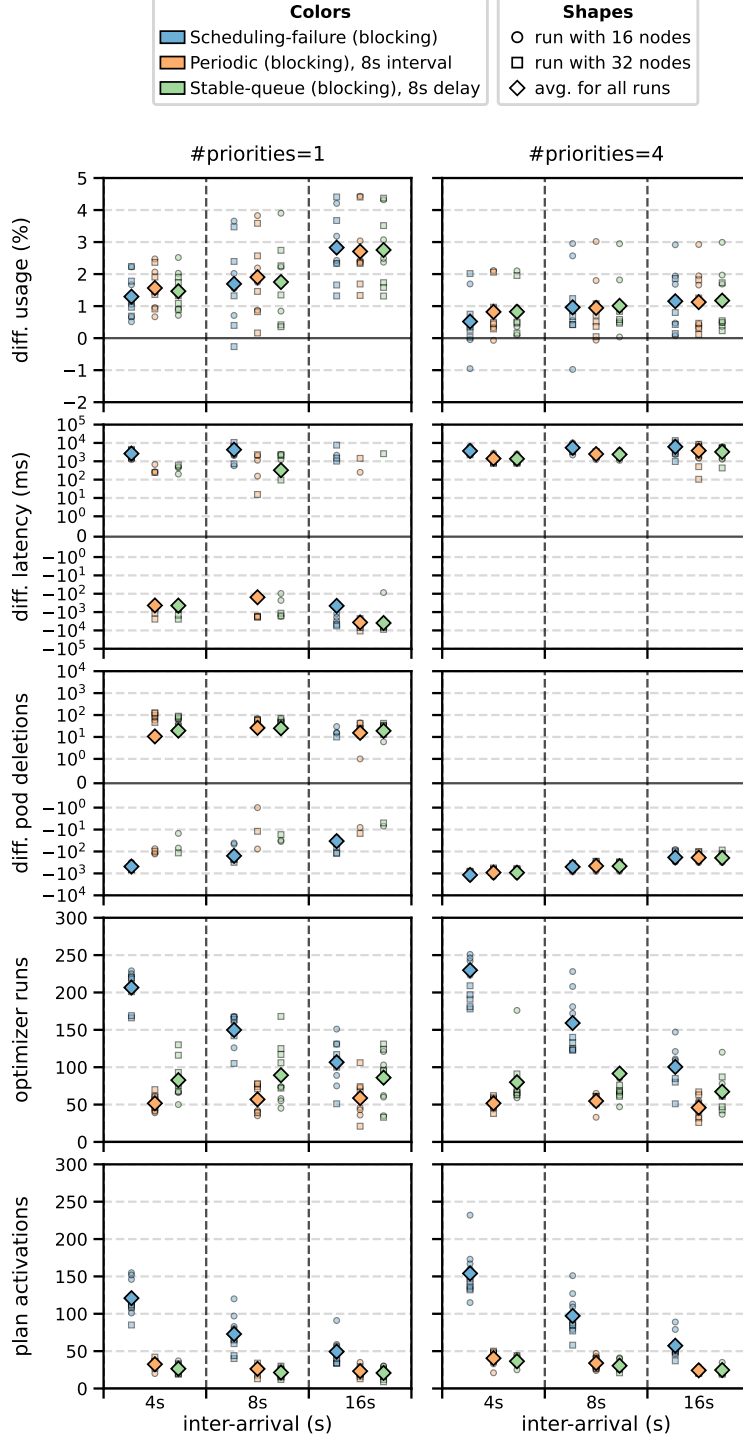


Figure 10. Comparison of OPSche modalities with *blocking* variants and *DefaultPreemption* disabled. Each point shows the difference (OPSche’s mode minus default) for a single run (circles: 16 nodes; squares: 32 nodes), and diamonds denote the mean difference per trace setting. Left column: one priority level; right column: four priority levels. Rows report effective usage (%), scheduling latency (ms), pod deletions, solver runs, and plan activations.

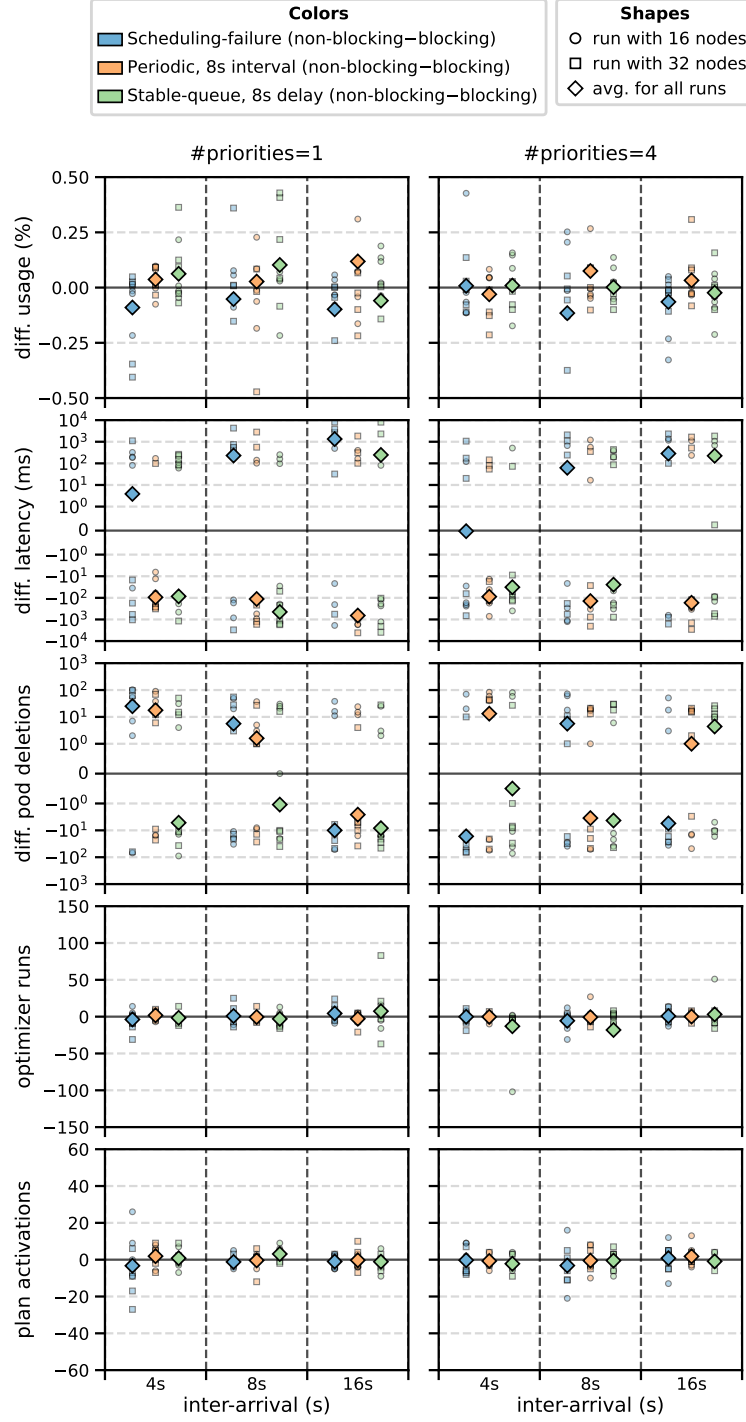


Figure 11. Comparison of OPSche modalities (*non-blocking* vs. *blocking*; *DefaultPreemption* disabled). Each point shows the difference (*non-blocking* minus *blocking*) for a single run (circles: 16 nodes; squares: 32 nodes), and diamonds denote the mean difference per trace setting (mean across five seeds). Left column: one priority level; right column: four priority levels. Rows report effective usage (%), scheduling latency (ms), pod deletions, solver runs, and plan activations.

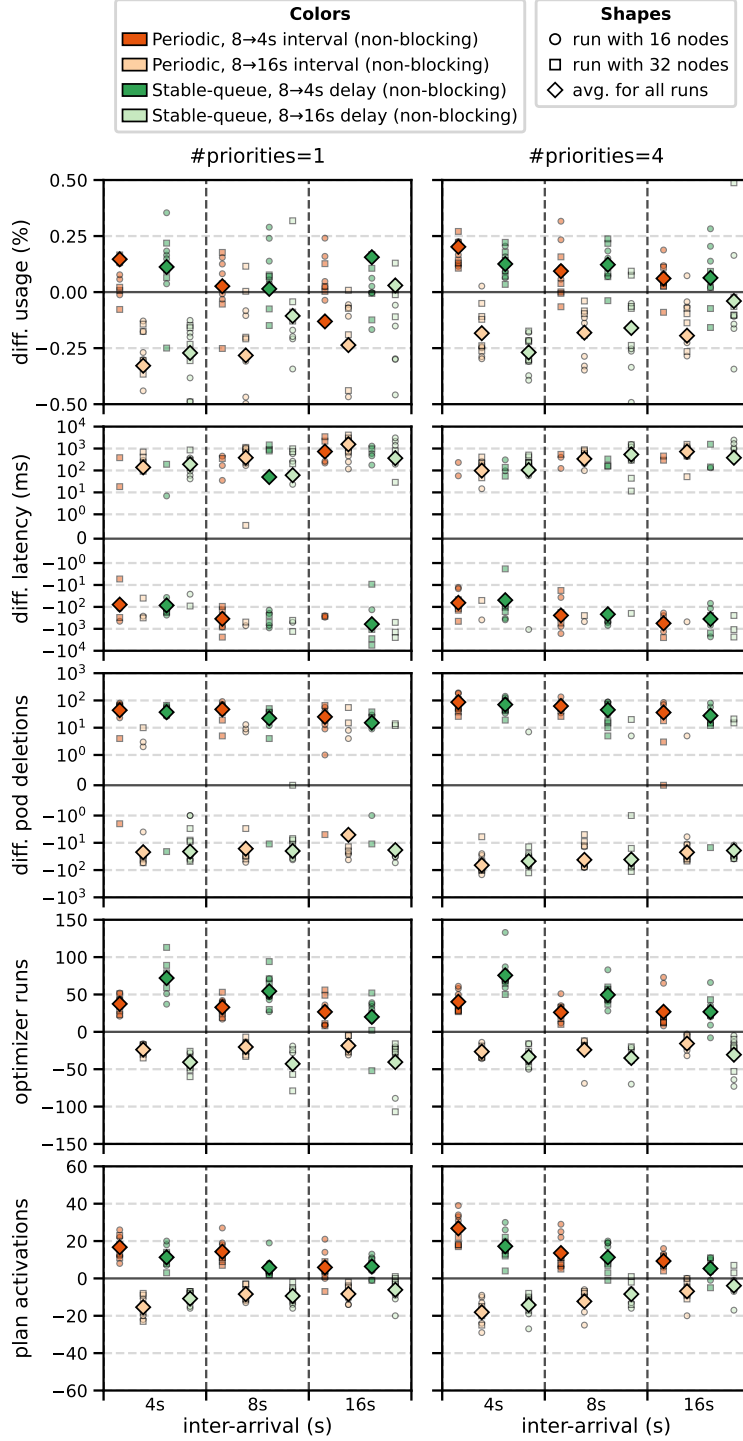


Figure 12. Comparison of OPSche modalities (*non-blocking*; *DefaultPreemption* disabled) with a sensitivity analysis of the *periodic* interval and *stable-queue* idle window. The sensitivity setting compares a shorter (4 s) and longer (16 s) interval/idle window against the default 8 s configuration (8 s→4 s and 8 s→16 s). Each point shows the difference (OPSche minus default) for a single run (circles: 16 nodes; squares: 32 nodes), and diamonds denote the mean difference per trace setting (mean across five seeds). Left column: one priority level; right column: four priority levels. Rows report effective usage (%), scheduling latency (ms), pod deletions, solver runs, and plan activations.

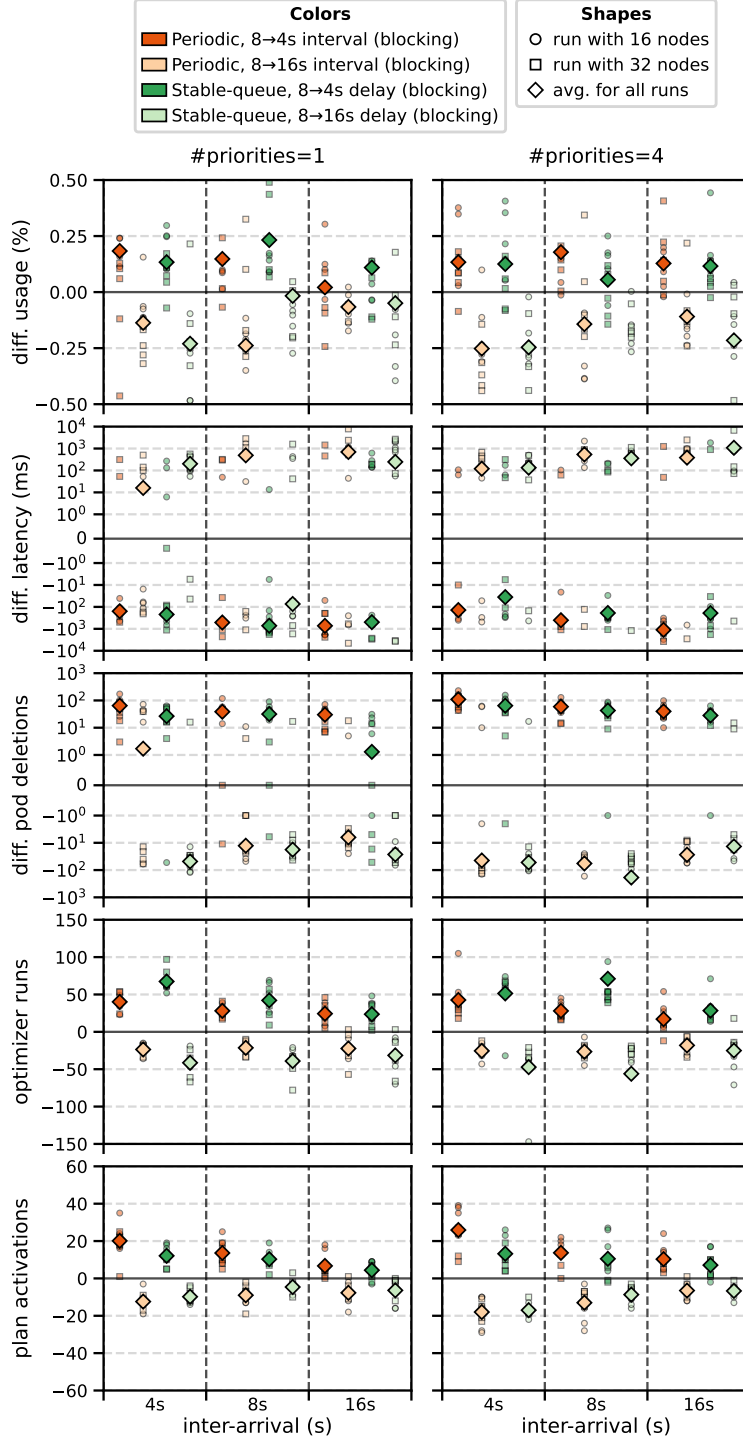


Figure 13. Comparison of OPSche modalities (*blocking*; *DefaultPreemption* disabled) with a sensitivity analysis of the *periodic* interval and *stable-queue* idle window. The sensitivity setting compares a shorter (4 s) and longer (16 s) interval/idle window against the default 8 s configuration (8 s→4 s and 8 s→16 s). Each point shows the difference (OPSche minus default) for a single run (circles: 16 nodes; squares: 32 nodes), and diamonds denote the mean difference per trace setting (mean across five seeds). Left column: one priority level; right column: four priority levels. Rows report effective usage (%), scheduling latency (ms), pod deletions, solver runs, and plan activations.

Table 15. Mean paired differences in effective resource usage (%) between the plugin with DefaultPreemption disabled and the default scheduler for runs with one priority (mean $\pm$ std).

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	+1.2 $\pm$ 0.7	+1.9 $\pm$ 1.1	+3.0 $\pm$ 0.8	+1.4 $\pm$ 0.6	+1.5 $\pm$ 1.5	+2.7 $\pm$ 1.3
Scheduling-failure (non-blocking)	+1.2 $\pm$ 0.8	+1.9 $\pm$ 1.1	+3.0 $\pm$ 0.7	+1.2 $\pm$ 0.4	+1.4 $\pm$ 1.2	+2.5 $\pm$ 1.2
Periodic (blocking), 4s interval	+2.0 $\pm$ 0.6	+2.2 $\pm$ 1.0	+3.1 $\pm$ 0.8	+1.5 $\pm$ 0.4	+1.9 $\pm$ 1.2	+2.4 $\pm$ 1.1
Periodic (non-blocking), 4s interval	+1.8 $\pm$ 0.6	+2.2 $\pm$ 1.0	+3.1 $\pm$ 0.8	+1.7 $\pm$ 0.6	+1.8 $\pm$ 0.9	+2.3 $\pm$ 1.3
Periodic (blocking), 8s interval	+1.5 $\pm$ 0.8	+2.1 $\pm$ 1.1	+3.0 $\pm$ 0.9	+1.6 $\pm$ 0.6	+1.7 $\pm$ 1.4	+2.4 $\pm$ 1.2
Periodic (non-blocking), 8s interval	+1.6 $\pm$ 0.8	+2.1 $\pm$ 1.0	+3.0 $\pm$ 0.8	+1.6 $\pm$ 0.6	+1.8 $\pm$ 1.0	+2.6 $\pm$ 1.3
Periodic (blocking), 16s interval	+1.5 $\pm$ 0.7	+1.9 $\pm$ 1.1	+2.9 $\pm$ 0.8	+1.4 $\pm$ 0.5	+1.5 $\pm$ 1.0	+2.4 $\pm$ 1.5
Periodic (non-blocking), 16s interval	+1.3 $\pm$ 0.7	+1.8 $\pm$ 1.0	+2.9 $\pm$ 0.8	+1.2 $\pm$ 0.4	+1.5 $\pm$ 0.9	+2.3 $\pm$ 1.3
Stable-queue (blocking), 4s delay	+1.7 $\pm$ 0.7	+2.2 $\pm$ 1.1	+3.0 $\pm$ 0.9	+1.5 $\pm$ 0.5	+1.8 $\pm$ 1.0	+2.7 $\pm$ 1.3
Stable-queue (non-blocking), 4s delay	+1.7 $\pm$ 0.7	+2.2 $\pm$ 1.1	+3.1 $\pm$ 0.8	+1.5 $\pm$ 0.4	+1.5 $\pm$ 1.0	+2.6 $\pm$ 1.3
Stable-queue (blocking), 8s delay	+1.5 $\pm$ 0.8	+2.1 $\pm$ 1.1	+3.0 $\pm$ 0.8	+1.4 $\pm$ 0.4	+1.4 $\pm$ 1.1	+2.5 $\pm$ 1.4
Stable-queue (non-blocking), 8s delay	+1.6 $\pm$ 0.7	+2.1 $\pm$ 1.0	+3.1 $\pm$ 0.8	+1.5 $\pm$ 0.5	+1.6 $\pm$ 0.9	+2.3 $\pm$ 1.2
Stable-queue (blocking), 16s delay	+1.2 $\pm$ 0.8	+1.9 $\pm$ 1.1	+2.8 $\pm$ 0.7	+1.2 $\pm$ 0.5	+1.5 $\pm$ 1.3	+2.6 $\pm$ 1.4
Stable-queue (non-blocking), 16s delay	+1.3 $\pm$ 0.7	+1.9 $\pm$ 1.0	+2.8 $\pm$ 0.7	+1.2 $\pm$ 0.4	+1.6 $\pm$ 1.0	+2.6 $\pm$ 1.3

Table 16. Mean paired differences in effective resource usage (%) between the plugin with DefaultPreemption disabled and the default scheduler for runs with 4 priorities (mean $\pm$ std). p1 is the lowest priority.

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	+0.4 $\pm$ 1.0	+1.2 $\pm$ 1.6	+1.4 $\pm$ 1.1	+0.7 $\pm$ 0.8	+0.7 $\pm$ 0.3	+0.9 $\pm$ 0.7
Scheduling-failure (non-blocking)	+0.4 $\pm$ 1.0	+1.1 $\pm$ 1.3	+1.3 $\pm$ 1.2	+0.7 $\pm$ 0.7	+0.6 $\pm$ 0.3	+0.9 $\pm$ 0.7
Periodic (blocking), 4s interval	+1.0 $\pm$ 0.8	+1.4 $\pm$ 1.2	+1.6 $\pm$ 1.1	+0.9 $\pm$ 0.8	+0.8 $\pm$ 0.3	+1.0 $\pm$ 0.6
Periodic (non-blocking), 4s interval	+1.1 $\pm$ 0.7	+1.5 $\pm$ 1.0	+1.5 $\pm$ 1.0	+0.9 $\pm$ 0.7	+0.7 $\pm$ 0.3	+0.9 $\pm$ 0.6
Periodic (blocking), 8s interval	+0.8 $\pm$ 0.8	+1.3 $\pm$ 1.1	+1.5 $\pm$ 1.1	+0.9 $\pm$ 0.7	+0.5 $\pm$ 0.4	+0.8 $\pm$ 0.7
Periodic (non-blocking), 8s interval	+0.8 $\pm$ 0.8	+1.4 $\pm$ 1.1	+1.4 $\pm$ 1.1	+0.8 $\pm$ 0.8	+0.7 $\pm$ 0.2	+0.9 $\pm$ 0.6
Periodic (blocking), 16s interval	+0.6 $\pm$ 0.8	+1.1 $\pm$ 1.1	+1.3 $\pm$ 1.1	+0.5 $\pm$ 0.8	+0.5 $\pm$ 0.3	+0.7 $\pm$ 0.6
Periodic (non-blocking), 16s interval	+0.6 $\pm$ 0.8	+1.1 $\pm$ 1.2	+1.3 $\pm$ 1.2	+0.6 $\pm$ 0.7	+0.5 $\pm$ 0.2	+0.6 $\pm$ 0.8
Stable-queue (blocking), 4s delay	+1.1 $\pm$ 0.9	+1.5 $\pm$ 1.0	+1.7 $\pm$ 1.0	+0.8 $\pm$ 0.7	+0.7 $\pm$ 0.2	+0.9 $\pm$ 0.7
Stable-queue (non-blocking), 4s delay	+1.0 $\pm$ 0.9	+1.5 $\pm$ 1.1	+1.6 $\pm$ 1.1	+0.9 $\pm$ 0.8	+0.8 $\pm$ 0.3	+0.8 $\pm$ 0.7
Stable-queue (blocking), 8s delay	+0.9 $\pm$ 0.8	+1.3 $\pm$ 1.1	+1.5 $\pm$ 1.1	+0.8 $\pm$ 0.7	+0.7 $\pm$ 0.2	+0.8 $\pm$ 0.6
Stable-queue (non-blocking), 8s delay	+0.9 $\pm$ 0.9	+1.4 $\pm$ 1.1	+1.4 $\pm$ 1.0	+0.8 $\pm$ 0.7	+0.6 $\pm$ 0.2	+0.9 $\pm$ 0.6
Stable-queue (blocking), 16s delay	+0.6 $\pm$ 0.8	-16.4 $\pm$ 39.5	+1.4 $\pm$ 1.1	+0.5 $\pm$ 0.7	+0.5 $\pm$ 0.3	+0.6 $\pm$ 0.8
Stable-queue (non-blocking), 16s delay	+0.6 $\pm$ 0.9	+1.1 $\pm$ 1.2	+1.3 $\pm$ 1.1	+0.5 $\pm$ 0.7	+0.6 $\pm$ 0.3	+0.9 $\pm$ 0.5

Table 17. Mean paired differences in scheduling latency (ms) between the plugin with DefaultPreemption disabled and the default scheduler for runs with one priority (mean $\pm$ std).

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	+2349 $\pm$ 992	+2954 $\pm$ 1988	-865 $\pm$ 1938	+2904 $\pm$ 1329	+5572 $\pm$ 3599	-51 $\pm$ 5367
Scheduling-failure (non-blocking)	+2499 $\pm$ 1056	+2832 $\pm$ 1825	-1013 $\pm$ 1854	+2762 $\pm$ 1748	+6153 $\pm$ 4018	+2771 $\pm$ 6931
Periodic (blocking), 4s interval	-196 $\pm$ 471	-159 $\pm$ 1250	-3955 $\pm$ 1838	-988 $\pm$ 1193	-1180 $\pm$ 722	-4839 $\pm$ 4214
Periodic (non-blocking), 4s interval	-140 $\pm$ 553	-51 $\pm$ 1241	-3304 $\pm$ 1710	-1075 $\pm$ 1003	-1185 $\pm$ 1083	-3896 $\pm$ 3365
Periodic (blocking), 8s interval	+21 $\pm$ 551	+302 $\pm$ 1423	-2679 $\pm$ 1742	-889 $\pm$ 989	-613 $\pm$ 1804	-4658 $\pm$ 4628
Periodic (non-blocking), 8s interval	+19 $\pm$ 563	+29 $\pm$ 1053	-3438 $\pm$ 1709	-1076 $\pm$ 987	-567 $\pm$ 1377	-5216 $\pm$ 3605
Periodic (blocking), 16s interval	-20 $\pm$ 590	+629 $\pm$ 1936	-2404 $\pm$ 1617	-817 $\pm$ 992	+40 $\pm$ 1110	-3537 $\pm$ 3170
Periodic (non-blocking), 16s interval	+128 $\pm$ 544	+333 $\pm$ 1422	-2258 $\pm$ 2083	-909 $\pm$ 877	-108 $\pm$ 1236	-3272 $\pm$ 2836
Stable-queue (blocking), 4s delay	-159 $\pm$ 456	+104 $\pm$ 1428	-3005 $\pm$ 2140	-1166 $\pm$ 923	-898 $\pm$ 1436	-5875 $\pm$ 3659
Stable-queue (non-blocking), 4s delay	-193 $\pm$ 420	-13 $\pm$ 1373	-3064 $\pm$ 1577	-1032 $\pm$ 920	-132 $\pm$ 1692	-5557 $\pm$ 3254
Stable-queue (blocking), 8s delay	-14 $\pm$ 574	+309 $\pm$ 1547	-3192 $\pm$ 2121	-870 $\pm$ 1155	+343 $\pm$ 1809	-4695 $\pm$ 4385
Stable-queue (non-blocking), 8s delay	-95 $\pm$ 444	+366 $\pm$ 1666	-3541 $\pm$ 1722	-960 $\pm$ 877	-611 $\pm$ 1302	-3856 $\pm$ 4571
Stable-queue (blocking), 16s delay	+179 $\pm$ 599	+329 $\pm$ 1344	-2266 $\pm$ 2526	-661 $\pm$ 1113	+176 $\pm$ 1806	-5130 $\pm$ 3778
Stable-queue (non-blocking), 16s delay	+13 $\pm$ 476	+454 $\pm$ 1505	-2098 $\pm$ 1882	-685 $\pm$ 1058	-577 $\pm$ 1308	-4573 $\pm$ 3678

Table 18. Mean paired differences in scheduling latency (ms) between the plugin with DefaultPreemption disabled and the default scheduler for runs with 4 priorities (mean $\pm$ std). p1 is the lowest priority.

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	total: +4008 $\pm$ 1537	total: +5497 $\pm$ 3095	total: +5800 $\pm$ 3120	total: +3308 $\pm$ 790	total: +5485 $\pm$ 1550	total: +6531 $\pm$ 5197
	p1: +3569 $\pm$ 1367	p1: +5371 $\pm$ 3139	p1: +5010 $\pm$ 4586	p1: +3030 $\pm$ 1201	p1: +5190 $\pm$ 2668	p1: +6055 $\pm$ 8602
	p2: +4437 $\pm$ 1525	p2: +7288 $\pm$ 5947	p2: +6538 $\pm$ 3154	p2: +4086 $\pm$ 1245	p2: +6344 $\pm$ 1674	p2: +7024 $\pm$ 4928
	p3: +3704 $\pm$ 1504	p3: +4665 $\pm$ 2290	p3: +5792 $\pm$ 3615	p3: +3155 $\pm$ 888	p3: +5467 $\pm$ 1514	p3: +5812 $\pm$ 2624
Scheduling-failure (non-blocking)	p4: +4362 $\pm$ 2200	p4: +3958 $\pm$ 2193	p4: +5455 $\pm$ 2315	p4: +2872 $\pm$ 539	p4: +4531 $\pm$ 978	p4: +7176 $\pm$ 8146
	total: +3903 $\pm$ 1448	total: +5089 $\pm$ 2805	total: +6322 $\pm$ 3624	total: +3412 $\pm$ 788	total: +6018 $\pm$ 2442	total: +6579 $\pm$ 5330
	p1: +3570 $\pm$ 1465	p1: +4872 $\pm$ 2635	p1: +7336 $\pm$ 7818	p1: +2942 $\pm$ 474	p1: +7093 $\pm$ 5547	p1: +4527 $\pm$ 6558
	p2: +4286 $\pm$ 1301	p2: +6749 $\pm$ 6009	p2: +7018 $\pm$ 2464	p2: +4283 $\pm$ 1349	p2: +6667 $\pm$ 2535	p2: +8490 $\pm$ 6506
Periodic (blocking)	p3: +3515 $\pm$ 1246	p3: +4184 $\pm$ 2078	p3: +4225 $\pm$ 2475	p3: +3528 $\pm$ 825	p3: +5022 $\pm$ 1238	p3: +5713 $\pm$ 3208
	p4: +4263 $\pm$ 2134	p4: +3927 $\pm$ 1637	p4: +5492 $\pm$ 2450	p4: +2826 $\pm$ 768	p4: +3994 $\pm$ 716	p4: +7677 $\pm$ 9280
	total: +1605 $\pm$ 773	total: +2244 $\pm$ 895	total: +2879 $\pm$ 1780	total: +943 $\pm$ 326	total: +1910 $\pm$ 473	total: +2529 $\pm$ 2154
	p1: +821 $\pm$ 676	p1: +1045 $\pm$ 1853	p1: +168 $\pm$ 2965	p1: +74 $\pm$ 285	p1: +585 $\pm$ 763	p1: -611 $\pm$ 2896
4s interval	p2: +1669 $\pm$ 465	p2: +2553 $\pm$ 906	p2: +3912 $\pm$ 1889	p2: +1457 $\pm$ 679	p2: +2242 $\pm$ 1393	p2: +3439 $\pm$ 4148
	p3: +1831 $\pm$ 665	p3: +3126 $\pm$ 1859	p3: +3366 $\pm$ 2073	p3: +1384 $\pm$ 496	p3: +2923 $\pm$ 907	p3: +4190 $\pm$ 2901
	p4: +2473 $\pm$ 1939	p4: +2552 $\pm$ 1440	p4: +4714 $\pm$ 2820	p4: +1150 $\pm$ 226	p4: +2375 $\pm$ 897	p4: +4167 $\pm$ 5220
	total: +1590 $\pm$ 550	total: +2361 $\pm$ 809	total: +3681 $\pm$ 2779	total: +930 $\pm$ 335	total: +1834 $\pm$ 206	total: +2486 $\pm$ 2958
Periodic (non-blocking)	p1: +737 $\pm$ 458	p1: +1481 $\pm$ 1360	p1: +1474 $\pm$ 5720	p1: +128 $\pm$ 463	p1: +261 $\pm$ 814	p1: -1089 $\pm$ 3306
	p2: +1798 $\pm$ 678	p2: +2810 $\pm$ 1370	p2: +4312 $\pm$ 1942	p2: +1580 $\pm$ 602	p2: +2749 $\pm$ 1640	p2: +4484 $\pm$ 5009
	p3: +1912 $\pm$ 805	p3: +2403 $\pm$ 1188	p3: +4285 $\pm$ 3294	p3: +1242 $\pm$ 395	p3: +2135 $\pm$ 766	p3: +3133 $\pm$ 1337
	p4: +2204 $\pm$ 1115	p4: +2926 $\pm$ 1434	p4: +4714 $\pm$ 2217	p4: +968 $\pm$ 122	p4: +2734 $\pm$ 1084	p4: +4493 $\pm$ 6178
Periodic (blocking)	total: +1753 $\pm$ 729	total: +2499 $\pm$ 870	total: +4155 $\pm$ 2780	total: +1074 $\pm$ 357	total: +2465 $\pm$ 725	total: +3473 $\pm$ 3651
	p1: +979 $\pm$ 552	p1: +1320 $\pm$ 1909	p1: +1762 $\pm$ 4860	p1: +191 $\pm$ 419	p1: +799 $\pm$ 1212	p1: -1063 $\pm$ 2716
	p2: +2007 $\pm$ 602	p2: +2962 $\pm$ 1359	p2: +4760 $\pm$ 3036	p2: +1645 $\pm$ 565	p2: +3284 $\pm$ 1718	p2: +5081 $\pm$ 4737
	p3: +1846 $\pm$ 670	p3: +2725 $\pm$ 1129	p3: +5634 $\pm$ 4069	p3: +1417 $\pm$ 581	p3: +3172 $\pm$ 1775	p3: +3965 $\pm$ 1406
8s interval	p4: +2491 $\pm$ 1911	p4: +3317 $\pm$ 1296	p4: +4661 $\pm$ 2537	p4: +1364 $\pm$ 396	p4: +3217 $\pm$ 1306	p4: +8101 $\pm$ 13278
	total: +1551 $\pm$ 505	total: +2749 $\pm$ 1361	total: +4272 $\pm$ 3209	total: +1099 $\pm$ 268	total: +1933 $\pm$ 423	total: +3017 $\pm$ 2190
	p1: +586 $\pm$ 615	p1: +1356 $\pm$ 2304	p1: +2612 $\pm$ 6163	p1: +140 $\pm$ 380	p1: +514 $\pm$ 172	p1: -389 $\pm$ 2798
	p2: +1801 $\pm$ 446	p2: +3300 $\pm$ 1018	p2: +4088 $\pm$ 1939	p2: +1671 $\pm$ 568	p2: +2689 $\pm$ 1100	p2: +4340 $\pm$ 3382
Periodic (non-blocking)	p3: +1741 $\pm$ 741	p3: +3340 $\pm$ 1570	p3: +5187 $\pm$ 3310	p3: +1603 $\pm$ 712	p3: +2158 $\pm$ 1057	p3: +4656 $\pm$ 2556
	p4: +2498 $\pm$ 1857	p4: +3440 $\pm$ 1982	p4: +5127 $\pm$ 2637	p4: +1275 $\pm$ 285	p4: +2920 $\pm$ 1148	p4: +4583 $\pm$ 5776
	total: +1629 $\pm$ 546	total: +3317 $\pm$ 1448	total: +4516 $\pm$ 2889	total: +1438 $\pm$ 506	total: +2723 $\pm$ 329	total: +3879 $\pm$ 2260
	p1: +656 $\pm$ 346	p1: +2359 $\pm$ 2626	p1: +2544 $\pm$ 5847	p1: +626 $\pm$ 371	p1: +993 $\pm$ 798	p1: +79 $\pm$ 2768
16s interval	p2: +1937 $\pm$ 512	p2: +3572 $\pm$ 742	p2: +5760 $\pm$ 2553	p2: +1863 $\pm$ 969	p2: +3336 $\pm$ 1392	p2: +5840 $\pm$ 4863
	p3: +1963 $\pm$ 799	p3: +3323 $\pm$ 1947	p3: +4484 $\pm$ 2120	p3: +1926 $\pm$ 850	p3: +3939 $\pm$ 1860	p3: +5837 $\pm$ 2539
	p4: +2342 $\pm$ 883	p4: +4380 $\pm$ 2581	p4: +5219 $\pm$ 2178	p4: +1625 $\pm$ 486	p4: +3253 $\pm$ 1130	p4: +4627 $\pm$ 3755
	total: +1602 $\pm$ 552	total: +2932 $\pm$ 999	total: +4946 $\pm$ 3584	total: +1244 $\pm$ 396	total: +2425 $\pm$ 840	total: +3787 $\pm$ 2524
Periodic (non-blocking)	p1: +841 $\pm$ 534	p1: +1799 $\pm$ 1961	p1: +2608 $\pm$ 7538	p1: +630 $\pm$ 399	p1: +598 $\pm$ 970	p1: +55 $\pm$ 2520
	p2: +1871 $\pm$ 623	p2: +3122 $\pm$ 681	p2: +5325 $\pm$ 2820	p2: +1642 $\pm$ 523	p2: +3164 $\pm$ 1624	p2: +5220 $\pm$ 4024
	p3: +2121 $\pm$ 879	p3: +3536 $\pm$ 1458	p3: +5945 $\pm$ 3969	p3: +1570 $\pm$ 693	p3: +3201 $\pm$ 1377	p3: +5330 $\pm$ 2347
	p4: +1662 $\pm$ 575	p4: +3693 $\pm$ 2331	p4: +5982 $\pm$ 2889	p4: +1253 $\pm$ 347	p4: +3480 $\pm$ 1120	p4: +5649 $\pm$ 7035
Stable-queue (blocking)	total: +1603 $\pm$ 706	total: +2528 $\pm$ 1059	total: +3179 $\pm$ 2910	total: +1047 $\pm$ 330	total: +1819 $\pm$ 238	total: +2954 $\pm$ 2724
	p1: +724 $\pm$ 432	p1: +1507 $\pm$ 1646	p1: +1645 $\pm$ 6208	p1: +322 $\pm$ 505	p1: +126 $\pm$ 389	p1: -445 $\pm$ 3451
	p2: +1924 $\pm$ 810	p2: +3171 $\pm$ 1204	p2: +3664 $\pm$ 1787	p2: +1546 $\pm$ 665	p2: +2587 $\pm$ 1445	p2: +5336 $\pm$ 5192
	p3: +1750 $\pm$ 601	p3: +2839 $\pm$ 1583	p3: +2875 $\pm$ 1958	p3: +1294 $\pm$ 361	p3: +2684 $\pm$ 1114	p3: +3244 $\pm$ 1671
4s delay	p4: +2374 $\pm$ 1984	p4: +2707 $\pm$ 1646	p4: +4524 $\pm$ 2841	p4: +1258 $\pm$ 167	p4: +2541 $\pm$ 1085	p4: +4339 $\pm$ 4517
	total: +1619 $\pm$ 724	total: +2269 $\pm$ 795	total: +2918 $\pm$ 2044	total: +940 $\pm$ 226	total: +1975 $\pm$ 185	total: +3335 $\pm$ 2563
	p1: +792 $\pm$ 523	p1: +1048 $\pm$ 1498	p1: +801 $\pm$ 3877	p1: +99 $\pm$ 251	p1: +171 $\pm$ 315	p1: -684 $\pm$ 2813
	p2: +1774 $\pm$ 555	p2: +2934 $\pm$ 1476	p2: +4046 $\pm$ 1833	p2: +1561 $\pm$ 523	p2: +2684 $\pm$ 1152	p2: +4906 $\pm$ 4786
Stable-queue (non-blocking)	p3: +1845 $\pm$ 727	p3: +2716 $\pm$ 905	p3: +2827 $\pm$ 1966	p3: +1225 $\pm$ 402	p3: +3240 $\pm$ 1340	p3: +4562 $\pm$ 2953
	p4: +2442 $\pm$ 1985	p4: +2618 $\pm$ 1584	p4: +4268 $\pm$ 1829	p4: +1115 $\pm$ 264	p4: +2530 $\pm$ 770	p4: +6038 $\pm$ 8596
	total: +1717 $\pm$ 621	total: +2708 $\pm$ 1202	total: +3181 $\pm$ 2019	total: +1005 $\pm$ 264	total: +2018 $\pm$ 375	total: +3337 $\pm$ 2086
	p1: +781 $\pm$ 564	p1: +1350 $\pm$ 1864	p1: +413 $\pm$ 2974	p1: +149 $\pm$ 385	p1: +232 $\pm$ 781	p1: -114 $\pm$ 2575
8s delay	p2: +2005 $\pm$ 723	p2: +3326 $\pm$ 1779	p2: +4457 $\pm$ 2737	p2: +1639 $\pm$ 697	p2: +2836 $\pm$ 967	p2: +5527 $\pm$ 4524
	p3: +1933 $\pm$ 792	p3: +3113 $\pm$ 1607	p3: +3263 $\pm$ 1584	p3: +1347 $\pm$ 270	p3: +2605 $\pm$ 1152	p3: +4440 $\pm$ 2303
	p4: +2529 $\pm$ 1698	p4: +3439 $\pm$ 1995	p4: +5490 $\pm$ 2706	p4: +1124 $\pm$ 167	p4: +3182 $\pm$ 1039	p4: +4069 $\pm$ 4862
	total: +1703 $\pm$ 825	total: +2714 $\pm$ 945	total: +3529 $\pm$ 2297	total: +955 $\pm$ 241	total: +1963 $\pm$ 155	total: +3436 $\pm$ 2745
Stable-queue (non-blocking)	p1: +927 $\pm$ 610	p1: +1197 $\pm$ 1670	p1: +1200 $\pm$ 4651	p1: +185 $\pm$ 389	p1: +247 $\pm$ 544	p1: -286 $\pm$ 2627
	p2: +1941 $\pm$ 930	p2: +2861 $\pm$ 501	p2: +4463 $\pm$ 2672	p2: +1238 $\pm$ 167	p2: +2615 $\pm$ 1153	p2: +5373 $\pm$ 5504
	p3: +1832 $\pm$ 537	p3: +3791 $\pm$ 2667	p3: +4480 $\pm$ 2956	p3: +1475 $\pm$ 487	p3: +2681 $\pm$ 985	p3: +5192 $\pm$ 3245
	p4: +2406 $\pm$ 1850	p4: +3653 $\pm$ 2075	p4: +4087 $\pm$ 1839	p4: +1186 $\pm$ 157	p4: +3074 $\pm$ 1426	p4: +4304 $\pm$ 4412
16s delay	total: +1829 $\pm$ 735	total: +2984 $\pm$ 1339	total: +3857 $\pm$ 2400	total: +1160 $\pm$ 301	total: +2296 $\pm$ 571	total: +4824 $\pm$ 4532
	p1: +933 $\pm$ 365	p1: +1863 $\pm$ 2223	p1: +1816 $\pm$ 5070	p1: +353 $\pm$ 171	p1: +794 $\pm$ 862	p1: -267 $\pm$ 2083
	p2: +2027 $\pm$ 846	p2: +3267 $\pm$ 1426	p2: +3889 $\pm$ 1930	p2: +1458 $\pm$ 481	p2: +3163 $\pm$ 1481	p2: +6418 $\pm$ 5580
	p3: +2047 $\pm$ 627	p3: +3875 $\pm$ 3155	p3: +4236 $\pm$ 2249	p3: +1701 $\pm$ 860	p3: +2880 $\pm$ 1074	p3: +8910 $\pm$ 9426
Stable-queue (non-blocking)	p4: +2714 $\pm$ 1849	p4: +3149 $\pm$ 1373	p4: +5828 $\pm$ 3692	p4: +1408 $\pm$ 408	p4: +2767 $\pm$ 1599	p4: +4899 $\pm$ 6040
	total: +1594 $\pm$ 391	total: +3454 $\pm$ 1378	total: +4877 $\pm$ 2715	total: +1272 $\pm$ 297	total: +2292 $\pm$ 649	total: +2849 $\pm$ 2079
	p1: +696 $\pm$ 278	p1: +2206 $\pm$ 2342	p1: +2745 $\pm$ 6629	p1: +541 $\pm$ 340	p1: +932 $\pm$ 954	p1: +26 $\pm$ 2960
	p2: +1968 $\pm$ 803	p2: +3952 $\pm$ 1739	p2: +5358 $\pm$ 2301	p2: +1608 $\pm$ 388	p2: +3302 $\pm$ 1626	p2: +4036 $\pm$ 2346
16s delay	p3: +2119 $\pm$ 808	p3: +3491 $\pm$ 1818	p3: +5823 $\pm$ 3723	p3: +1645 $\pm$ 735	p3: +3082 $\pm$ 856	p3: +3933 $\pm$ 1580
	p4: +1751 $\pm$ 648	p4: +4632 $\pm$ 2742	p4: +5391 $\pm$ 2483	p4: +1556 $\pm$ 330	p4: +2042 $\pm$ 1101	p4: +4027 $\pm$ 3859

Table 19. Mean paired differences in number of pod deletions between the plugin with DefaultPreemption disabled and the default scheduler for runs with one priority (mean $\pm$ std).

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	-420.4 $\pm$ 95.7	-103.4 $\pm$ 56.2	-0.8 $\pm$ 30.3	-549.6 $\pm$ 121.6	-213.0 $\pm$ 71.6	-66.6 $\pm$ 55.0
Scheduling-failure (non-blocking)	-401.2 $\pm$ 84.5	-102.4 $\pm$ 73.8	-0.8 $\pm$ 28.1	-518.8 $\pm$ 151.1	-202.8 $\pm$ 97.7	-86.6 $\pm$ 57.4
Periodic (blocking), 4s interval	+53.8 $\pm$ 68.0	+69.0 $\pm$ 38.5	+60.2 $\pm$ 22.7	+96.4 $\pm$ 116.3	+60.2 $\pm$ 24.3	+30.6 $\pm$ 24.0
Periodic (non-blocking), 4s interval	+43.0 $\pm$ 61.9	+85.0 $\pm$ 28.2	+42.8 $\pm$ 22.7	+101.2 $\pm$ 78.6	+64.4 $\pm$ 31.9	+32.8 $\pm$ 20.2
Periodic (blocking), 8s interval	-37.2 $\pm$ 97.9	+15.4 $\pm$ 58.3	+13.2 $\pm$ 19.7	+58.4 $\pm$ 92.5	+36.6 $\pm$ 29.7	+17.6 $\pm$ 21.1
Periodic (non-blocking), 8s interval	-12.8 $\pm$ 66.8	+21.0 $\pm$ 40.0	+18.8 $\pm$ 23.4	+69.4 $\pm$ 61.7	+34.2 $\pm$ 36.1	+6.8 $\pm$ 34.6
Periodic (blocking), 16s interval	-14.8 $\pm$ 62.8	-1.8 $\pm$ 54.8	+4.8 $\pm$ 23.8	+39.4 $\pm$ 70.5	+28.2 $\pm$ 34.6	+13.4 $\pm$ 8.8
Periodic (non-blocking), 16s interval	-30.0 $\pm$ 64.7	+11.8 $\pm$ 38.9	+2.6 $\pm$ 28.9	+42.6 $\pm$ 44.1	+10.6 $\pm$ 38.0	+12.8 $\pm$ 16.1
Stable-queue (blocking), 4s delay	+35.4 $\pm$ 70.2	+64.6 $\pm$ 49.1	+29.8 $\pm$ 4.7	+56.4 $\pm$ 70.4	+48.4 $\pm$ 39.1	+10.8 $\pm$ 29.2
Stable-queue (non-blocking), 4s delay	+39.0 $\pm$ 73.7	+45.0 $\pm$ 54.7	+28.6 $\pm$ 25.3	+63.6 $\pm$ 67.1	+46.8 $\pm$ 48.5	+23.2 $\pm$ 25.6
Stable-queue (blocking), 8s delay	+10.8 $\pm$ 53.5	+12.6 $\pm$ 43.8	+15.2 $\pm$ 16.3	+28.0 $\pm$ 85.4	+37.2 $\pm$ 32.5	+22.8 $\pm$ 18.8
Stable-queue (non-blocking), 8s delay	-5.4 $\pm$ 71.3	+23.8 $\pm$ 58.3	+14.8 $\pm$ 18.4	+33.8 $\pm$ 59.1	+23.8 $\pm$ 34.2	+6.6 $\pm$ 23.5
Stable-queue (blocking), 16s delay	-63.4 $\pm$ 81.8	-9.2 $\pm$ 52.2	-14.8 $\pm$ 18.4	+5.6 $\pm$ 64.6	+23.4 $\pm$ 41.2	-0.6 $\pm$ 23.1
Stable-queue (non-blocking), 16s delay	-28.6 $\pm$ 66.0	-7.4 $\pm$ 52.4	-15.0 $\pm$ 28.9	+15.0 $\pm$ 77.2	+15.0 $\pm$ 31.2	-0.4 $\pm$ 31.3

Table 20. Mean paired differences in number of pod deletions between the plugin with DefaultPreemption disabled and the default scheduler for runs with 4 priorities (mean $\pm$ std). p1 is the lowest priority.

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	total: -1352.8 $\pm$ 229.0 p1: -553.6 $\pm$ 220.5 p2: -357.4 $\pm$ 20.7 p3: -246.4 $\pm$ 38.0 p4: -195.4 $\pm$ 7.3	total: -539.0 $\pm$ 186.0 p1: -189.8 $\pm$ 164.8 p2: -149.0 $\pm$ 19.4 p3: -117.4 $\pm$ 21.6 p4: -82.8 $\pm$ 10.3	total: -210.4 $\pm$ 94.3 p1: -48.2 $\pm$ 75.2 p2: -71.8 $\pm$ 8.4 p3: -54.0 $\pm$ 26.1 p4: -36.4 $\pm$ 9.0	total: -993.4 $\pm$ 194.8 p1: -372.4 $\pm$ 111.1 p2: -284.6 $\pm$ 58.9 p3: -194.8 $\pm$ 41.8 p4: -141.6 $\pm$ 13.4	total: -468.4 $\pm$ 106.0 p1: -188.2 $\pm$ 64.6 p2: -118.8 $\pm$ 30.7 p3: -93.8 $\pm$ 16.6 p4: -67.6 $\pm$ 13.1	total: -162.2 $\pm$ 61.6 p1: -49.6 $\pm$ 47.4 p2: -49.2 $\pm$ 15.0 p3: -38.2 $\pm$ 10.3 p4: -25.2 $\pm$ 7.5
Scheduling-failure (non-blocking)	total: -1352.8 $\pm$ 224.5 p1: -549.4 $\pm$ 208.7 p2: -358.4 $\pm$ 12.8 p3: -247.0 $\pm$ 32.0 p4: -198.0 $\pm$ 11.7	total: -522.2 $\pm$ 156.8 p1: -176.6 $\pm$ 148.0 p2: -147.2 $\pm$ 15.1 p3: -116.8 $\pm$ 22.3 p4: -81.6 $\pm$ 12.8	total: -208.6 $\pm$ 73.5 p1: -44.0 $\pm$ 55.0 p2: -70.8 $\pm$ 11.1 p3: -57.0 $\pm$ 22.9 p4: -36.8 $\pm$ 6.3	total: -1026.8 $\pm$ 167.1 p1: -391.4 $\pm$ 96.9 p2: -283.0 $\pm$ 52.1 p3: -207.4 $\pm$ 37.0 p4: -145.0 $\pm$ 19.1	total: -474.0 $\pm$ 100.4 p1: -193.4 $\pm$ 70.7 p2: -113.8 $\pm$ 25.0 p3: -96.0 $\pm$ 17.6 p4: -70.8 $\pm$ 12.0	total: -175.0 $\pm$ 66.4 p1: -55.8 $\pm$ 48.9 p2: -50.4 $\pm$ 13.6 p3: -41.6 $\pm$ 11.9 p4: -27.2 $\pm$ 4.5
Periodic (blocking) 4s interval	total: -1013.0 $\pm$ 200.8 p1: -559.4 $\pm$ 164.4 p2: -195.2 $\pm$ 17.0 p3: -139.8 $\pm$ 31.5 p4: -118.6 $\pm$ 22.7	total: -482.4 $\pm$ 164.0 p1: -250.0 $\pm$ 140.1 p2: -93.0 $\pm$ 16.9 p3: -75.4 $\pm$ 20.6 p4: -64.0 $\pm$ 14.2	total: -194.0 $\pm$ 73.1 p1: -81.0 $\pm$ 45.1 p2: -48.4 $\pm$ 19.1 p3: -39.2 $\pm$ 18.4 p4: -25.4 $\pm$ 9.8	total: -604.2 $\pm$ 112.6 p1: -313.8 $\pm$ 73.9 p2: -129.2 $\pm$ 33.9 p3: -85.4 $\pm$ 21.0 p4: -75.8 $\pm$ 10.4	total: -314.6 $\pm$ 85.5 p1: -162.4 $\pm$ 68.5 p2: -58.4 $\pm$ 16.4 p3: -50.8 $\pm$ 15.1 p4: -43.0 $\pm$ 10.5	total: -108.2 $\pm$ 35.5 p1: -46.4 $\pm$ 25.1 p2: -24.0 $\pm$ 11.7 p3: -21.2 $\pm$ 5.0 p4: -16.6 $\pm$ 5.9
Periodic (non-blocking) 4s interval	total: -1019.6 $\pm$ 192.7 p1: -555.8 $\pm$ 150.9 p2: -197.0 $\pm$ 25.3 p3: -144.0 $\pm$ 37.4 p4: -122.8 $\pm$ 25.0	total: -483.0 $\pm$ 154.6 p1: -245.2 $\pm$ 124.5 p2: -95.0 $\pm$ 14.8 p3: -77.0 $\pm$ 19.7 p4: -65.8 $\pm$ 21.0	total: -193.4 $\pm$ 82.4 p1: -84.4 $\pm$ 44.7 p2: -46.6 $\pm$ 22.0 p3: -38.0 $\pm$ 21.9 p4: -24.4 $\pm$ 10.1	total: -617.0 $\pm$ 105.8 p1: -319.2 $\pm$ 86.5 p2: -129.6 $\pm$ 34.8 p3: -89.4 $\pm$ 18.8 p4: -78.8 $\pm$ 6.7	total: -314.6 $\pm$ 69.1 p1: -167.0 $\pm$ 48.3 p2: -54.4 $\pm$ 15.7 p3: -51.6 $\pm$ 14.3 p4: -41.6 $\pm$ 12.1	total: -114.2 $\pm$ 42.7 p1: -46.0 $\pm$ 25.5 p2: -26.2 $\pm$ 12.0 p3: -23.8 $\pm$ 7.2 p4: -18.2 $\pm$ 5.4
Periodic (blocking) 8s interval	total: -1150.8 $\pm$ 161.8 p1: -674.4 $\pm$ 135.3 p2: -204.6 $\pm$ 17.2 p3: -148.4 $\pm$ 31.2 p4: -123.4 $\pm$ 22.2	total: -559.4 $\pm$ 139.8 p1: -314.8 $\pm$ 123.8 p2: -95.2 $\pm$ 9.4 p3: -83.8 $\pm$ 18.8 p4: -65.6 $\pm$ 17.9	total: -239.0 $\pm$ 59.6 p1: -125.0 $\pm$ 31.3 p2: -45.4 $\pm$ 17.8 p3: -42.0 $\pm$ 18.7 p4: -26.6 $\pm$ 12.7	total: -685.4 $\pm$ 98.9 p1: -380.0 $\pm$ 76.5 p2: -137.0 $\pm$ 35.4 p3: -89.8 $\pm$ 11.2 p4: -78.6 $\pm$ 4.3	total: -355.6 $\pm$ 77.6 p1: -202.4 $\pm$ 62.6 p2: -57.6 $\pm$ 14.0 p3: -49.8 $\pm$ 18.0 p4: -45.8 $\pm$ 9.0	total: -142.8 $\pm$ 36.9 p1: -70.6 $\pm$ 33.0 p2: -28.8 $\pm$ 9.0 p3: -24.2 $\pm$ 6.7 p4: -19.2 $\pm$ 3.9
Periodic (non-blocking) 8s interval	total: -1129.8 $\pm$ 146.2 p1: -662.2 $\pm$ 123.6 p2: -203.4 $\pm$ 19.5 p3: -139.8 $\pm$ 33.4 p4: -124.4 $\pm$ 22.1	total: -558.0 $\pm$ 136.0 p1: -316.4 $\pm$ 120.0 p2: -94.6 $\pm$ 10.6 p3: -80.8 $\pm$ 18.8 p4: -66.2 $\pm$ 17.9	total: -247.2 $\pm$ 73.7 p1: -122.0 $\pm$ 46.4 p2: -52.6 $\pm$ 21.3 p3: -42.8 $\pm$ 19.6 p4: -29.8 $\pm$ 10.7	total: -680.0 $\pm$ 131.7 p1: -377.8 $\pm$ 95.5 p2: -138.6 $\pm$ 39.1 p3: -86.8 $\pm$ 18.1 p4: -76.8 $\pm$ 5.9	total: -364.0 $\pm$ 59.4 p1: -206.8 $\pm$ 47.6 p2: -56.2 $\pm$ 10.9 p3: -55.0 $\pm$ 14.9 p4: -46.0 $\pm$ 12.5	total: -132.6 $\pm$ 36.5 p1: -66.8 $\pm$ 26.3 p2: -23.0 $\pm$ 9.1 p3: -23.2 $\pm$ 4.8 p4: -19.6 $\pm$ 5.3
Periodic (blocking) 16s interval	total: -1153.4 $\pm$ 136.6 p1: -703.8 $\pm$ 98.9 p2: -188.0 $\pm$ 19.8 p3: -142.4 $\pm$ 32.8 p4: -119.2 $\pm$ 30.0	total: -632.2 $\pm$ 138.8 p1: -364.0 $\pm$ 110.2 p2: -109.8 $\pm$ 17.9 p3: -85.4 $\pm$ 23.6 p4: -73.0 $\pm$ 19.7	total: -279.0 $\pm$ 61.6 p1: -154.6 $\pm$ 30.6 p2: -51.6 $\pm$ 16.0 p3: -44.4 $\pm$ 18.3 p4: -28.4 $\pm$ 10.1	total: -771.8 $\pm$ 126.2 p1: -433.6 $\pm$ 86.4 p2: -147.4 $\pm$ 39.6 p3: -97.0 $\pm$ 8.6 p4: -93.8 $\pm$ 10.0	total: -397.2 $\pm$ 74.6 p1: -232.4 $\pm$ 51.1 p2: -61.4 $\pm$ 12.6 p3: -55.6 $\pm$ 15.7 p4: -47.8 $\pm$ 16.4	total: -157.6 $\pm$ 37.6 p1: -83.8 $\pm$ 24.5 p2: -26.8 $\pm$ 9.1 p3: -27.8 $\pm$ 7.3 p4: -19.2 $\pm$ 5.3
Periodic (non-blocking) 16s interval	total: -1200.8 $\pm$ 182.6 p1: -716.2 $\pm$ 128.2 p2: -210.2 $\pm$ 22.0 p3: -146.2 $\pm$ 34.0 p4: -128.2 $\pm$ 25.4	total: -609.6 $\pm$ 149.8 p1: -359.6 $\pm$ 110.4 p2: -101.6 $\pm$ 16.3 p3: -78.8 $\pm$ 19.6 p4: -69.6 $\pm$ 24.1	total: -257.6 $\pm$ 65.3 p1: -140.0 $\pm$ 31.3 p2: -49.2 $\pm$ 15.4 p3: -40.8 $\pm$ 23.1 p4: -27.6 $\pm$ 9.1	total: -741.6 $\pm$ 100.3 p1: -432.8 $\pm$ 72.3 p2: -134.4 $\pm$ 29.1 p3: -91.0 $\pm$ 17.6 p4: -83.4 $\pm$ 7.2	total: -396.6 $\pm$ 80.1 p1: -233.8 $\pm$ 56.3 p2: -59.0 $\pm$ 14.1 p3: -53.8 $\pm$ 17.7 p4: -50.0 $\pm$ 11.0	total: -166.8 $\pm$ 27.1 p1: -92.2 $\pm$ 21.8 p2: -27.8 $\pm$ 6.1 p3: -26.6 $\pm$ 3.6 p4: -20.2 $\pm$ 5.1
Stable-queue (blocking) 4s delay	total: -1040.6 $\pm$ 160.1 p1: -601.4 $\pm$ 137.2 p2: -190.4 $\pm$ 16.3 p3: -133.8 $\pm$ 30.0 p4: -115.0 $\pm$ 16.8	total: -507.0 $\pm$ 141.8 p1: -270.2 $\pm$ 126.0 p2: -92.6 $\pm$ 12.8 p3: -81.0 $\pm$ 18.5 p4: -63.2 $\pm$ 14.6	total: -211.6 $\pm$ 51.6 p1: -96.8 $\pm$ 20.4 p2: -46.6 $\pm$ 16.0 p3: -41.2 $\pm$ 16.8 p4: -27.0 $\pm$ 8.3	total: -681.2 $\pm$ 117.4 p1: -381.2 $\pm$ 83.6 p2: -133.2 $\pm$ 36.2 p3: -88.0 $\pm$ 17.6 p4: -78.8 $\pm$ 7.0	total: -342.0 $\pm$ 64.5 p1: -195.8 $\pm$ 45.4 p2: -53.8 $\pm$ 13.8 p3: -50.0 $\pm$ 15.4 p4: -42.4 $\pm$ 8.5	total: -125.6 $\pm$ 47.6 p1: -59.6 $\pm$ 37.4 p2: -26.0 $\pm$ 8.9 p3: -23.2 $\pm$ 5.5 p4: -16.8 $\pm$ 4.8
Stable-queue (non-blocking) 4s delay	total: -1045.0 $\pm$ 134.2 p1: -604.4 $\pm$ 125.8 p2: -189.2 $\pm$ 11.9 p3: -139.4 $\pm$ 29.6 p4: -112.0 $\pm$ 12.8	total: -511.2 $\pm$ 123.1 p1: -270.4 $\pm$ 111.8 p2: -95.2 $\pm$ 11.2 p3: -79.4 $\pm$ 18.5 p4: -66.2 $\pm$ 15.7	total: -201.0 $\pm$ 56.1 p1: -95.2 $\pm$ 37.2 p2: -42.2 $\pm$ 15.5 p3: -39.4 $\pm$ 12.8 p4: -24.2 $\pm$ 8.7	total: -665.6 $\pm$ 131.7 p1: -375.4 $\pm$ 92.5 p2: -121.8 $\pm$ 36.1 p3: -91.2 $\pm$ 17.6 p4: -77.2 $\pm$ 9.4	total: -341.4 $\pm$ 78.5 p1: -196.0 $\pm$ 57.3 p2: -52.0 $\pm$ 13.4 p3: -49.0 $\pm$ 17.4 p4: -44.4 $\pm$ 10.7	total: -128.2 $\pm$ 43.1 p1: -64.0 $\pm$ 27.9 p2: -24.0 $\pm$ 10.2 p3: -21.6 $\pm$ 4.0 p4: -18.6 $\pm$ 7.5
Stable-queue (blocking) 8s delay	total: -1139.4 $\pm$ 155.7 p1: -666.2 $\pm$ 118.2 p2: -206.0 $\pm$ 22.5 p3: -144.2 $\pm$ 33.4 p4: -123.0 $\pm$ 16.2	total: -563.0 $\pm$ 141.7 p1: -317.6 $\pm$ 116.3 p2: -101.0 $\pm$ 11.2 p3: -80.6 $\pm$ 25.5 p4: -63.8 $\pm$ 19.5	total: -242.4 $\pm$ 53.1 p1: -126.6 $\pm$ 36.0 p2: -48.4 $\pm$ 16.5 p3: -39.8 $\pm$ 12.3 p4: -27.6 $\pm$ 7.6	total: -711.0 $\pm$ 106.6 p1: -401.0 $\pm$ 78.2 p2: -135.4 $\pm$ 35.3 p3: -90.4 $\pm$ 13.4 p4: -84.2 $\pm$ 4.8	total: -371.4 $\pm$ 77.6 p1: -218.4 $\pm$ 58.7 p2: -58.0 $\pm$ 16.6 p3: -51.0 $\pm$ 18.7 p4: -44.0 $\pm$ 8.1	total: -151.0 $\pm$ 45.0 p1: -81.8 $\pm$ 31.7 p2: -26.0 $\pm$ 10.4 p3: -24.6 $\pm$ 5.3 p4: -18.6 $\pm$ 6.8
Stable-queue (non-blocking) 8s delay	total: -1136.6 $\pm$ 149.9 p1: -672.6 $\pm$ 119.4 p2: -199.8 $\pm$ 16.0 p3: -145.8 $\pm$ 29.7 p4: -118.4 $\pm$ 14.9	total: -577.4 $\pm$ 131.8 p1: -326.2 $\pm$ 115.3 p2: -105.0 $\pm$ 14.1 p3: -81.0 $\pm$ 20.2 p4: -65.2 $\pm$ 16.9	total: -245.2 $\pm$ 56.9 p1: -126.2 $\pm$ 33.0 p2: -48.0 $\pm$ 16.0 p3: -42.0 $\pm$ 18.7 p4: -29.0 $\pm$ 9.1	total: -714.8 $\pm$ 118.9 p1: -410.2 $\pm$ 83.7 p2: -134.2 $\pm$ 37.5 p3: -90.2 $\pm$ 14.4 p4: -80.2 $\pm$ 7.5	total: -365.6 $\pm$ 72.7 p1: -216.8 $\pm$ 52.1 p2: -55.0 $\pm$ 10.1 p3: -50.0 $\pm$ 15.9 p4: -43.8 $\pm$ 15.0	total: -139.4 $\pm$ 36.2 p1: -74.2 $\pm$ 19.3 p2: -24.6 $\pm$ 9.3 p3: -22.2 $\pm$ 5.1 p4: -18.4 $\pm$ 6.3
Stable-queue (blocking) 16s delay	total: -1195.4 $\pm$ 151.0 p1: -719.0 $\pm$ 109.2 p2: -203.0 $\pm$ 12.9 p3: -148.2 $\pm$ 28.8 p4: -125.2 $\pm$ 24.8	total: -897.8 $\pm$ 572.0 p1: -460.4 $\pm$ 211.7 p2: -172.0 $\pm$ 139.0 p3: -151.0 $\pm$ 142.0 p4: -114.4 $\pm$ 93.9	total: -268.4 $\pm$ 64.5 p1: -152.0 $\pm$ 35.4 p2: -48.6 $\pm$ 17.5 p3: -42.0 $\pm$ 15.4 p4: -25.8 $\pm$ 11.2	total: -760.2 $\pm$ 126.0 p1: -446.4 $\pm$ 78.8 p2: -138.6 $\pm$ 42.2 p3: -91.6 $\pm$ 11.8 p4: -83.6 $\pm$ 8.8	total: -411.8 $\pm$ 81.9 p1: -240.0 $\pm$ 59.5 p2: -64.2 $\pm$ 14.5 p3: -57.0 $\pm$ 16.5 p4: -50.6 $\pm$ 13.2	total: -152.2 $\pm$ 37.3 p1: -88.4 $\pm$ 29.2 p2: -23.2 $\pm$ 5.7 p3: -22.2 $\pm$ 8.9 p4: -18.4 $\pm$ 5.5
Stable-queue (non-blocking) 16s delay	total: -1176.8 $\pm$ 171.5 p1: -715.4 $\pm$ 124.9 p2: -197.4 $\pm$ 20.5 p3: -139.2 $\pm$ 23.4 p4: -124.8 $\pm$ 23.8	total: -622.4 $\pm$ 143.6 p1: -365.8 $\pm$ 107.0 p2: -102.0 $\pm$ 15.7 p3: -83.6 $\pm$ 19.6 p4: -71.0 $\pm$ 22.8	total: -273.8 $\pm$ 60.2 p1: -147.0 $\pm$ 31.2 p2: -53.0 $\pm$ 16.3 p3: -43.8 $\pm$ 19.8 p4: -30.0 $\pm$ 11.5	total: -769.0 $\pm$ 90.2 p1: -450.4 $\pm$ 67.3 p2: -137.0 $\pm$ 29.4 p3: -94.8 $\pm$ 8.0 p4: -86.8 $\pm$ 5.8	total: -402.8 $\pm$ 99.9 p1: -236.8 $\pm$ 73.6 p2: -62.6 $\pm$ 16.8 p3: -54.2 $\pm$ 9.9 p4: -49.2 $\pm$ 11.3	total: -149.2 $\pm$ 30.5 p1: -72.0 $\pm$ 15.9 p2: -29.2 $\pm$ 7.5 p3: -27.2 $\pm$ 5.7 p4: -20.8 $\pm$ 7.4

Table 21. Mean paired differences in number of solver runs between the plugin with DefaultPreemption disabled and the default scheduler for runs with one priority (mean $\pm$ std).

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	218 $\pm$ 9.9	154 $\pm$ 17.3	111 $\pm$ 30.7	195 $\pm$ 26.0	145 $\pm$ 24.8	102 $\pm$ 30.6
Scheduling-failure (non-blocking)	223 $\pm$ 11.7	152 $\pm$ 19.7	111 $\pm$ 27.3	182 $\pm$ 19.2	149 $\pm$ 33.8	111 $\pm$ 40.2
Periodic (blocking), 4s interval	84 $\pm$ 17.6	77 $\pm$ 15.9	76 $\pm$ 26.4	100 $\pm$ 20.4	93 $\pm$ 23.0	90 $\pm$ 42.2
Periodic (non-blocking), 4s interval	86 $\pm$ 13.8	80 $\pm$ 15.6	72 $\pm$ 24.3	96 $\pm$ 18.0	99 $\pm$ 31.2	93 $\pm$ 39.2
Periodic (blocking), 8s interval	46 $\pm$ 8.7	46 $\pm$ 10.2	52 $\pm$ 14.8	57 $\pm$ 9.9	68 $\pm$ 16.3	66 $\pm$ 30.5
Periodic (non-blocking), 8s interval	47 $\pm$ 5.6	45 $\pm$ 8.9	49 $\pm$ 12.9	60 $\pm$ 11.7	68 $\pm$ 19.0	63 $\pm$ 23.0
Periodic (blocking), 16s interval	23 $\pm$ 2.6	25 $\pm$ 6.1	30 $\pm$ 8.7	33 $\pm$ 4.3	46 $\pm$ 15.6	43 $\pm$ 17.0
Periodic (non-blocking), 16s interval	24 $\pm$ 1.6	28 $\pm$ 6.5	27 $\pm$ 7.6	35 $\pm$ 10.1	44 $\pm$ 16.6	48 $\pm$ 17.9
Stable-queue (blocking), 4s delay	130 $\pm$ 18.6	112 $\pm$ 25.6	103 $\pm$ 41.7	170 $\pm$ 38.9	151 $\pm$ 51.4	116 $\pm$ 45.3
Stable-queue (non-blocking), 4s delay	128 $\pm$ 26.6	112 $\pm$ 25.2	109 $\pm$ 43.1	178 $\pm$ 40.8	169 $\pm$ 60.8	118 $\pm$ 39.5
Stable-queue (blocking), 8s delay	68 $\pm$ 12.5	61 $\pm$ 12.4	76 $\pm$ 35.0	97 $\pm$ 25.6	118 $\pm$ 34.7	96 $\pm$ 38.7
Stable-queue (non-blocking), 8s delay	67 $\pm$ 10.5	65 $\pm$ 16.4	78 $\pm$ 38.3	95 $\pm$ 22.0	107 $\pm$ 39.1	109 $\pm$ 63.7
Stable-queue (blocking), 16s delay	31 $\pm$ 4.1	30 $\pm$ 9.3	43 $\pm$ 15.6	51 $\pm$ 8.0	70 $\pm$ 18.9	66 $\pm$ 31.6
Stable-queue (non-blocking), 16s delay	30 $\pm$ 5.7	29 $\pm$ 7.7	39 $\pm$ 15.0	51 $\pm$ 11.9	58 $\pm$ 19.3	67 $\pm$ 34.2

Table 22. Mean paired differences in number of solver runs between the plugin with DefaultPreemption disabled and the default scheduler for runs with 4 priorities (mean $\pm$ std). p1 is the lowest priority.

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	268 $\pm$ 63.3	189 $\pm$ 28.3	117 $\pm$ 18.6	191 $\pm$ 12.5	129 $\pm$ 7.5	84 $\pm$ 20.5
Scheduling-failure (non-blocking)	273 $\pm$ 64.2	180 $\pm$ 32.7	117 $\pm$ 21.4	186 $\pm$ 12.2	127 $\pm$ 6.3	86 $\pm$ 20.7
Periodic (blocking), 4s interval	107 $\pm$ 31.9	90 $\pm$ 15.4	73 $\pm$ 25.3	81 $\pm$ 15.5	76 $\pm$ 4.0	53 $\pm$ 17.2
Periodic (non-blocking), 4s interval	98 $\pm$ 16.0	87 $\pm$ 14.2	85 $\pm$ 39.7	85 $\pm$ 9.8	73 $\pm$ 11.7	60 $\pm$ 18.6
Periodic (blocking), 8s interval	51 $\pm$ 6.1	53 $\pm$ 12.2	46 $\pm$ 8.8	53 $\pm$ 9.8	56 $\pm$ 4.4	46 $\pm$ 18.4
Periodic (non-blocking), 8s interval	48 $\pm$ 10.2	57 $\pm$ 20.8	47 $\pm$ 11.5	55 $\pm$ 7.9	51 $\pm$ 4.1	45 $\pm$ 14.8
Periodic (blocking), 16s interval	22 $\pm$ 5.4	26 $\pm$ 4.6	26 $\pm$ 4.2	31 $\pm$ 6.5	30 $\pm$ 6.0	29 $\pm$ 8.9
Periodic (non-blocking), 16s interval	22 $\pm$ 3.4	27 $\pm$ 5.2	29 $\pm$ 1.8	28 $\pm$ 3.9	32 $\pm$ 5.1	32 $\pm$ 16.3
Stable-queue (blocking), 4s delay	130 $\pm$ 12.2	211 $\pm$ 187.6	105 $\pm$ 52.6	133 $\pm$ 9.5	114 $\pm$ 1.8	86 $\pm$ 20.4
Stable-queue (non-blocking), 4s delay	149 $\pm$ 34.5	128 $\pm$ 26.0	106 $\pm$ 54.9	136 $\pm$ 16.9	118 $\pm$ 8.1	88 $\pm$ 25.4
Stable-queue (blocking), 8s delay	86 $\pm$ 50.4	116 $\pm$ 111.6	74 $\pm$ 30.0	74 $\pm$ 11.3	67 $\pm$ 5.8	60 $\pm$ 17.3
Stable-queue (non-blocking), 8s delay	65 $\pm$ 6.7	78 $\pm$ 30.0	82 $\pm$ 42.4	69 $\pm$ 11.8	69 $\pm$ 6.6	59 $\pm$ 23.8
Stable-queue (blocking), 16s delay	27 $\pm$ 3.3	33 $\pm$ 8.4	37 $\pm$ 13.3	39 $\pm$ 6.9	42 $\pm$ 6.7	47 $\pm$ 25.4
Stable-queue (non-blocking), 16s delay	28 $\pm$ 6.3	39 $\pm$ 13.0	46 $\pm$ 13.7	39 $\pm$ 7.5	38 $\pm$ 6.1	34 $\pm$ 10.2

Table 23. Mean paired differences in number of plan activations between the plugin with DefaultPreemption disabled and the default scheduler for runs with one priority (mean $\pm$ std).

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	136 $\pm$ 22.7	91 $\pm$ 18.4	62 $\pm$ 16.9	106 $\pm$ 12.0	55 $\pm$ 12.2	37 $\pm$ 4.6
Scheduling-failure (non-blocking)	140 $\pm$ 31.8	89 $\pm$ 19.9	60 $\pm$ 16.1	95 $\pm$ 18.6	54 $\pm$ 11.2	37 $\pm$ 6.0
Periodic (blocking), 4s interval	53 $\pm$ 2.1	45 $\pm$ 7.6	37 $\pm$ 9.9	52 $\pm$ 11.9	35 $\pm$ 4.6	23 $\pm$ 6.5
Periodic (non-blocking), 4s interval	50 $\pm$ 6.3	47 $\pm$ 2.8	35 $\pm$ 9.5	52 $\pm$ 11.7	34 $\pm$ 5.4	22 $\pm$ 6.6
Periodic (blocking), 8s interval	30 $\pm$ 6.2	28 $\pm$ 2.2	27 $\pm$ 4.6	35 $\pm$ 5.5	24 $\pm$ 8.5	19 $\pm$ 6.0
Periodic (non-blocking), 8s interval	33 $\pm$ 8.2	28 $\pm$ 2.7	26 $\pm$ 4.3	36 $\pm$ 11.1	23 $\pm$ 4.8	20 $\pm$ 4.4
Periodic (blocking), 16s interval	18 $\pm$ 1.7	18 $\pm$ 4.9	17 $\pm$ 4.0	22 $\pm$ 5.6	16 $\pm$ 4.2	14 $\pm$ 3.3
Periodic (non-blocking), 16s interval	17 $\pm$ 5.1	18 $\pm$ 2.9	15 $\pm$ 1.6	21 $\pm$ 4.9	17 $\pm$ 5.4	14 $\pm$ 4.3
Stable-queue (blocking), 4s delay	45 $\pm$ 5.0	38 $\pm$ 2.4	32 $\pm$ 4.9	32 $\pm$ 7.2	26 $\pm$ 4.1	19 $\pm$ 3.1
Stable-queue (non-blocking), 4s delay	41 $\pm$ 5.2	38 $\pm$ 2.0	32 $\pm$ 7.9	36 $\pm$ 8.4	23 $\pm$ 6.8	20 $\pm$ 5.8
Stable-queue (blocking), 8s delay	29 $\pm$ 5.5	24 $\pm$ 5.2	26 $\pm$ 4.3	24 $\pm$ 5.7	18 $\pm$ 6.9	16 $\pm$ 6.4
Stable-queue (non-blocking), 8s delay	28 $\pm$ 4.4	29 $\pm$ 5.2	25 $\pm$ 5.5	26 $\pm$ 4.7	20 $\pm$ 6.1	14 $\pm$ 4.5
Stable-queue (blocking), 16s delay	18 $\pm$ 4.5	18 $\pm$ 5.3	16 $\pm$ 1.9	16 $\pm$ 5.7	15 $\pm$ 5.1	13 $\pm$ 2.7
Stable-queue (non-blocking), 16s delay	16 $\pm$ 2.7	17 $\pm$ 4.2	15 $\pm$ 4.1	17 $\pm$ 4.4	14 $\pm$ 3.2	12 $\pm$ 2.6

Table 24. Mean paired differences in number of plan activations between the plugin with DefaultPreemption disabled and the default scheduler for runs with 4 priorities (mean $\pm$ std). p1 is the lowest priority.

Trigger Mode	# Nodes = 16			# Nodes = 32		
	Inter-arrival = 4s	8s	16s	4s	8s	16s
Scheduling-failure (blocking)	169 $\pm$ 41.8	116 $\pm$ 26.3	67 $\pm$ 16.8	139 $\pm$ 6.3	79 $\pm$ 12.2	48 $\pm$ 6.8
Scheduling-failure (non-blocking)	173 $\pm$ 44.2	114 $\pm$ 23.9	68 $\pm$ 13.5	135 $\pm$ 5.2	74 $\pm$ 13.9	48 $\pm$ 10.5
Periodic (blocking), 4s interval	69 $\pm$ 14.4	54 $\pm$ 11.1	39 $\pm$ 7.3	64 $\pm$ 11.3	41 $\pm$ 8.1	30 $\pm$ 4.6
Periodic (non-blocking), 4s interval	68 $\pm$ 11.6	53 $\pm$ 7.1	39 $\pm$ 7.0	65 $\pm$ 6.3	41 $\pm$ 8.6	32 $\pm$ 3.2
Periodic (blocking), 8s interval	36 $\pm$ 10.6	36 $\pm$ 10.6	26 $\pm$ 3.8	45 $\pm$ 5.4	32 $\pm$ 4.9	23 $\pm$ 2.6
Periodic (non-blocking), 8s interval	35 $\pm$ 11.5	35 $\pm$ 9.6	29 $\pm$ 7.3	44 $\pm$ 3.5	32 $\pm$ 6.6	24 $\pm$ 2.1
Periodic (blocking), 16s interval	18 $\pm$ 5.9	20 $\pm$ 5.6	18 $\pm$ 3.5	27 $\pm$ 3.5	22 $\pm$ 6.5	18 $\pm$ 3.1
Periodic (non-blocking), 16s interval	18 $\pm$ 3.7	20 $\pm$ 5.1	20 $\pm$ 3.9	25 $\pm$ 3.2	23 $\pm$ 7.6	18 $\pm$ 4.7
Stable-queue (blocking), 4s delay	53 $\pm$ 10.8	51 $\pm$ 9.0	37 $\pm$ 8.0	46 $\pm$ 6.3	31 $\pm$ 7.9	27 $\pm$ 3.7
Stable-queue (non-blocking), 4s delay	55 $\pm$ 10.5	49 $\pm$ 7.6	33 $\pm$ 8.7	48 $\pm$ 3.7	34 $\pm$ 5.8	25 $\pm$ 3.6
Stable-queue (blocking), 8s delay	35 $\pm$ 7.5	34 $\pm$ 6.2	28 $\pm$ 4.3	38 $\pm$ 3.7	27 $\pm$ 4.1	22 $\pm$ 2.6
Stable-queue (non-blocking), 8s delay	34 $\pm$ 9.0	32 $\pm$ 6.6	27 $\pm$ 4.5	35 $\pm$ 3.3	28 $\pm$ 7.7	21 $\pm$ 2.6
Stable-queue (blocking), 16s delay	18 $\pm$ 4.4	19 $\pm$ 3.4	19 $\pm$ 2.1	21 $\pm$ 4.5	22 $\pm$ 4.8	17 $\pm$ 2.4
Stable-queue (non-blocking), 16s delay	17 $\pm$ 4.6	21 $\pm$ 5.8	20 $\pm$ 3.4	23 $\pm$ 4.0	22 $\pm$ 3.3	20 $\pm$ 7.0

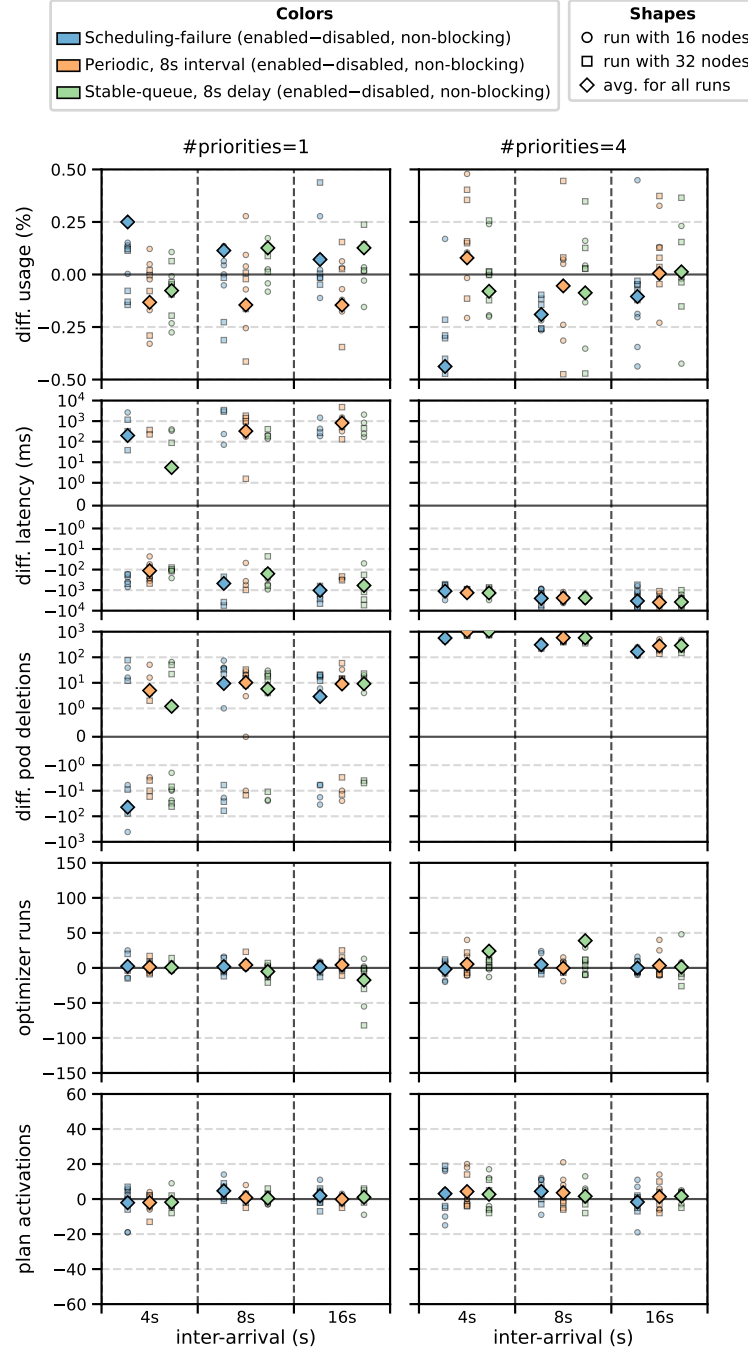


Figure 14. Comparison of OPSche modalities (*DefaultPreemption* enabled vs. disabled) with *non-blocking* variants. Each point shows the difference (*DefaultPreemption* enabled minus *DefaultPreemption* disabled) for a single run (circles: 16 nodes; squares: 32 nodes), and diamonds denote the mean difference per trace setting (mean across five seeds). Left column: one priority level; right column: four priority levels. Rows report effective usage (%), scheduling latency (ms), pod deletions, solver runs, and plan activations.

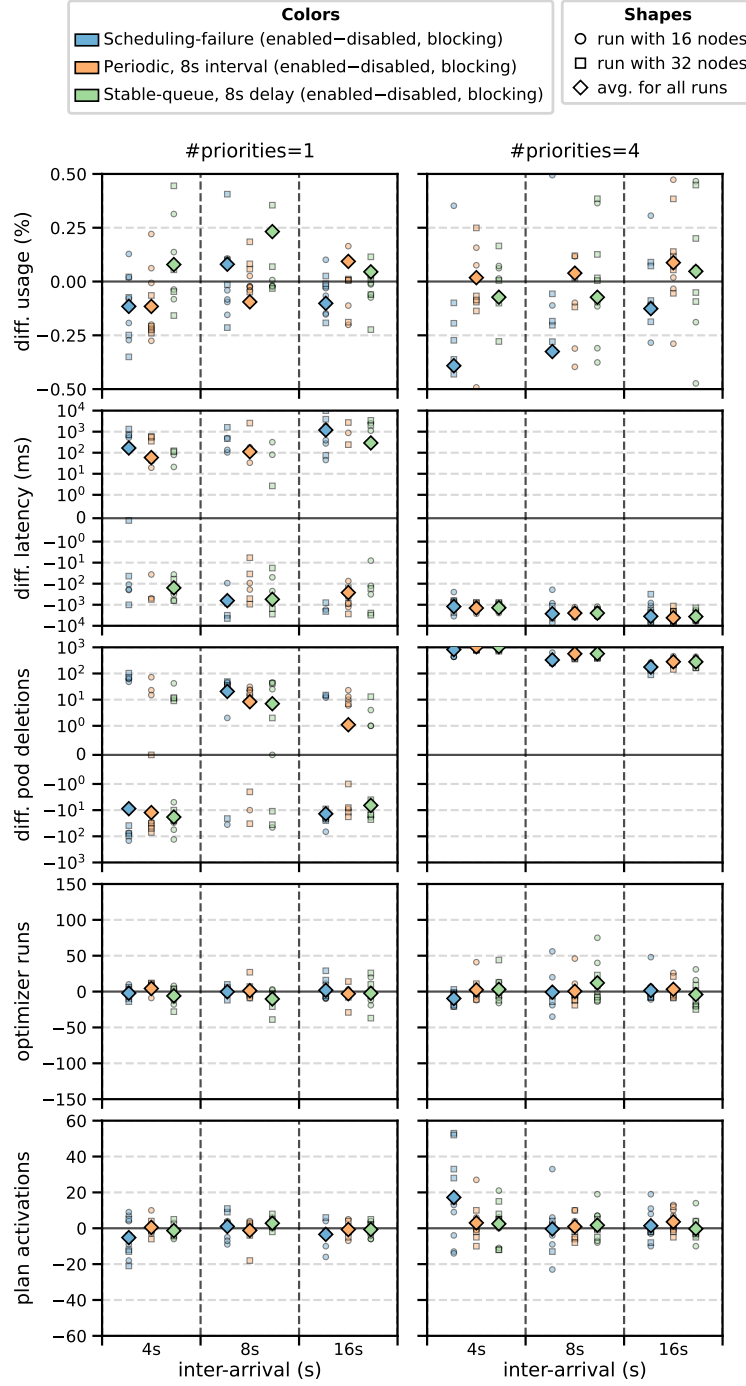


Figure 15. Comparison of OPSche modalities (*DefaultPreemption* enabled vs. disabled) with *blocking* variants. Each point shows the difference (*DefaultPreemption* enabled minus *DefaultPreemption* disabled) for a single run (circles: 16 nodes; squares: 32 nodes), and diamonds denote the mean difference per trace setting (mean across five seeds). Left column: one priority level; right column: four priority levels. Rows report effective usage (%), scheduling latency (ms), pod deletions, solver runs, and plan activations.